THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms **SHAHEEN FATHIMA S B (20524UCSC039),** a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course In-charge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Mini Project**		
	Annexures: • **Khan Academy** • **Hacker Rank** • **Code Chef**		

EX. NO:01 CREATION OF A DDL OPERATIONS USING EMPLOYEE TABLE

DATE:

Aim

To create an Employee table, insert employee records, display all records, and retrieve employees joined from the year 2020 onwards.

Software Used

Oracle /MySQL/sqlite3

Description

DDL (Data Definition Language), it is used to create and modify the structure of database objects in database

DDL COMMANDS:

- Create
- Alter
- Drop
- Truncate

CREATE:

Create command is used to create a new table and also use constraints.

Syntax

```
CREATE TABLE table_name (
   column1 datatype constraints,
   column2 datatype constraints,
   column3 datatype constraints
);
```

Table Structure

```
SQL> CREATE TABLE employee (
        id INTEGER PRIMARY KEY,
        name TEXT,
        joining_year INTEGER
       );
```

Table Created

ALTER:

Alter command is used to alter a table or make some changes in a table. It has 3 methods such as ADD, MODIFY, DROP

ADD:

Add command is used to insert a new attribute/column with data type.

Syntax

```
ALTER TABLE table_name ADD column_name datatype;
```

Query

```
SQL> ALTER TABLE employee ADD salary INTEGER;
```

Table Altered

MODIFY:

Modify command is used to modify or change the data type and to increase the data size in the table not the attribute/column.

Syntax

ALTER TABLE table_name MODIFY column_name datatype;

Query

SQL>ALTER TABLE employee MODIFY name VARCHAR(150);

ALTER TABLE employee ALTER COLUMN name VARCHAR(150);

DROP:

Drop command is used to delete the attribute/column in a table.

Syntax

ALTER TABLE table_name DROP COLUMN column_name;

Query

SQL> ALTER TABLE employee DROP COLUMN salary;

RENAME:

Rename command is used to re-change the table name.

SQL> RENAME TABLE employee TO employees;

Table renamed.

(OR)

ALTER TABLE employee RENAME TO employees;

Column Rename

ALTER TABLE employee RENAME COLUMN name TO employee_name;

TRUNCATE:

Truncate command is used to delete a value in the table.

Syntax

TRUNCATE TABLE tablename;

Query

TRUNCATE TABLE employee;

DROP:

Deletes the table and all its data permanently.

Syntax:DROP TABLE tablename;

Query

DROP TABLE employee;

Explanation

• Create a table named employee.
• Insert employee details into the table.
• Display all employee records.
• Retrieve employees joined in or after the year 2020 and sort them by joining year.

Create Table

```
CREATE TABLE employee (
    id INTEGER PRIMARY KEY,
    name TEXT,
    joining_year INTEGER
```

Sample Data

INSERT INTO employee VALUES (1, 'Arun', 2021);

INSERT INTO employee VALUES (2, 'Meena', 2019);

INSERT INTO employee VALUES (3, 'Kavin', 2022);

INSERT INTO employee VALUES (4, 'Divya', 2020);

INSERT INTO employee VALUES (5, 'Rahul', 2018);

INSERT INTO employee VALUES (6, 'Priya', 2023);

SQL Query

Query 1: Display all employee records

SQL> SELECT * FROM employee;

Resultant Query

id	Name	joining_year
1	Arun	2021
2	Meena	2019
3	Kavin	2022
4	Divya	2020
5	Rahul	2018
6	Priya	2023

Query 2: Display employees joined from the year 2020 onwards

SQL> SELECT *

FROM employee

WHERE joining_year >= 2020

ORDER BY joining_year;

Resultant Query

id	name	joining_year
4	Divya	2020
1	Arun	2021
3	Kavin	2022
6	Priya	2023

RESULT:

Thus, the Employee table was created successfully, records were inserted, all employee details were displayed, and the employees joined from the year 2020 onwards were retrieved successfully.

EX. NO:02 CREATION OF DML OPERATIONS USING PRODUC TABLE

DATE:

Aim

To create a Product table, insert product records, display all records, update product details, delete a product record, and retrieve products based on their price.

Software Used

Oracle / MySQL / sqlite3

Description

DML (Data Manipulation Language) is used to insert, modify, delete, and retrieve data stored in database tables.

DML COMMANDS:

•INSERT
•UPDATE
•DELETE
• SELECT

INSERT:

INSERT command is used to add new records into a table.

Syntax

INSERT INTO table_name (column1, column2, column3)

VALUES (value1, value2, value3);

Query

SQL> INSERT INTO product VALUES (1, 'Laptop', 55000);

1 row inserted.

UPDATE:

UPDATE command is used to modify existing records in a table.

Syntax

UPDATE table_name

SET column_name = value

WHERE condition;

Query

SQL> UPDATE product

SET price = 60000

WHERE id = 1;

1 row updated.

DELETE:

DELETE command is used to remove existing records from a table.

Syntax

DELETE FROM table_name

WHERE condition;

Query

SQL> DELETE FROM product

WHERE id = 5;

1 row deleted.

SELECT:

SELECT command is used to retrieve and display records from a table.

Syntax

SELECT * FROM table_name;

Query

SQL> SELECT * FROM product;

Explanation

•Create a table named product.

• Insert product details into the table.

• Display all product records.

• Update the price of a product.

• Delete a product record.

• Retrieve products whose price is 50000 or above and sort them by price.

Create Table

```
CREATE TABLE product (
   id INTEGER PRIMARY KEY,
   name TEXT,
   price INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO product VALUES (1, 'Laptop', 55000);
INSERT INTO product VALUES (2, 'Mobile', 25000);
```

INSERT INTO product VALUES (3, 'Television', 45000);

INSERT INTO product VALUES (4, 'Refrigerator', 38000);

INSERT INTO product VALUES (5, 'Washing Machine', 42000);

INSERT INTO product VALUES (6, 'Air Conditioner', 60000);

Records Inserted

SQL Query

Query 1: Display all product records

SQL> SELECT * FROM product;

Resultant Query

id	name	price
1	Laptop	55000
2	Mobile	25000
3	Television	45000
4	Refrigerator	38000
5	Washing Machine	42000
6	Air Conditioner	60000

Query 2: Update the price of Laptop

SQL> UPDATE product

SET price = 60000

WHERE id = 1;

1 row updated.

Updated Record

id	name	price
1	Laptop	60000

Query 3: Delete Washing Machine's record

SQL> DELETE FROM product

WHERE id = 5;

1 row deleted.

Remaining Records

id	name	price
1	Laptop	60000
2	Mobile	25000
3	Television	45000
4	Refrigerator	38000
6	Air Conditioner	60000

Query 4: Display products whose price is 50000 or above

SQL> SELECT *

FROM product

WHERE price >= 50000

ORDER BY price;

Resultant Query

id	name	price
1	Laptop	60000
6	Air Conditioner	60000

RESULT

Thus, the Product table was created successfully, records were inserted, product details were displayed, the product price was updated, a product record was deleted, and the products whose price was 50000 or above were retrieved successfully.

EX. NO:03 CREATION OF DQL OPERATIONS USING LIBRARY TABLE

DATE:

Aim

To create a Library table, insert book records, display all records, and retrieve books based on their price.

Software Used

Oracle / MySQL / sqlite3

Description

DQL (Data Query Language) is used to retrieve and display data stored in database tables.

DQL COMMANDS:

• SELECT

SELECT:

SELECT command is used to retrieve and display records from a table.

Syntax

SELECT * FROM table_name;

Query

SQL> SELECT * FROM library;

Explanation

• Create a table named library.

• Insert book details into the table.

• Display all book records.

• Retrieve books whose price is 500 or above and sort them by price.

Create Table

```
CREATE TABLE library (
    id INTEGER PRIMARY KEY,
    book_name TEXT,
    price INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO library VALUES (1, 'The Alchemist', 450);
INSERT INTO library VALUES (2, 'Wings of Fire', 550);
INSERT INTO library VALUES (3, 'Harry Potter', 700);
INSERT INTO library VALUES (4, 'Rich Dad Poor Dad', 600);
INSERT INTO library VALUES (5, 'Atomic Habits', 800);
INSERT INTO library VALUES (6, 'The Hobbit', 650);
```

Records Inserted

SQL Query

Query 1: Display all book records

```
SQL> SELECT * FROM library;
```

Resultant Query

id	book_name	price
1	The Alchemist	450
2	Wings of Fire	550
3	Harry Potter	700
4	Rich Dad Poor Dad	600
5	Atomic Habits	800
6	The Hobbit	650

Query 2: Display books whose price is 500 or above

SQL> SELECT *

FROM library

WHERE price >= 500

ORDER BY price;

Resultant Query

id	book_name	price
2	Wings of Fire	550
4	Rich Dad Poor Dad	600
6	The Hobbit	650
3	Harry Potter	700
5	Atomic Habits	800

RESULT:

Thus, the Library table was created successfully, records were inserted, all book details were displayed, and the books whose price was 500 or above were retrieved successfully.

EX. NO:04 CREATION OF TRANSACTION OPERATIONS USING BANK TABLE

DATE:

Aim

To create a Bank table, insert bank records, perform transaction operations using COMMIT, ROLLBACK, and SAVEPOINT, and verify the changes made to the database.

Software Used

Oracle / MySQL / sqlite3

Description

Transaction Control Language (TCL) is used to manage transactions in a database. It helps to save or undo the changes made to the database.

TRANSACTION COMMANDS:

- COMMIT
- ROLLBACK
- SAVEPOINT

COMMIT:

COMMIT command is used to permanently save all the changes made during a transaction.

Syntax

COMMIT;

Query

SQL> COMMIT;

Transaction committed.

ROLLBACK:

ROLLBACK command is used to undo the changes made during a transaction before the changes are committed.

Syntax

ROLLBACK;

Query

SQL> ROLLBACK;

Transaction rolled back.

SAVEPOINT:

SAVEPOINT command is used to create a temporary point within a transaction. The transaction can be rolled back to that specific point.

Syntax

SAVEPOINT savepoint_name;

Query

SQL> SAVEPOINT sp1;

Savepoint created.

ROLLBACK TO SAVEPOINT

ROLLBACK TO SAVEPOINT is used to undo the changes made after a particular savepoint without undoing the entire transaction.

Syntax

ROLLBACK TO SAVEPOINT savepoint_name;

Query

SQL> ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed.

Explanation

• Create a table named bank.
• Insert bank details into the table.
• Use COMMIT to permanently save the inserted records.
• Update a bank account balance and create a SAVEPOINT.
• Make another change after the savepoint.
• Use ROLLBACK TO SAVEPOINT to undo the changes made after the savepoint.
• Use ROLLBACK to undo uncommitted changes.

Create Table

```
CREATE TABLE bank (
    account_id INTEGER PRIMARY KEY,
    customer_name TEXT,
    balance INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO bank VALUES (201, 'Arun', 25000);
INSERT INTO bank VALUES (202, 'Meena', 30000);
INSERT INTO bank VALUES (203, 'Kavin', 20000);
INSERT INTO bank VALUES (204, 'Divya', 35000);
```

INSERT INTO bank VALUES (205, 'Priya', 40000);

Records Inserted

SQL Query

Query 1: Display all bank records

SQL> SELECT * FROM bank;

Resultant Query

account_id	customer_name	balance
201	Arun	25000
202	Meena	30000
203	Kavin	20000
204	Divya	35000
205	Priya	40000

Query 2: Commit the inserted records

SQL> COMMIT;

Transaction committed.

The inserted bank records are now permanently saved.

Query 3: Update Arun's bank balance

SQL> UPDATE bank

SET balance = 30000

WHERE account_id = 201;

1 row updated.

Updated Record

account_id	customer_name	balance
201	Arun	30000

Query 4: Create a SAVEPOINT

SQL> SAVEPOINT sp1;

Savepoint created.

Query 5: Update Meena's bank balance

SQL> UPDATE bank

SET balance = 35000

WHERE account_id = 202;

1 row updated.

Updated Record

account_id	customer_name	balance
202	Meena	35000

Query 6: Rollback to SAVEPOINT

SQL> ROLLBACK TO SAVEPOINT sp1;

Rollback to savepoint completed.

The changes made after SAVEPOINT sp1 are undone.

Resultant Query

account_id	customer_name	balance
201	Arun	30000
202	Meena	30000

Query 7: Rollback an uncommitted change

SQL> UPDATE bank

SET balance = 15000

WHERE account_id = 203;

1 row updated.

SQL> ROLLBACK;

Transaction rolled back.

The balance of Kavin is restored to its previous value.

Resultant Query

account_id	customer_name	balance
203	Kavin	20000

RESULT:

Thus, the Bank table was created successfully, bank records were inserted, and the transaction operations COMMIT, ROLLBACK, SAVEPOINT, and ROLLBACK TO SAVEPOINT were performed successfully.

EX. NO:05 CREATION OF A JOIN OPERATIONS USING STUDENTS AND COURSES TABLE

DATE:

AIM

To create Students and Courses tables, insert records, and retrieve related data using different types of joins such as Inner Join, Left Join, Right Join, Full Join, and Self Join.

Software Used

Oracle / MySQL / sqlite3

Description

Joins are used to combine records from two or more tables based on a related column between them.

JOIN COMMANDS:

- Inner Join
- Left Join
- Right Join
- Full Join
- Self Join

INNER JOIN:

Inner Join returns only the records that have matching values in both tables.

Syntax

SELECT columns

FROM table1

INNER JOIN table2

ON table1.column = table2.column;

LEFT JOIN:

Left Join returns all records from the left table and the matching records from the right table.

Syntax

SELECT columns

FROM table1

LEFT JOIN table2

ON table1.column = table2.column;

RIGHT JOIN:

Right Join returns all records from the right table and the matching records from the left table.

Syntax

SELECT columns

FROM table1

RIGHT JOIN table2

ON table1.column = table2.column;

FULL JOIN:

Full Join returns all records from both tables. It includes matching and non-matching records.

Syntax

SELECT columns

FROM table1

FULL JOIN table2

ON table1.column = table2.column;

SELF JOIN:

Self Join is used to join a table with itself. It is useful when records in the same table are related to each other.

Syntax

SELECT A.column, B.column

FROM table_name A

JOIN table_name B

ON A.column = B.column;

Explanation

• Create Students and Courses tables.
• Insert student and course details into the tables.
• Display matching student and course records using Inner Join.
• Display all students and their matching courses using Left Join.
• Display all courses and their matching students using Right Join.
• Display all matching and non-matching records using Full Join.
• Use Self Join to display students and their classmates.

Create Tables

```
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY,
    student_name TEXT,
    city TEXT
);
```

```
CREATE TABLE courses (
    course_id INTEGER PRIMARY KEY,
    student_id INTEGER,
    course_name TEXT,
    fees INTEGER
);
```

Tables Created

Sample Data

```
INSERT INTO students VALUES (1, 'Kavin', 'Chennai');
INSERT INTO students VALUES (2, 'Meena', 'Villupuram');
INSERT INTO students VALUES (3, 'Arun', 'Salem');
INSERT INTO students VALUES (4, 'Divya', 'Trichy');
INSERT INTO students VALUES (5, 'Priya', 'Madurai');

INSERT INTO courses VALUES (101, 1, 'Python', 5000);
INSERT INTO courses VALUES (102, 2, 'Java', 6000);
INSERT INTO courses VALUES (103, 1, 'SQL', 4000);
INSERT INTO courses VALUES (104, 3, 'C Programming', 3000);
INSERT INTO courses VALUES (105, 6, 'HTML', 2000);
```

Records Inserted

SQL Query

Query 1: Inner Join

```
SQL> SELECT students.student_name, courses.course_name, courses.fees
FROM students
INNER JOIN courses
ON students.student_id = courses.student_id;
```

Resultant Query

student_name	course_name	fees
Kavin	Python	5000
Meena	Java	6000
Kavin	SQL	4000
Arun	C Programming	3000

Query 2: Left Join

```
SQL> SELECT students.student_name, courses.course_name, courses.fees
FROM students
LEFT JOIN courses
ON students.student_id = courses.student_id;
```

Resultant Query

student_name	course_name	fees
Kavin	Python	5000
Meena	Java	6000
Arun	C Programming	3000
Divya	NULL	NULL
Priya	NULL	NULL
Kavin	SQL	4000

Query 3: Right Join

SQL> SELECT students.student_name, courses.course_name, courses.fees

FROM students

RIGHT JOIN courses

ON students.student_id = courses.student_id;

Resultant Query

student_name	course_name	fees
Kavin	Python	5000
Meena	Java	6000
Kavin	SQL	4000
Arun	C Programming	3000
NULL	HTML	2000

Query 4: Full Join

SQL> SELECT students.student_name, courses.course_name, courses.fees

FROM students

FULL JOIN courses

ON students.student_id = courses.student_id;

Resultant Query

student_name	course_name	fees
Kavin	Python	5000
Meena	Java	6000
Kavin	SQL	4000
Arun	C Programming	3000
Divya	NULL	NULL
Priya	NULL	NULL
NULL	HTML	2000

Query 5: Self Join

Create Student Table

```
CREATE TABLE student_group (
    student_id INTEGER PRIMARY KEY,
    student_name TEXT,
    partner_id INTEGER
);
```

Table Created

Sample Data

INSERT INTO student_group VALUES (1, 'Kavin', NULL);

INSERT INTO student_group VALUES (2, 'Meena', 1);

INSERT INTO student_group VALUES (3, 'Arun', 1);

INSERT INTO student_group VALUES (4, 'Divya', 2);

INSERT INTO student_group VALUES (5, 'Priya', 2);

Records Inserted

SQL> SELECT A.student_name AS Student,

B.student_name AS Partner

FROM student_group A

LEFT JOIN student_group B

ON A.partner_id = B.student_id;

Resultant Query

Student	**Partner**
Kavin	NULL
Meena	Kavin
Arun	Kavin
Divya	Meena
Priya	Meena

RESULT:

Thus, the Students and Courses tables were created successfully, and the required records were retrieved successfully using Inner Join, Left Join, Right Join, Full Join, and Self Join operations.

EX. NO:06 CREATION OF SQL FUNCTIONS USING EMPLOYEE TABLE

DATE:

Aim

To create an Employees table, insert employee records, and retrieve data using different SQL functions such as Aggregate Functions, String Functions, and Date Functions.

Software Used

Oracle / MySQL / sqlite3

Description

SQL Functions are built-in functions used to perform calculations, manipulate text, and work with date values in a database.

SQL FUNCTION TYPES:

- Aggregate Functions
- String Functions
- Date Functions

AGGREGATE FUNCTIONS:

Aggregate functions are used to perform calculations on a group of rows and return a single result.

Common aggregate functions are:

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

Syntax

SELECT aggregate_function(column_name)

FROM table_name;

STRING FUNCTIONS:

String functions are used to perform operations on text or character values.

Common string functions are:

- UPPER()
- LOWER()
- LENGTH()

Syntax

SELECT string_function(column_name)

FROM table_name;

DATE FUNCTIONS

Date functions are used to perform operations on date values.

Common date functions are:

- CURRENT_DATE
- YEAR()
- MONTH()
- DAY()

Syntax

SELECT date_function(date_column)

FROM table_name;

Explanation

- Create a table named employees.
- Insert employee details into the table.
- Display all employee records.
- Calculate the total number of employees using COUNT().
- Calculate the total and average salary using SUM() and AVG().
- Find the highest and lowest salary using MAX() and MIN().
- Display employee names in uppercase and lowercase using UPPER() and LOWER().

- Find the length of employee names using LENGTH().
- Retrieve employee details based on the joining date.

Create Table

```
CREATE TABLE employees (
    employee_id INTEGER PRIMARY KEY,
    employee_name TEXT,
    department TEXT,
    salary INTEGER,
    joining_date DATE
);
```

Table Created

Sample Data

```
INSERT INTO employees VALUES (1, 'Arun', 'HR', 30000, '2026-01-10');
INSERT INTO employees VALUES (2, 'Meena', 'Sales', 35000, '2026-02-15');
INSERT INTO employees VALUES (3, 'Kavin', 'IT', 45000, '2026-03-05');
INSERT INTO employees VALUES (4, 'Divya', 'Finance', 40000, '2026-04-12');
INSERT INTO employees VALUES (5, 'Rahul', 'IT', 50000, '2026-05-20');
INSERT INTO employees VALUES (6, 'Priya', 'HR', 32000, '2026-06-18');
```

Records Inserted

SQL Query

Query 1: Display all employee records

```
SQL> SELECT * FROM employees;
```

Resultant Query

employee_id	employee_name	department	salary	joining_date
1	Arun	HR	30000	2026-01-10
2	Meena	Sales	35000	2026-02-15
3	Kavin	IT	45000	2026-03-05
4	Divya	Finance	40000	2026-04-12
5	Rahul	IT	50000	2026-05-20
6	Priya	HR	32000	2026-06-18

AGGREGATE FUNCTIONS:

Query 2: Count the total number of employees

SQL> SELECT COUNT(*) AS Total_Employees

FROM employees;

Resultant Query

Total_Employees
5

Query 3: Calculate the total salary of all employees

SQL> SELECT SUM(salary) AS Total_Salary

FROM employees;

Resultant Query

Total_Salary
232000

Query 4: Calculate the average salary of employees

SQL> SELECT AVG(salary) AS Average_Salary

FROM employees;

Resultant Query

Average_Salary

38666.67

Query 5: Find the highest and lowest employee salary

SQL> SELECT MAX(salary) AS Highest_Salary,

MIN(salary) AS Lowest_Salary

FROM employees;

Resultant Query

Highest_Salary	Lowest_Salary
50000	30000

STRING FUNCTIONS:

Query 6: Display employee names in uppercase

SQL> SELECT employee_name, UPPER(employee_name) AS Upper_Name

FROM employees;

Resultant Query

employee_name	Upper_Name
Arun	ARUN
Meena	MEENA
Kavin	KAVIN
Divya	DIVYA
Rahul	RAHUL
Priya	PRIYA

Query 7: Display employee names in lowercase

SQL> SELECT employee_name, LOWER(employee_name) AS Lower_Name

FROM employees;

Resultant Query

employee_name	Lower_Name
Arun	arun
Meena	meena
Kavin	kavin
Divya	divya
Rahul	rahul
Priya	priya

Query 8: Display the length of each employee name

SQL> SELECT employee_name, LENGTH(employee_name) AS Name_Length

FROM employees;

Resultant Query

employee_name	Name_Length
Arun	4
Meena	5
Kavin	5
Divya	5
Rahul	5
Priya	5

DATE FUNCTIONS

Query 9: Display the current date

SQL> SELECT CURRENT_DATE AS Current_Date;

Resultant Query

Current_Date

Current system date

Query 10: Display employees joined after March 2026

SQL> SELECT employee_name, joining_date

FROM employees

WHERE joining_date > '2026-03-31';

Resultant Query

employee_name	joining_date
Divya	2026-04-12
Rahul	2026-05-20
Priya	2026-06-18

RESULT:

Thus, the Employees table was created successfully, employee records were inserted, and the required data was retrieved successfully using Aggregate Functions, String Functions, and Date Functions.

EX. NO:07 CREATION OF SUBQUERIES AND NESTED QUERIES USING STUDENTS

DATE:

Aim

To create a Students table, insert student records, and retrieve data using Subqueries and Nested Queries.

Software Used

Oracle / MySQL / sqlite3

Description

A Subquery is a query written inside another SQL query. It is used to retrieve intermediate results and use them in the main query.

A Nested Query is a query placed inside another query to perform operations based on the result of the inner query.

SUBQUERY TYPES:

• Single Row Subquery

• Multiple Row Subquery

• Nested Subquery

SINGLE ROW SUBQUERY:

A Single Row Subquery returns only one value or one row and is used with comparison operators such as =, >, <, >= and <=.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name > (SELECT column_name
                     FROM table_name
                     WHERE condition);
```

MULTIPLE ROW SUBQUERY:

A Multiple Row Subquery returns more than one value or row. It is commonly used with IN, ANY, and ALL operators.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name IN (SELECT column_name
                      FROM table_name
                      WHERE condition);
```

NESTED SUBQUERY:

A Nested Subquery contains one query inside another query. The result of the inner query is used by the outer query.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name = (SELECT column_name
                     FROM table_name
                     WHERE condition);
```

Explanation

• Create a table named students.

• Insert student details into the table.

• Display all student records.

• Find students whose marks are greater than the average marks.

• Find students who belong to the Computer Science or Commerce departments.

• Find the student who has the highest marks.

• Retrieve students based on the result of another query.

Create Table

```
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY,
    student_name TEXT,
    department TEXT,
    marks INTEGER
);
```

Table Created

Sample Data

```
INSERT INTO students VALUES (1, 'Kavin', 'Computer Science', 85);
INSERT INTO students VALUES (2, 'Meena', 'Commerce', 72);
INSERT INTO students VALUES (3, 'Arun', 'Mathematics', 78);
INSERT INTO students VALUES (4, 'Divya', 'Computer Science', 92);
INSERT INTO students VALUES (5, 'Rahul', 'Physics', 65);
INSERT INTO students VALUES (6, 'Priya', 'Commerce', 88);
```

Records Inserted

SQL Query

Query 1: Display all student records

```
SQL> SELECT * FROM students;
```

Resultant Query

student_id	student_name	department	marks
1	Kavin	Computer Science	85
2	Meena	Commerce	72
3	Arun	Mathematics	78
4	Divya	Computer Science	92
5	Rahul	Physics	65
6	Priya	Commerce	88

Query 2: Display students whose marks are greater than the average marks

```
SQL> SELECT student_name, marks
FROM students
WHERE marks > (SELECT AVG(marks)
        FROM students);
```

Resultant Query

student_name	marks
Kavin	85
Divya	92
Priya	88

Query 3: Display students who belong to Computer Science or Commerce department

```
SQL> SELECT student_name, department
FROM students
WHERE department IN (SELECT department
        FROM students
        WHERE department IN ('Computer Science', 'Commerce'));
```

Resultant Query

student_name	department
Kavin	Computer Science
Meena	Commerce
Divya	Computer Science
Priya	Commerce

Query 4: Display the student with the highest marks

```
SQL> SELECT student_name, marks
FROM students
WHERE marks = (SELECT MAX(marks)
        FROM students);
```

Resultant Query

```
student_name marks
Divya        92
```

Query 5: Display students whose marks are greater than Meena's marks

```
SQL> SELECT student_name, marks
FROM students
WHERE marks > (SELECT marks
        FROM students
        WHERE student_name = 'Meena');
```

Resultant Query

student_name	marks
Kavin	85
Arun	78
Divya	92
Priya	88

RESULT:

Thus, the Students table was created successfully, student records were inserted, and the required student details were retrieved successfully using Subqueries and Nested Queries.

EX. NO:08 CREATION OF VIEWS, INDEXES AND SEQUENCES USING STUDENT TABLE

DATE:

Aim

To create and perform operations on Views, Indexes, and Sequences using SQL commands.

Software Used

Oracle SQL / SQL Command Line

Description

View:
A View is a virtual table created from one or more tables using a SELECT query. It does not store the actual data separately.

Index:
An Index is used to improve the speed of data retrieval from a table.

Sequence:
A Sequence is a database object used to generate unique numeric values automatically.

1. VIEW

Definition

A View is a virtual table based on the result of a SQL query.

Syntax

CREATE VIEW view_name AS

SELECT column1, column2

FROM table_name

WHERE condition;

Create Table

CREATE TABLE student_details (

student_id NUMBER PRIMARY KEY,

```
    student_name VARCHAR2(50),
    department VARCHAR2(30),
    marks NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO student_details VALUES (101, 'Kavin', 'Computer Science', 85);
INSERT INTO student_details VALUES (102, 'Meena', 'Commerce', 72);
INSERT INTO student_details VALUES (103, 'Arun', 'Mathematics', 78);
INSERT INTO student_details VALUES (104, 'Divya', 'Computer Science', 92);
INSERT INTO student_details VALUES (105, 'Priya', 'Commerce', 88);
COMMIT;
```

Records Inserted

Create View

```
CREATE VIEW computer_students AS
SELECT student_id, student_name, marks
FROM student_details
WHERE department = 'Computer Science';
```

View Created

Display View

```
SELECT * FROM computer_students;
```

Resultant Query

student_id	student_name	marks
101	Kavin	85
104	Divya	92

Explanation

The computer_students view displays only the students who belong to the Computer Science department.

2. INDEX

Definition

An Index is a database object that improves the speed of searching and retrieving records from a table.

Syntax

CREATE INDEX index_name

ON table_name(column_name);

Create Index

CREATE INDEX idx_student_name

ON student_details(student_name);

Index Created

Use Index in Search

SELECT *

FROM student_details

WHERE student_name = 'Kavin';

Resultant Query

student_id	student_name	department	marks
101	Kavin	Computer Science	85

Explanation

The index idx_student_name is created on the student_name column. It helps improve the performance of searches based on student names.

3. SEQUENCE

Definition

A Sequence is a database object that automatically generates a series of unique numeric values.

Syntax

CREATE SEQUENCE sequence_name

START WITH initial_value

INCREMENT BY increment_value;

Create Sequence

CREATE SEQUENCE student_seq

START WITH 106

INCREMENT BY 1;

Sequence Created

Insert Record Using Sequence

INSERT INTO student_details

VALUES (student_seq.NEXTVAL, 'Rahul', 'Physics', 76);

COMMIT;

Record Inserted

Display Sequence Value

SELECT student_seq.CURRVAL AS current_sequence

FROM dual;

Resultant Query

current_sequence
106

Insert Another Record Using Sequence

INSERT INTO student_details

VALUES (student_seq.NEXTVAL, 'Anitha', 'Chemistry', 81);

COMMIT;

Record Inserted

Display All Student Records

SELECT * FROM student_details

ORDER BY student_id;

Resultant Query

student_id	student_name	department	marks
101	Kavin	Computer Science	85
102	Meena	Commerce	72
103	Arun	Mathematics	78
104	Divya	Computer Science	92
105	Priya	Commerce	88
106	Rahul	Physics	76
107	Anitha	Chemistry	81

Important Commands

Drop View

DROP VIEW computer_students;

View Dropped

Drop Index

DROP INDEX idx_student_name;

Index Dropped

Drop Sequence

DROP SEQUENCE student_seq;

Sequence Dropped

Result:

Thus, the View, Index, and Sequence were created and successfully operated using SQL commands. The View was used to display selected records, the Index was used to improve data retrieval, and the Sequence was used to generate unique numeric values automatically.

EX. NO:09 CREATION OF STORED PROCEDURES AND FUNCTIONS USING EMPLOYEE TABLE

DATE:

Aim

To create and execute Stored Procedures and Functions using PL/SQL.

Software Used

Oracle SQL / SQL Command Line

Description

Stored Procedure:
A Stored Procedure is a named PL/SQL program stored in the database. It is used to perform a specific task and may or may not return a value.

Function:
A Function is a named PL/SQL program that performs a specific operation and always returns a single value.

1. STORED PROCEDURE

Definition

A Stored Procedure is a stored PL/SQL block that performs a particular operation. It can accept input parameters and can modify or display data.

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name

(parameter_name IN datatype)

IS

BEGIN

   statements;

END;

/
```

Create Table

```
CREATE TABLE employee_salary (
    employee_id NUMBER PRIMARY KEY,
    employee_name VARCHAR2(50),
    salary NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO employee_salary VALUES (101, 'Kavin', 45000);
INSERT INTO employee_salary VALUES (102, 'Meena', 32000);
INSERT INTO employee_salary VALUES (103, 'Arun', 50000);
INSERT INTO employee_salary VALUES (104, 'Divya', 38000);
INSERT INTO employee_salary VALUES (105, 'Priya', 55000);
COMMIT;
```

Records Inserted

Display Employee Records

```
SELECT * FROM employee_salary;
```

Resultant Query

employee_id	employee_name	salary
101	Kavin	45000
102	Meena	32000
103	Arun	50000
104	Divya	38000
105	Priya	55000

Create Stored Procedure

The following procedure displays the details of an employee based on the employee ID.

```
CREATE OR REPLACE PROCEDURE display_employee (
    p_employee_id IN NUMBER
)
IS
    v_name employee_salary.employee_name%TYPE;
    v_salary employee_salary.salary%TYPE;
BEGIN
    SELECT employee_name, salary
    INTO v_name, v_salary
    FROM employee_salary
    WHERE employee_id = p_employee_id;

    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);
END;
/
```

Procedure Created

Execute Stored Procedure

```
SET SERVEROUTPUT ON;

EXEC display_employee(103);
```

Resultant Query

Employee Name	Salary
Arun	50000

Output

Employee Name: Arun
Salary: 50000

Explanation

The procedure display_employee accepts the employee ID as an input parameter and retrieves the corresponding employee name and salary from the employee_salary table.

2. FUNCTION

Definition

A Function is a stored PL/SQL program that performs a specific operation and returns a single value.

Syntax

```
CREATE OR REPLACE FUNCTION function_name
(parameter_name IN datatype)
RETURN datatype
IS
BEGIN
   statements;
   RETURN value;
END;
/
```

Create Function

The following function returns the salary grade based on the employee's salary.

```
CREATE OR REPLACE FUNCTION get_salary_grade (
   p_salary IN NUMBER
)
RETURN VARCHAR2
```

```
IS
   v_grade VARCHAR2(5);
BEGIN
   IF p_salary >= 50000 THEN
      v_grade := 'A+';
   ELSIF p_salary >= 40000 THEN
      v_grade := 'A';
   ELSIF p_salary >= 30000 THEN
      v_grade := 'B';
   ELSE
      v_grade := 'C';
   END IF;

   RETURN v_grade;
END;
/
```

Function Created

Execute Function

```
SELECT employee_id, employee_name, salary,
       get_salary_grade(salary) AS grade
FROM employee_salary;
```

Resultant Query

employee_id	employee_name	salary	grade
101	Kavin	45000	A
102	Meena	32000	B
103	Arun	50000	A+
104	Divya	38000	B
105	Priya	55000	A+

Explanation

The function get_salary_grade accepts the employee's salary as an input and returns the corresponding salary grade. The function is called inside the SELECT statement for each employee.

Important Commands

Display Stored Procedure

```
SELECT object_name, object_type
FROM user_objects
WHERE object_name = 'DISPLAY_EMPLOYEE';
```

Resultant Query

```
object_name                object_type
DISPLAY_EMPLOYEE PROCEDURE
```

Display Stored Function

```
SELECT object_name, object_type
FROM user_objects
WHERE object_name = 'GET_SALARY_GRADE';
```

Resultant Query

object_name	object_type
GET_SALARY_GRADE	FUNCTION

Drop Procedure

DROP PROCEDURE display_employee;

Procedure Dropped

Drop Function

DROP FUNCTION get_salary_grade;

Function Dropped

Result:

Thus, the Stored Procedure and Function were created and executed successfully using PL/SQL. The Stored Procedure was used to retrieve employee details, and the Function was used to calculate the salary grade based on the employee's salary.

EX. NO:10 Creation of Triggers method using product stock table

DATE:

Aim

To create and execute Triggers using PL/SQL to automatically perform an action when an INSERT, UPDATE, or DELETE operation occurs on a table.

Software Used

Oracle SQL / SQL Command Line

Description

A Trigger is a stored PL/SQL program that is automatically executed when a specified event occurs on a table or database.

Triggers can be used to automatically perform operations such as inserting, updating, or maintaining records when a particular event takes place.

1. CREATE TABLE

```
CREATE TABLE product_stock (
   product_id NUMBER PRIMARY KEY,
   product_name VARCHAR2(50),
   quantity NUMBER,
   price NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO product_stock VALUES (201, 'Laptop', 10, 55000);
INSERT INTO product_stock VALUES (202, 'Mobile', 20, 25000);
INSERT INTO product_stock VALUES (203, 'Keyboard', 15, 2000);
```

INSERT INTO product_stock VALUES (204, 'Headphone', 12, 3000);

INSERT INTO product_stock VALUES (205, 'Mouse', 25, 1000);

COMMIT;

Records Inserted

Display Product Records

SELECT * FROM product_stock;

Resultant Query

product_id	product_name	quantity	price
201	Laptop	10	55000
202	Mobile	20	25000
203	Keyboard	15	2000
204	Headphone	12	3000
205	Mouse	25	1000

2. BEFORE INSERT TRIGGER

Definition

A BEFORE INSERT Trigger is executed automatically before a new record is inserted into a table.

Syntax

CREATE OR REPLACE TRIGGER trigger_name

BEFORE INSERT ON table_name

FOR EACH ROW

BEGIN

```
    statements;
END;
/
```

Create Trigger

The following trigger checks whether the quantity is negative before inserting a product record.

```
CREATE OR REPLACE TRIGGER check_product_quantity
BEFORE INSERT ON product_stock
FOR EACH ROW
BEGIN
    IF :NEW.quantity < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Quantity cannot be negative');
    END IF;
END;
/
```

Trigger Created

Execute Trigger

```
INSERT INTO product_stock
VALUES (206, 'Printer', -5, 12000);
```

Output

ORA-20001: Quantity cannot be negative

The trigger prevents the insertion of an invalid quantity.

3. BEFORE UPDATE TRIGGER

Definition

A BEFORE UPDATE Trigger is executed automatically before an existing record is updated.

Create Trigger

The following trigger prevents the product price from being updated to a negative value.

```
CREATE OR REPLACE TRIGGER check_product_price
BEFORE UPDATE OF price ON product_stock
FOR EACH ROW
BEGIN
   IF :NEW.price < 0 THEN
      RAISE_APPLICATION_ERROR(-20002, 'Price cannot be negative');
   END IF;
END;
/
```

Trigger Created

Execute Trigger

```
UPDATE product_stock
SET price = -500
WHERE product_id = 203;
```

Output

ORA-20002: Price cannot be negative

The trigger prevents the invalid price from being updated.

4. AFTER INSERT TRIGGER

Definition

An AFTER INSERT Trigger is executed automatically after a new record is successfully inserted into a table.

Create Audit Table

```
CREATE TABLE product_audit (
   audit_id NUMBER,
   product_id NUMBER,
   action_type VARCHAR2(20)
);
```

Table Created

Create Trigger

```
CREATE OR REPLACE TRIGGER product_insert_audit
AFTER INSERT ON product_stock
FOR EACH ROW
BEGIN
   INSERT INTO product_audit
   VALUES (product_audit_seq.NEXTVAL, :NEW.product_id, 'INSERT');
END;
/
```

Trigger Created

Before executing the above trigger, create the required sequence:

CREATE SEQUENCE product_audit_seq

START WITH 1

INCREMENT BY 1;

Sequence Created

Insert New Product

INSERT INTO product_stock

VALUES (206, 'Printer', 8, 12000);

COMMIT;

Record Inserted

Display Audit Records

SELECT * FROM product_audit;

Resultant Query

audit_id	product_id	action_type
1	206	INSERT

5. DISPLAY FINAL PRODUCT RECORDS

SELECT * FROM product_stock

ORDER BY product_id;

Resultant Query

product_id	product_name	quantity	price
201	Laptop	10	55000
202	Mobile	20	25000
203	Keyboard	15	2000
204	Headphone	12	3000
205	Mouse	25	1000
206	Printer	8	12000

Important Commands

Display Triggers

SELECT trigger_name, status

FROM user_triggers;

Resultant Query

trigger_name	status
CHECK_PRODUCT_QUANTITY	ENABLED
CHECK_PRODUCT_PRICE	ENABLED
PRODUCT_INSERT_AUDIT	ENABLED

Drop Trigger

DROP TRIGGER check_product_quantity;

Trigger Dropped

DROP TRIGGER check_product_price;

Trigger Dropped

DROP TRIGGER product_insert_audit;

Trigger Dropped

Result

Thus, the Triggers were created and executed successfully using PL/SQL. The triggers automatically validated product quantity and price before modification and recorded product insertion details in the audit table.

EX. NO:11 Creation of cursor implementation using book table

DATE:

Aim

To implement and execute a Cursor using PL/SQL to retrieve and process records from a database table.

Software Used

Oracle SQL / SQL Command Line

Description

A Cursor is a pointer used to process the rows returned by a SQL query one row at a time.

Cursors are useful when each record needs to be processed individually.

There are two types of cursors:

1. Implicit Cursor
2. Explicit Cursor

In this practical, an Explicit Cursor is implemented.

1. CREATE TABLE

```
CREATE TABLE book_details (
   book_id NUMBER PRIMARY KEY,
   book_name VARCHAR2(50),
   author_name VARCHAR2(50),
   price NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO book_details VALUES (301, 'Java Programming', 'James', 550);
```

INSERT INTO book_details VALUES (302, 'Database Systems', 'Korth', 650);

INSERT INTO book_details VALUES (303, 'Python Basics', 'Guido', 450);

INSERT INTO book_details VALUES (304, 'Web Development', 'Thomas', 700);

INSERT INTO book_details VALUES (305, 'Data Structures', 'Robert', 600);

COMMIT;

Records Inserted

Display Book Records

SELECT * FROM book_details;

Resultant Query

book_id	book_name	author_name	price
301	Java Programming	James	550
302	Database Systems	Korth	650
303	Python Basics	Guido	450
304	Web Development	Thomas	700
305	Data Structures	Robert	600

2. EXPLICIT CURSOR

Definition

An Explicit Cursor is a cursor that is declared and controlled by the programmer. It is used to retrieve multiple rows from a table and process them one by one.

Syntax

```
DECLARE
   CURSOR cursor_name IS
      SELECT statement;
```

```
BEGIN
   OPEN cursor_name;

   LOOP
      FETCH cursor_name INTO variables;

      EXIT WHEN cursor_name%NOTFOUND;

      statements;
   END LOOP;

   CLOSE cursor_name;
END;
/
```

Create Explicit Cursor

The following cursor retrieves each book name and price from the book_details table.

```
SET SERVEROUTPUT ON;

DECLARE
   CURSOR book_cursor IS
      SELECT book_name, price
      FROM book_details;

   v_book_name book_details.book_name%TYPE;
```

```
    v_price book_details.price%TYPE;
BEGIN
    OPEN book_cursor;

    LOOP
        FETCH book_cursor INTO v_book_name, v_price;

        EXIT WHEN book_cursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Book: ' || v_book_name || ' | Price: ₹' || v_price
        );
    END LOOP;

    CLOSE book_cursor;
END;
/
```

Output

Book: Java Programming | Price: ₹550

Book: Database Systems | Price: ₹650

Book: Python Basics | Price: ₹450

Book: Web Development | Price: ₹700

Book: Data Structures | Price: ₹600

3. CURSOR WITH CONDITION

The following cursor displays only the books whose price is greater than ₹500.

```
SET SERVEROUTPUT ON;

DECLARE
   CURSOR book_cursor IS
      SELECT book_id, book_name, price
      FROM book_details
      WHERE price > 500;

   v_book_id book_details.book_id%TYPE;
   v_book_name book_details.book_name%TYPE;
   v_price book_details.price%TYPE;
BEGIN
   OPEN book_cursor;

   LOOP
      FETCH book_cursor INTO v_book_id, v_book_name, v_price;

      EXIT WHEN book_cursor%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE(
         'Book ID: ' || v_book_id ||
         ' | Book: ' || v_book_name ||
```

```
        ' | Price: ₹' || v_price
      );
  END LOOP;
  CLOSE book_cursor;
END;
/
```

Resultant Query

book_id	book_name	price
301	Java Programming	550
302	Database Systems	650
304	Web Development	700
305	Data Structures	600

4. CURSOR ATTRIBUTES

Cursor attributes provide information about the current status of a cursor.

Important Cursor Attributes

Cursor Attribute	Description
%FOUND	Returns TRUE if the last fetch returned a row
%NOTFOUND	Returns TRUE if the last fetch did not return a row
%ROWCOUNT	Returns the number of rows processed
%ISOPEN	Returns TRUE if the cursor is currently open

5. CURSOR USING %ROWCOUNT

The following program displays the total number of books processed by the cursor.

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR book_cursor IS
      SELECT book_name, price
      FROM book_details;

   v_book_name book_details.book_name%TYPE;
   v_price book_details.price%TYPE;
BEGIN
   OPEN book_cursor;

   LOOP
      FETCH book_cursor INTO v_book_name, v_price;

      EXIT WHEN book_cursor%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE(
         'Book: ' || v_book_name ||
         ' | Price: ₹' || v_price
      );
   END LOOP;
```

```
    DBMS_OUTPUT.PUT_LINE(
        'Total Books Processed: ' || book_cursor%ROWCOUNT
    );

    CLOSE book_cursor;
END;
/
```

Output

```
Book: Java Programming | Price: ₹550
Book: Database Systems | Price: ₹650
Book: Python Basics | Price: ₹450
Book: Web Development | Price: ₹700
Book: Data Structures | Price: ₹600
Total Books Processed: 5
```

Result

Thus, the Cursor was successfully implemented using PL/SQL. The explicit cursor was used to retrieve and process book records one by one, apply conditions, and count the number of processed records.

EX. NO:12 Creation of PL/SQL Exception Handling using hotel booking table

DATE:26/09/2026

Aim

To implement Exception Handling in PL/SQL and handle runtime errors using predefined and user-defined exceptions.

Software Used

Oracle SQL / SQL Command Line

Description

An Exception is an error that occurs during the execution of a PL/SQL program.

Exception Handling is used to handle runtime errors without terminating the program abnormally.

The basic structure of exception handling is:

```
DECLARE
   declarations;
BEGIN
   statements;
EXCEPTION
   WHEN exception_name THEN
      statements;
END;
/
```

Types of Exceptions

Type	Description
Predefined Exception	Built-In Exceptions Provided By Oracle
User-Defined Exception	Exception Created By The Programmer

1. CREATE TABLE

```
CREATE TABLE hotel_bookings (
   booking_id NUMBER PRIMARY KEY,
   guest_name VARCHAR2(50),
   room_type VARCHAR2(30),
   room_charge NUMBER
);
```

Table Created

Insert Records

```
INSERT INTO hotel_bookings VALUES (501, 'Rahul', 'Deluxe', 4500);
INSERT INTO hotel_bookings VALUES (502, 'Anitha', 'Standard', 3000);
INSERT INTO hotel_bookings VALUES (503, 'Vikram', 'Suite', 7500);
INSERT INTO hotel_bookings VALUES (504, 'Sneha', 'Deluxe', 5000);
INSERT INTO hotel_bookings VALUES (505, 'Karthik', 'Standard', 2800);


COMMIT;
```

Records Inserted

Display Booking Records

```
SELECT * FROM hotel_bookings;
```

Resultant Query

Booking_Id	Guest_Name	Room_Type	Room_Charge
501	Rahul	Deluxe	4500
502	Anitha	Standard	3000
503	Vikram	Suite	7500
504	Sneha	Deluxe	5000
505	Karthik	Standard	2800

2. NO_DATA_FOUND EXCEPTION

Definition

NO_DATA_FOUND occurs when a SELECT INTO statement does not return any record.

Program

```
SET SERVEROUTPUT ON;


DECLARE
   v_name hotel_bookings.guest_name%TYPE;
   v_charge hotel_bookings.room_charge%TYPE;
BEGIN
   SELECT guest_name, room_charge
   INTO v_name, v_charge
   FROM hotel_bookings
   WHERE booking_id = 599;


   DBMS_OUTPUT.PUT_LINE('Guest Name: ' || v_name);
   DBMS_OUTPUT.PUT_LINE('Room Charge: ₹' || v_charge);


EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Exception: Booking not found.');
END;
/
```

Output

Exception: Booking not found.

Exception Handled Successfully

3. TOO_MANY_ROWS EXCEPTION

Definition

TOO_MANY_ROWS occurs when a SELECT INTO statement returns more than one record.

Program

```
SET SERVEROUTPUT ON;


DECLARE
   v_name hotel_bookings.guest_name%TYPE;
BEGIN
   SELECT guest_name
   INTO v_name
   FROM hotel_bookings
   WHERE room_charge > 3000;


   DBMS_OUTPUT.PUT_LINE('Guest Name: ' || v_name);


EXCEPTION
   WHEN TOO_MANY_ROWS THEN
      DBMS_OUTPUT.PUT_LINE(
         'Exception: More than one booking found.'
      );
END;
/
```

Output

Exception: More than one booking found.

Exception Handled Successfully

4. ZERO_DIVIDE EXCEPTION

Definition

ZERO_DIVIDE occurs when a number is divided by zero.

Program

```
SET SERVEROUTPUT ON;


DECLARE
   v_charge NUMBER := 4500;
   v_days NUMBER := 0;
   v_result NUMBER;
BEGIN
   v_result := v_charge / v_days;


   DBMS_OUTPUT.PUT_LINE('Result: ' || v_result);


EXCEPTION
   WHEN ZERO_DIVIDE THEN
      DBMS_OUTPUT.PUT_LINE(
         'Exception: Cannot divide by zero.'
      );
END;
/
```

Output

Exception: Cannot divide by zero.

Exception Handled Successfully

5. USER-DEFINED EXCEPTION

Definition

A User-Defined Exception is an exception created by the programmer to handle a specific business condition.

In this example, a hotel booking is allowed only when the room charge is within the available booking amount.

Program

```
SET SERVEROUTPUT ON;


DECLARE
   v_charge hotel_bookings.room_charge%TYPE;
   v_payment NUMBER := 2000;
   insufficient_payment EXCEPTION;
BEGIN
   SELECT room_charge
   INTO v_charge
   FROM hotel_bookings
   WHERE booking_id = 501;


   IF v_payment < v_charge THEN
      RAISE insufficient_payment;
   END IF;
```

```
    UPDATE hotel_bookings
    SET room_charge = room_charge - v_payment
    WHERE booking_id = 501;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Payment Successful.');

EXCEPTION
    WHEN insufficient_payment THEN
        DBMS_OUTPUT.PUT_LINE(
            'Exception: Insufficient payment.'
        );
END;
/
```

Output

Exception: Insufficient payment.

User-Defined Exception Handled Successfully

6. EXCEPTION HANDLING WITH SUCCESSFUL TRANSACTION

The following program performs a valid payment when sufficient payment amount is available.

```
SET SERVEROUTPUT ON;

DECLARE
    v_charge hotel_bookings.room_charge%TYPE;
```

```
    v_payment NUMBER := 1500;
BEGIN
    SELECT room_charge
    INTO v_charge
    FROM hotel_bookings
    WHERE booking_id = 501;

    IF v_payment > v_charge THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Payment amount is greater than room charge'
        );
    END IF;

    UPDATE hotel_bookings
    SET room_charge = room_charge - v_payment
    WHERE booking_id = 501;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Payment Successful.');
    DBMS_OUTPUT.PUT_LINE(
        'Paid Amount: ₹' || v_payment
    );
```

```
EXCEPTION
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE(
         'Error: ' || SQLERRM
      );
END;
/
```

Output

Payment Successful.

Paid Amount: ₹1500

Display Updated Booking

```
SELECT * FROM hotel_bookings
WHERE booking_id = 501;
```

Resultant Query

Booking_Id	Guest_Name	Room_Type	Room_Charge
501	Rahul	Delux	3000

Important Predefined Exceptions

Exception	**Description**
NO_DATA_FOUND	No record is found
TOO_MANY_ROWS	More than one record is returned
ZERO_DIVIDE	Division by zero occurs
DUP_VAL_ON_INDEX	Duplicate value is inserted into a unique/primary key column
VALUE_ERROR	Invalid value or conversion error occurs

Result

Thus, **PL/SQL Exception Handling** was successfully implemented. The NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, and user-defined exceptions were handled successfully using appropriate exception-handling mechanisms.

EX. NO:13 Creation of Database Normalization using Students Course Table

DATE:27/09/2026

Aim

To study and implement Database Normalization up to Boyce-Codd Normal Form (BCNF) to reduce data redundancy and improve data integrity.

Software Used

Oracle SQL / SQL Command Line

Description

Normalization is the process of organizing data in a database to reduce redundancy and avoid insertion, deletion, and update anomalies.

The major normal forms covered are:

1. First Normal Form (1NF)
2. Second Normal Form (2NF)
3. Third Normal Form (3NF)
4. Boyce-Codd Normal Form (BCNF)

1. UNNORMALIZED TABLE

Consider the following Students Course table.

Unnormalized Table

The table contains repeated information. For example, the instructor name Anand is repeated for students taking the Python course.

Student_Id	Student_Name	Course_Id	Course_Name	Instructor_Id	Instructor_Name
601	Aarav	C01	Python	I01	Anand
602	Bhavana	C02	DBMS	I02	Meera
603	Charan	C03	Python	I01	Anand
604	Diya	C04	Java	I03	Ravi

605	Eshan	C05	DBMS	I02	Meera

2. FIRST NORMAL FORM (1NF)

Definition

A relation is in 1NF when:

- Each column contains atomic values.
- There are no repeating groups.
- Each row is uniquely identifiable.

The above table already contains atomic values, so it can be considered as the 1NF table.

1NF Table

Student_Id	Student_Name	Course_Id	Course_Name	Instructor_Id	Instructor_Name
601	Aarav	C01	Python	I01	Anand
602	Bhavana	C02	DBMS	I02	Meera
603	Charan	C03	Python	I01	Anand
604	Diya	C04	Java	I03	Ravi
605	Eshan	C05	DBMS	I02	Meera

Primary Key: student_id, course_id

3. SECOND NORMAL FORM (2NF)

Definition

A relation is in 2NF when:

5. It is already in 1NF.
6. There is no partial dependency.
7. Every non-key attribute must depend on the complete primary key.

In the 1NF table, some attributes depend only on part of the composite key.

For example:

• student_name depends only on student_id.
• course_name depends only on course_id.
• instructor_id depends on course_id.

Therefore, the table is divided into separate tables.

Student Table

Student_Id	Student_Name
601	Aarav
602	Bhavana
603	Charan
604	Dhiya
605	Eshan

Course Table

Course_Id	Course_Name	Instructor_Id
C01	Python	I01
C02	DBMS	I02
C03	Java	I03

Enrollment Table

student_id	course_id
601	C01
602	C02
603	C01
604	C03
605	C02

The partial dependencies are removed.

Therefore, the tables are in 2NF.

4. THIRD NORMAL FORM (3NF)

Definition

A relation is in 3NF when:

8. It is already in 2NF.
9. There is no transitive dependency.
10. Non-key attributes must depend only on the primary key.

In the Course table:

course_id → instructor_id

and

instructor_id → instructor_name

Therefore, instructor_name has a transitive dependency on course_id.

To remove this dependency, the instructor information is separated into another table.

Course Table

Course_Id	Course_Name	Instructor_Id
C01	Python	I01
C02	DBMS	I02
C03	Java	I03

Instructor Table

Instructor_Id	Instructor_Name
I01	Anand
I02	Meera
I03	Ravi

Enrollment Table

student_id	course_id
601	C01
602	C02
603	C01
604	C03

605	C02

Therefore, the tables are in 3NF.

5. BOYCE-CODD NORMAL FORM (BCNF)

Definition

A relation is in BCNF when, for every non-trivial functional dependency:

The determinant must be a candidate key.

BCNF is a stronger version of 3NF.

Example

Consider a Student_Course table.

student_id	course_id
601	C01
602	C02
603	C01
604	C03
605	C02

Assume that each instructor is responsible for only one course.

Consider the following relationship:

instructor_id	course_id
I01	C01
I02	C02
I03	C03

Functional dependency:

instructor_id → course_id

Here, instructor_id determines course_id.

If instructor_id is not a candidate key in the given relationship, the relation violates BCNF.

Therefore, the relation can be decomposed into separate tables.

Instructor Table

instructor_id	course_id
I01	C01
I02	C02
I03	C03

Course Table

Course_Id
C01
C02
C03

After decomposition, the determinant is treated as a key in its corresponding relation.

Therefore, the relation is decomposed to satisfy BCNF.

Result

Thus, the Database Normalization process was studied and implemented up to 1NF, 2NF, 3NF, and BCNF. The normalization process reduces data redundancy, removes dependency-related anomalies, and improves database consistency and integrity.

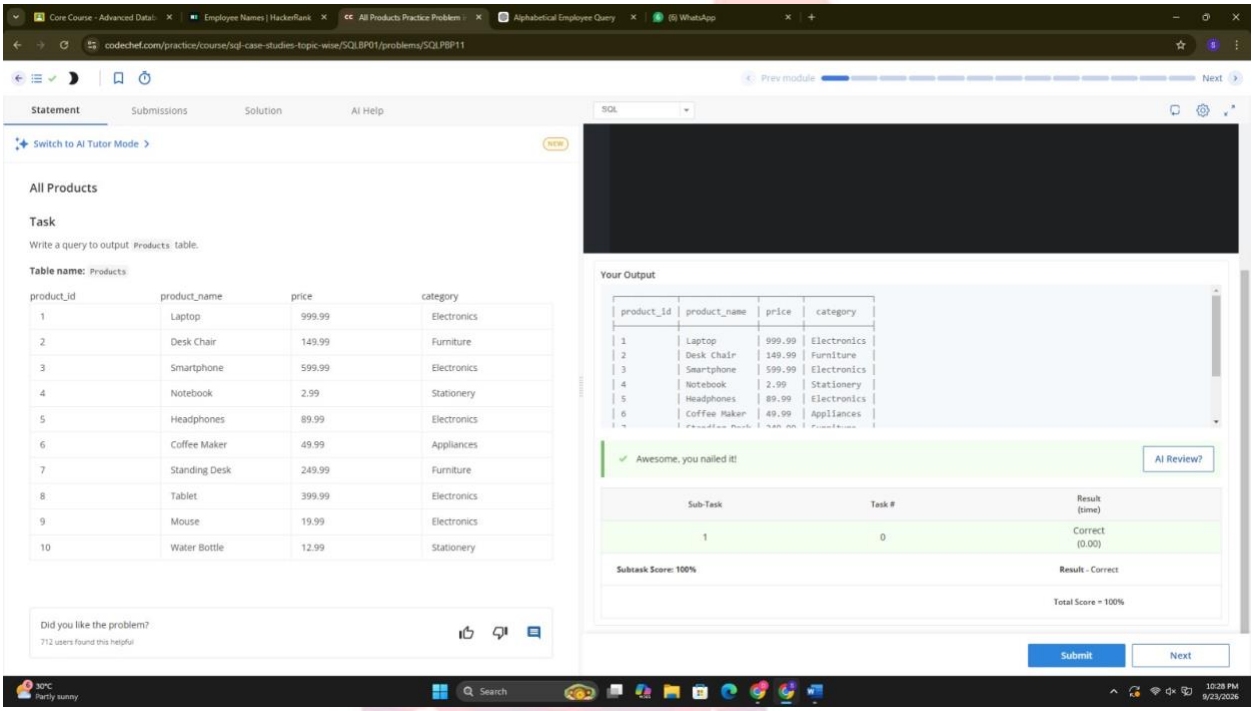
Codechef

1.ALL PRODUCTS

PROGRAM:

```
select * from products;
```

OUTPUT:

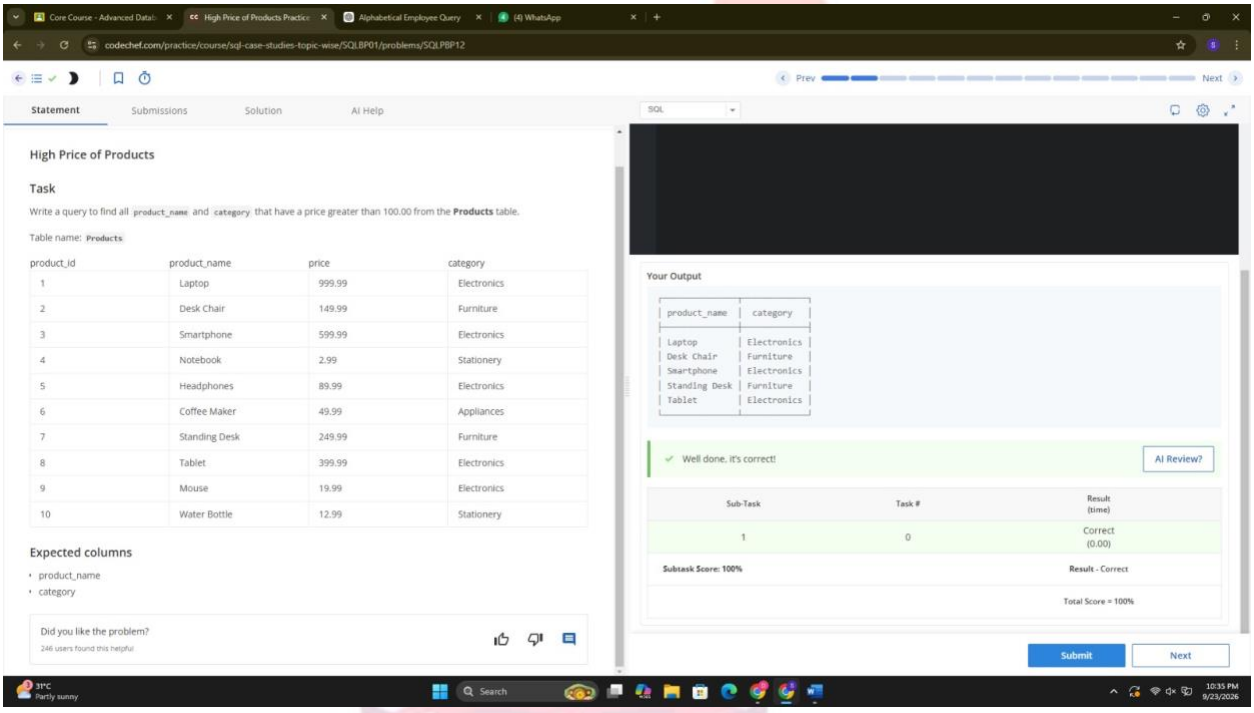


2. HIGH PRICE OF PRODUCTS

PROGRAM:

```
select product_name, category from products where price > 100.00;
```

OUTPUT:

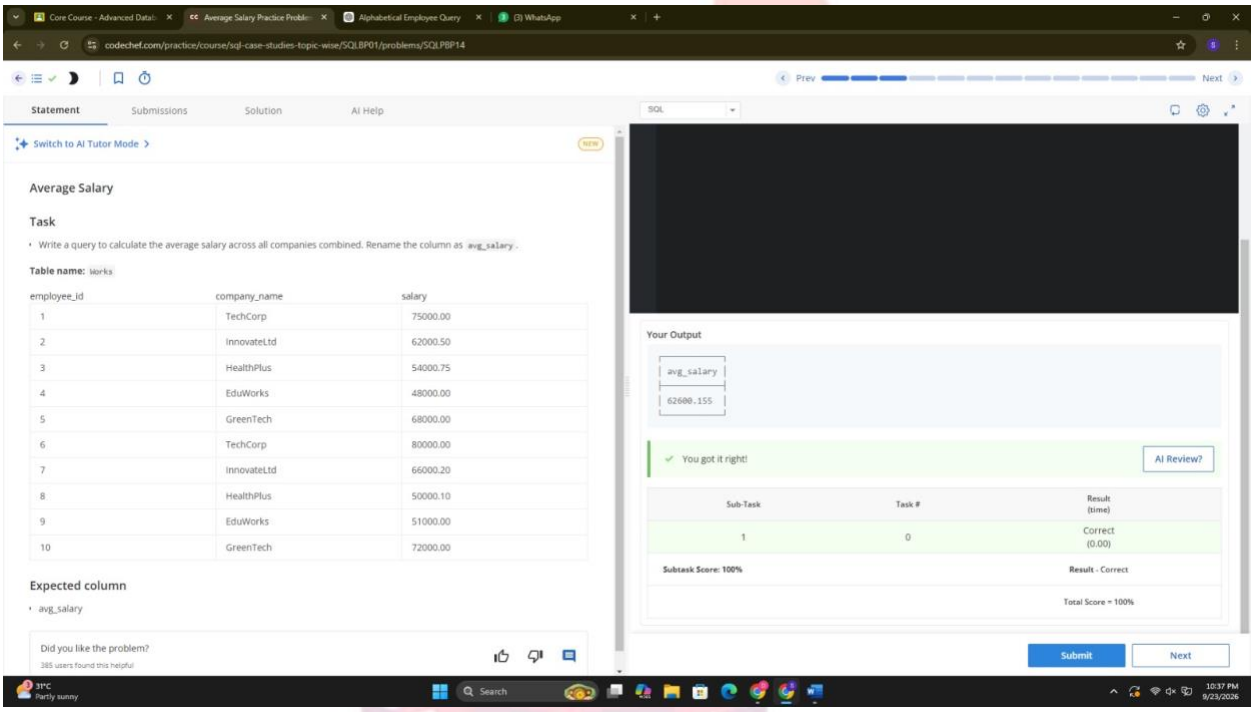


3. AVERAGE SALARY

PROGRAM:

```
select avg(salary) as avg_salary from works;
```

OUTPUT:

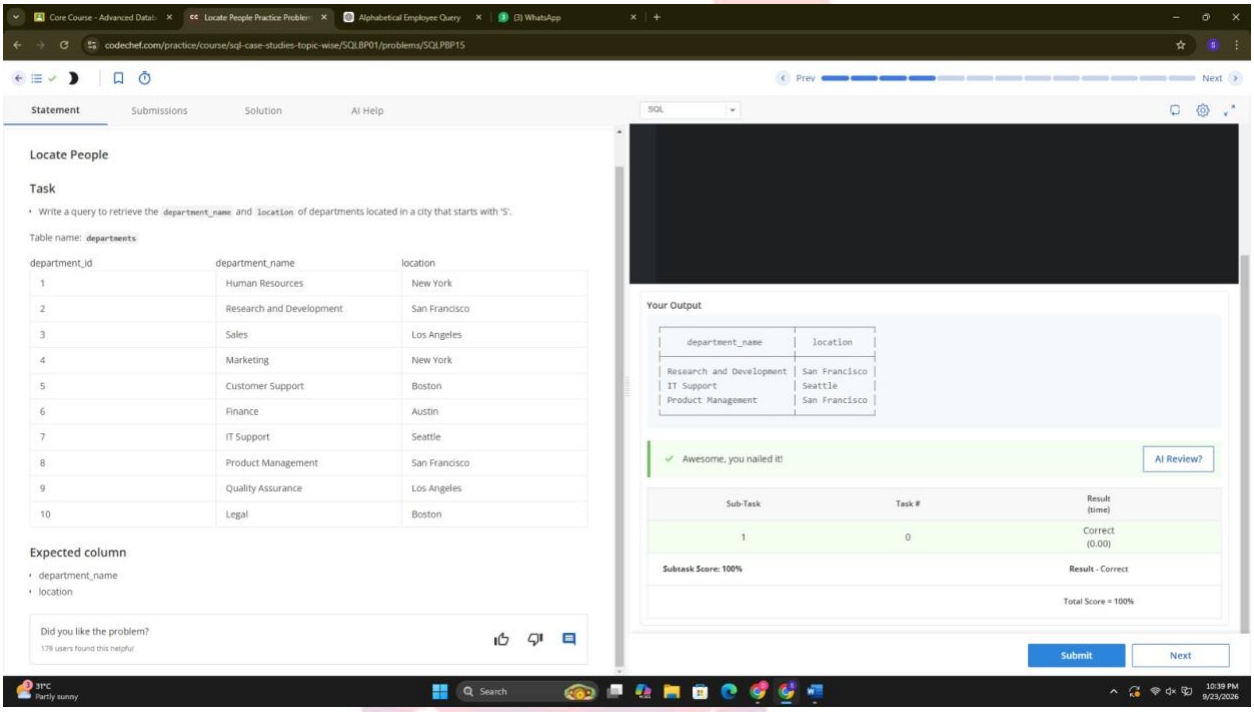


4. LOCATE PEOPLE

PROGRAM:

```
select department_name, location from departments where location like 's%';
```

OUTPUT:

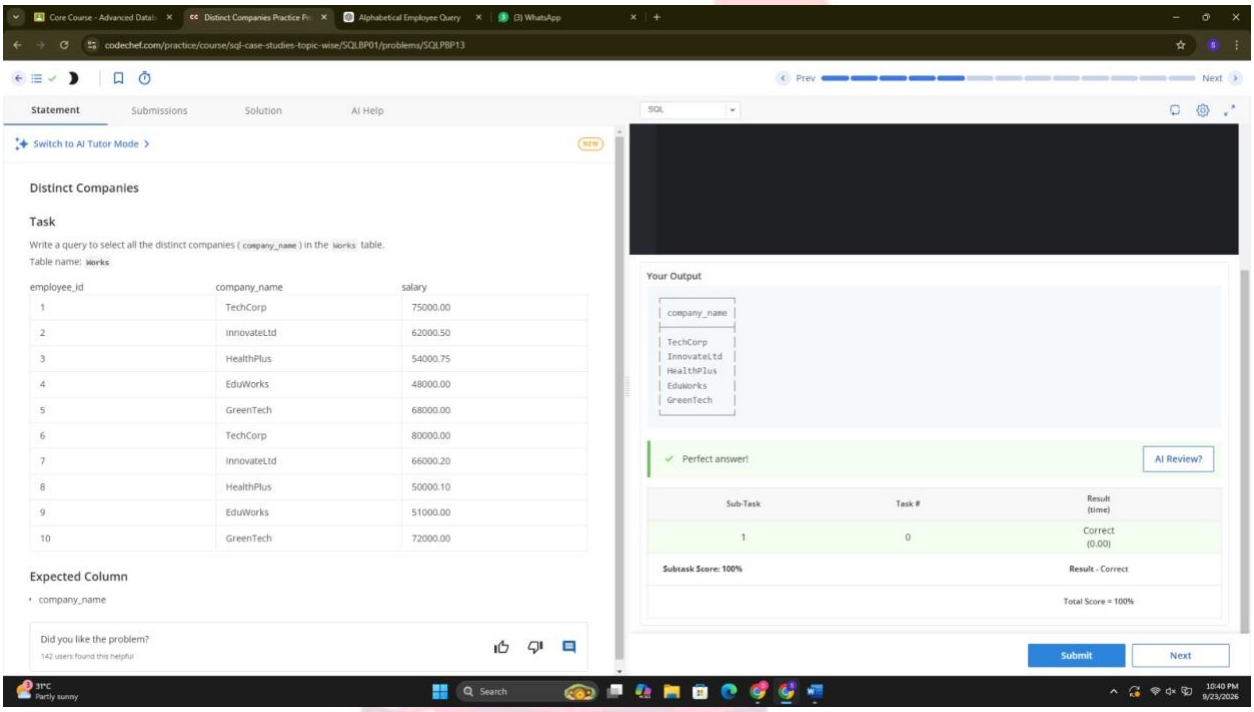


5. DISTINCT COMPANIES

PROGRAM:

```
select distinct company_name from works;
```

OUTPUT:

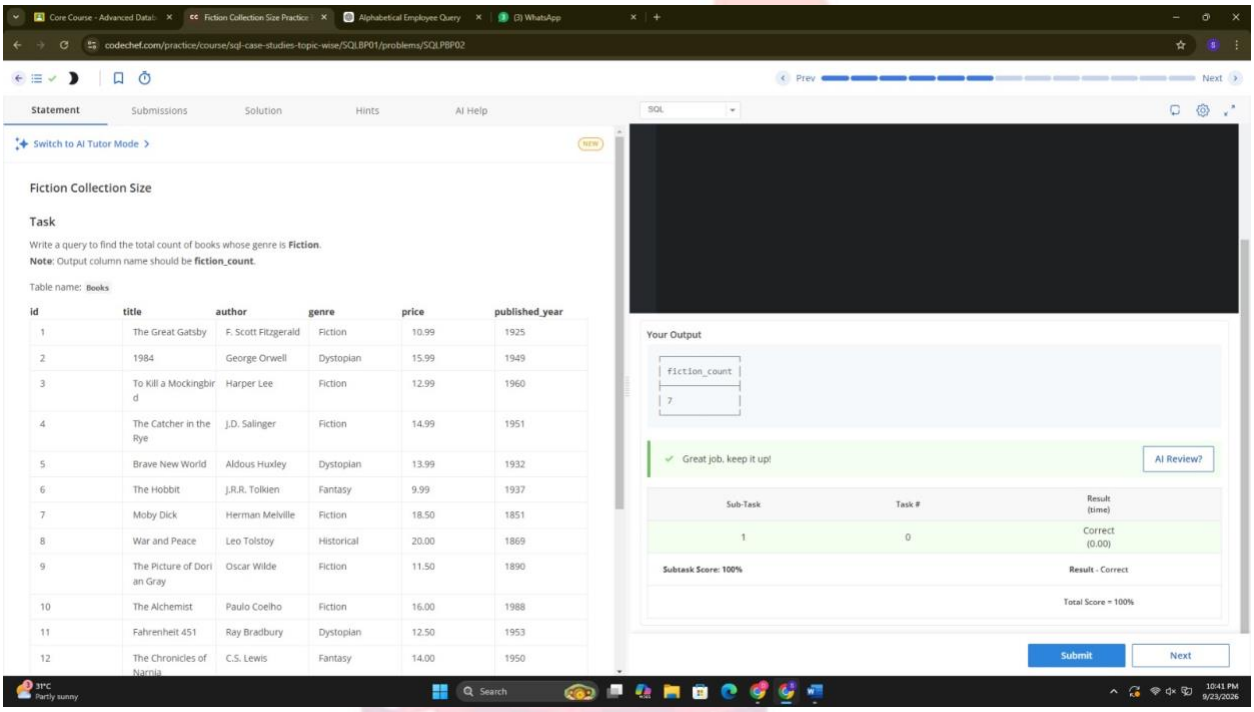


6. FICTION COLLECTION SIZE

PROGRAM:

```
select count(*) as fiction_count from books where genre = 'Fiction';
```

OUTPUT:

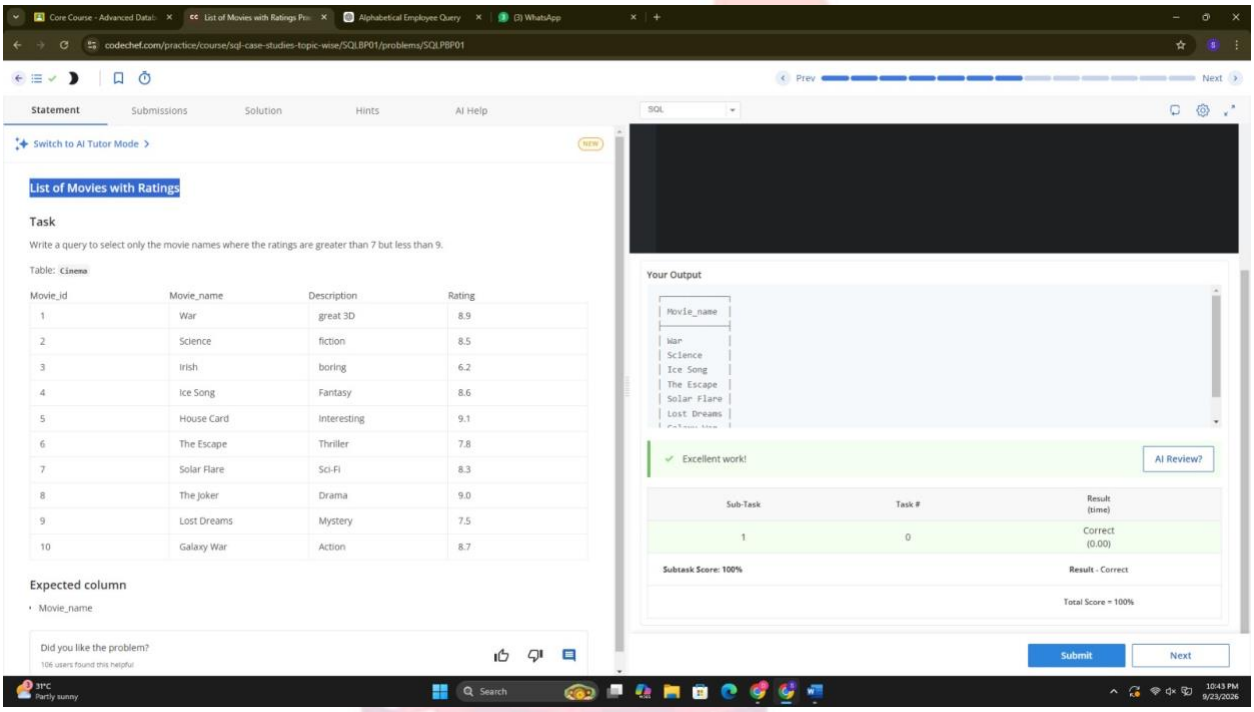


7. LIST OF MOVIES WITH RATINGS

PROGRAM:

```
select movie_name from cinema where rating > 7 and rating < 9;
```

OUTPUT:

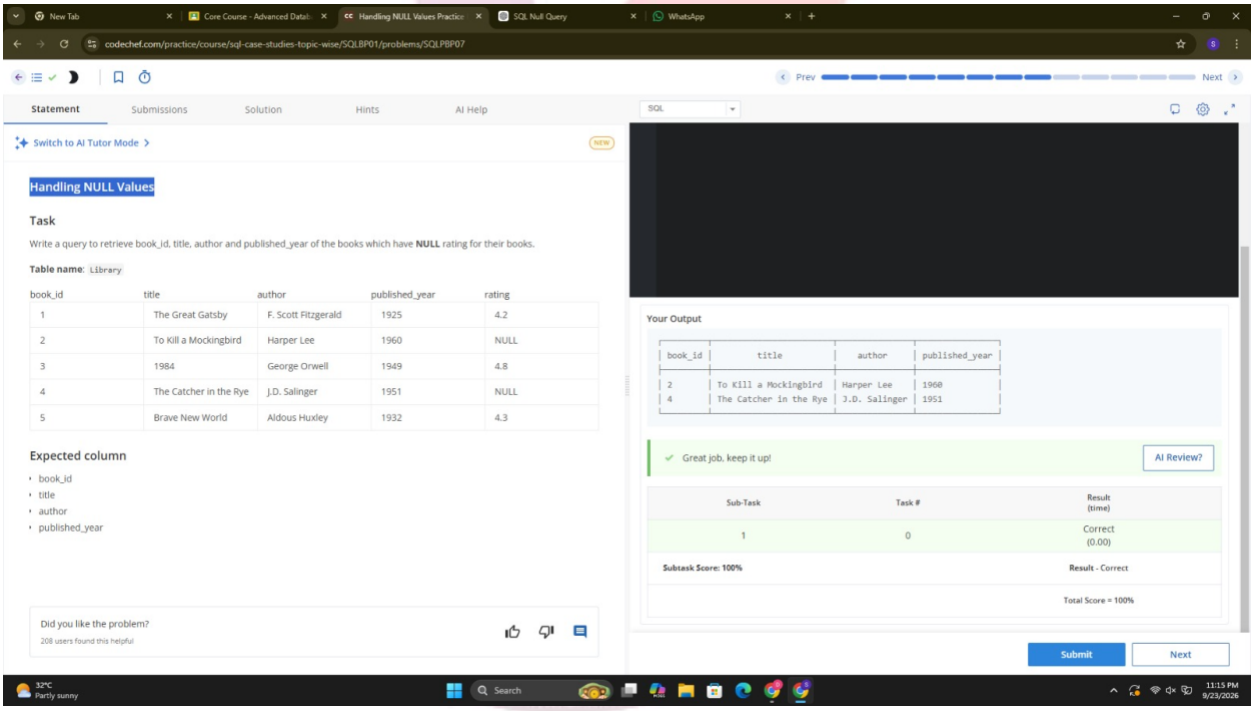


8. HANDLING NULL VALUES

PROGRAM:

```
select book_id, title, author, published_year
from library
where rating is null;
```

OUTPUT:

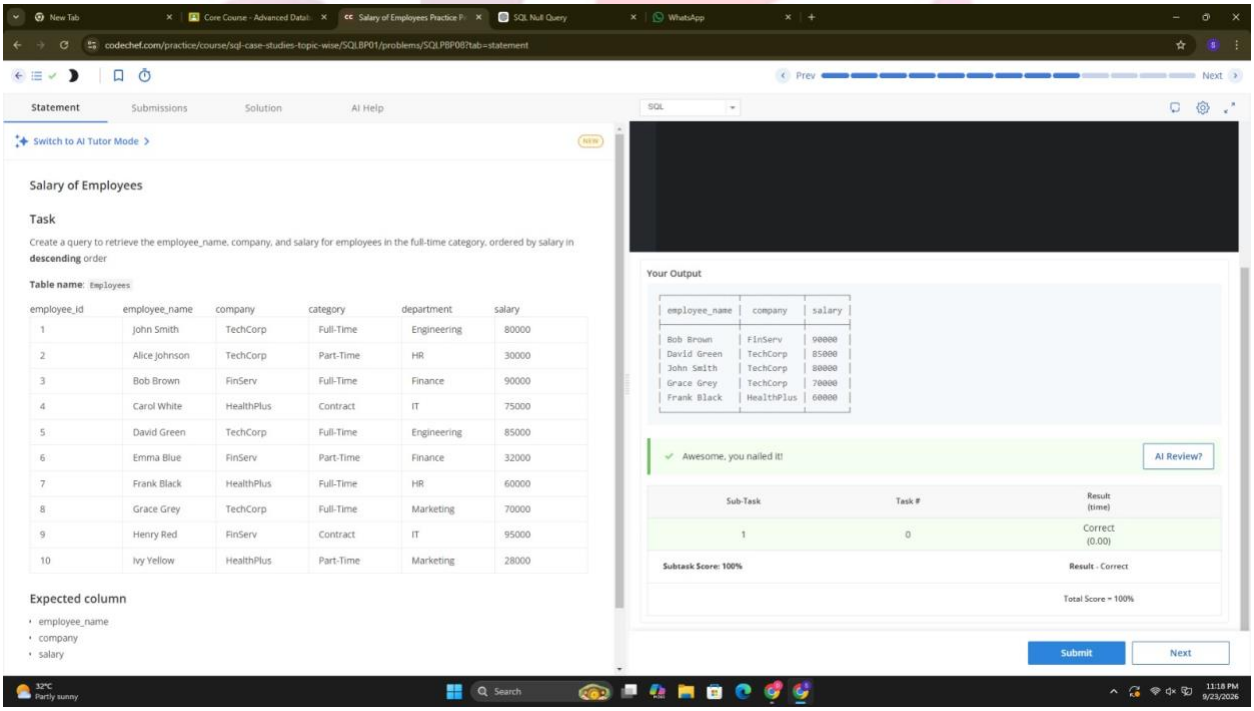


9. SALARY OF EMPLOYEES

PROGRAM:

```
SELECT employee_name, company, salary
FROM Employees
WHERE category = 'Full-Time'
ORDER BY salary DESC;
```

OUTPUT:

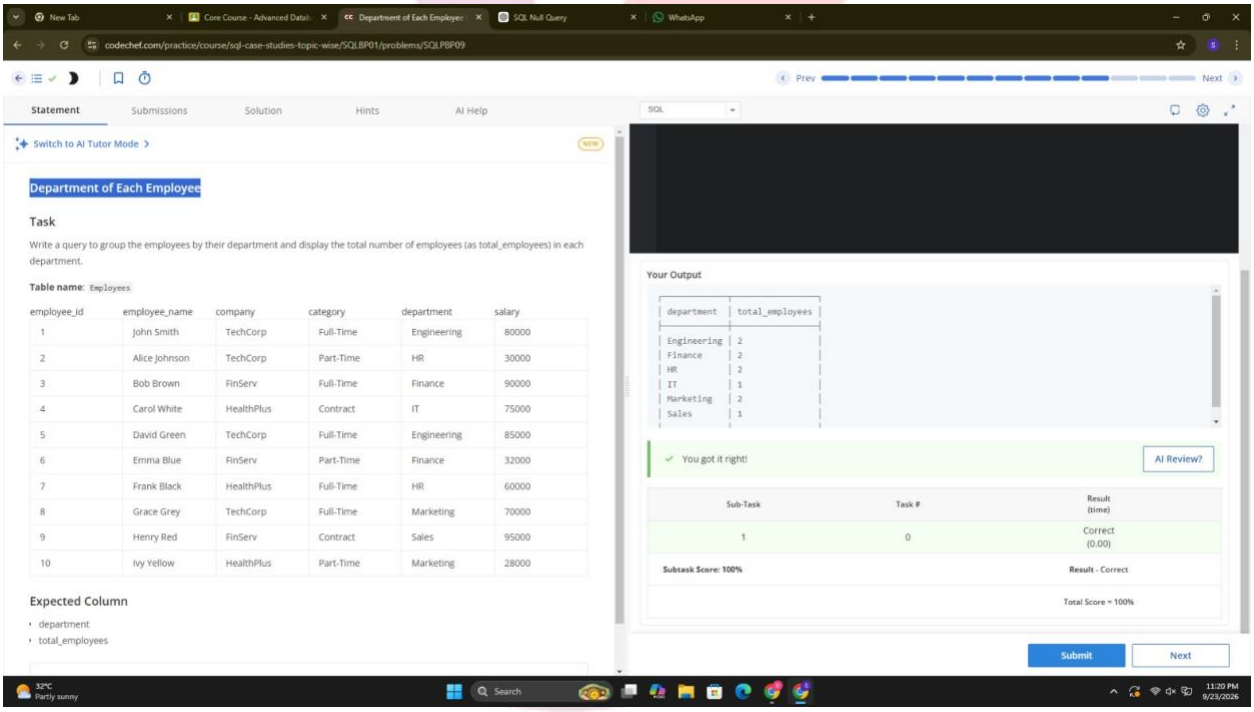


10. DEPARTMENT OF EACH EMPLOYEE

PROGRAM:

```
select department, count(*) as total_employees
from employees
group by department;
```

OUTPUT:

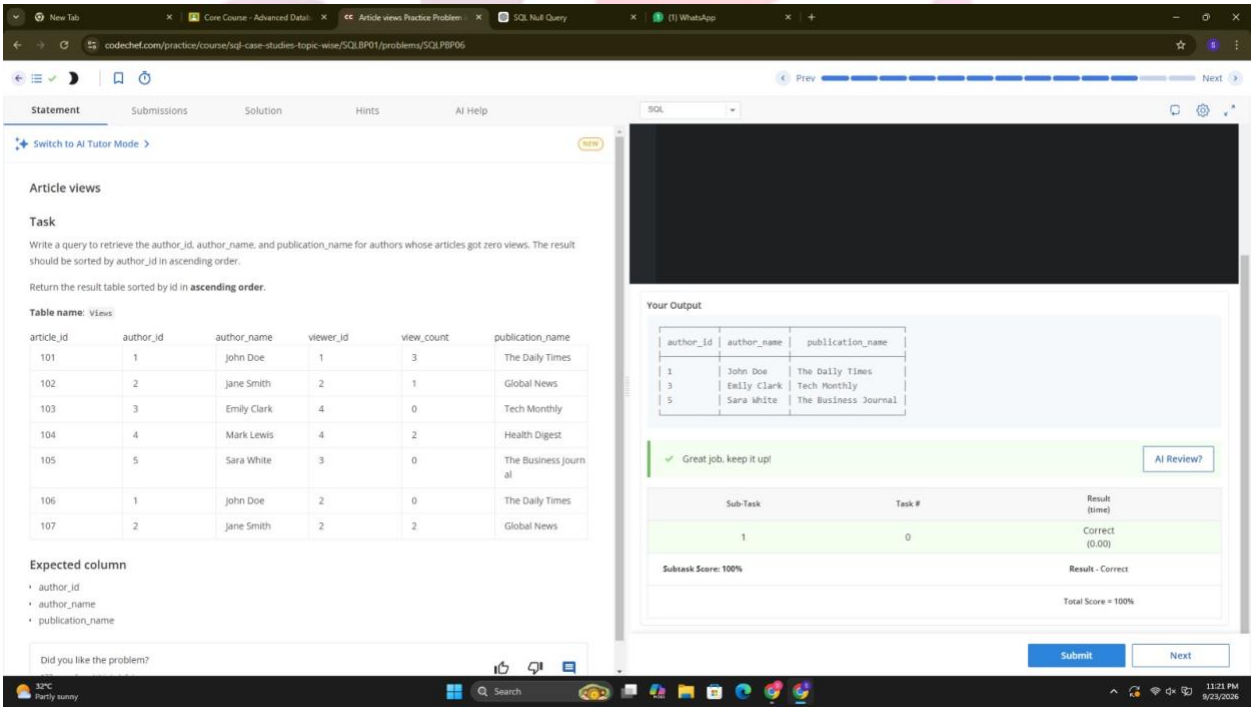


11. ARTICLE VIEWS

PROGRAM:

```
select author_id, author_name, publication_name
from views
where view_count = 0
order by author_id asc;
```

OUTPUT:

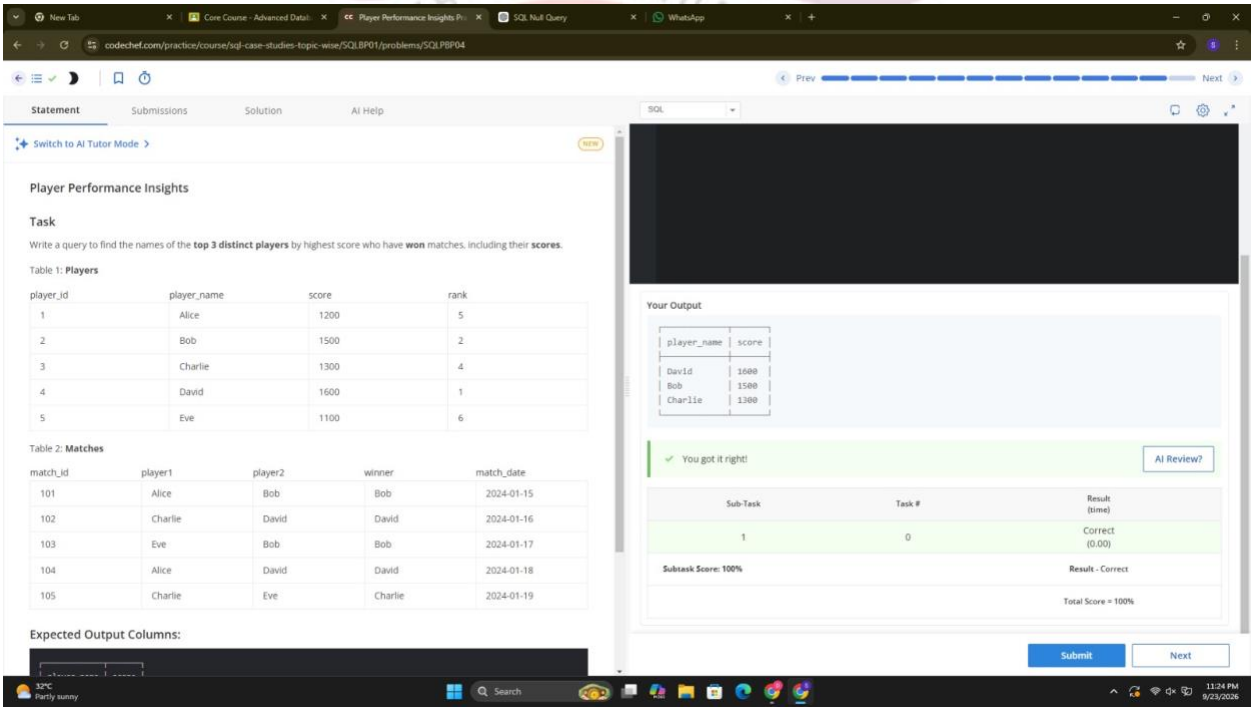


12. PLAYER PERFORMANCE INSIGHTS

PROGRAM:

```
select distinct p.player_name, p.score
from players p
join matches m
on p.player_name = m.winner
order by p.score desc
limit 3;
```

OUTPUT:

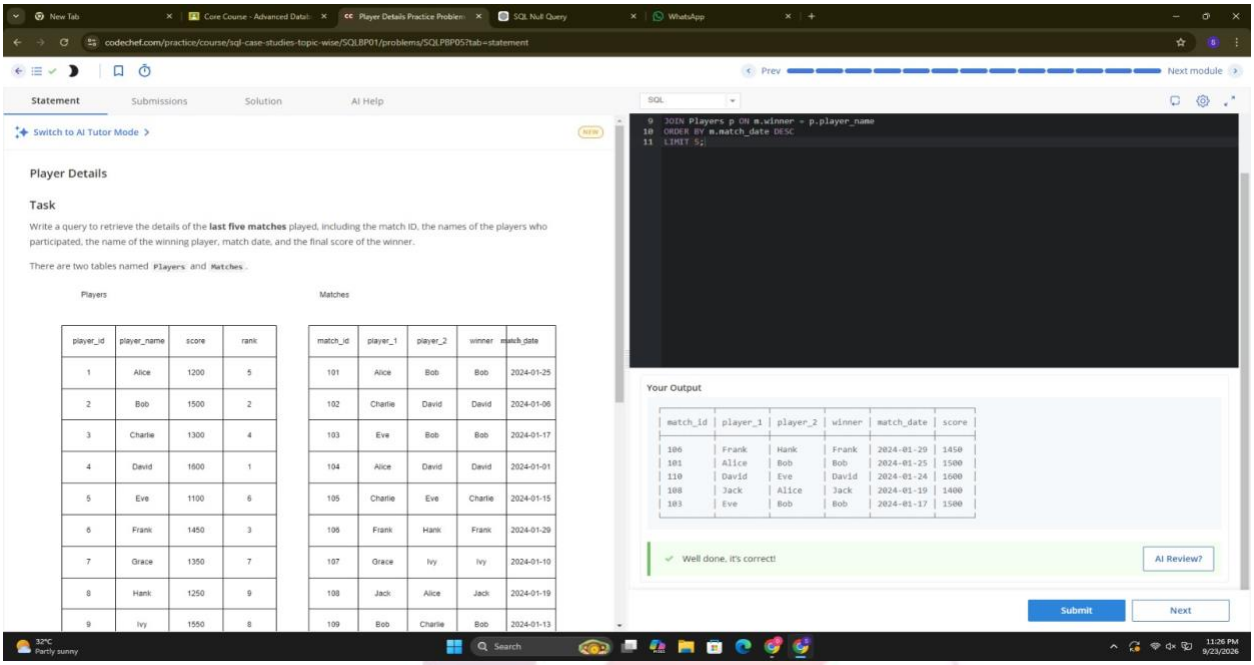


13. PLAYER DETAILS

PROGRAM:

```
SELECT
    m.match_id,
    m.player_1,
    m.player_2,
    m.winner,
    m.match_date,
    p.score
FROM Matches m
JOIN Players p ON m.winner = p.player_name
ORDER BY m.match_date DESC
LIMIT 5;
```

OUTPUT:



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

1. CREATING A TABLE AND INSERTING DATA

PROGRAM:

```
/** Grocery list:

Bananas (4)

Peanut Butter (1)

Dark Chocolate Bars (2)

**/

CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER );

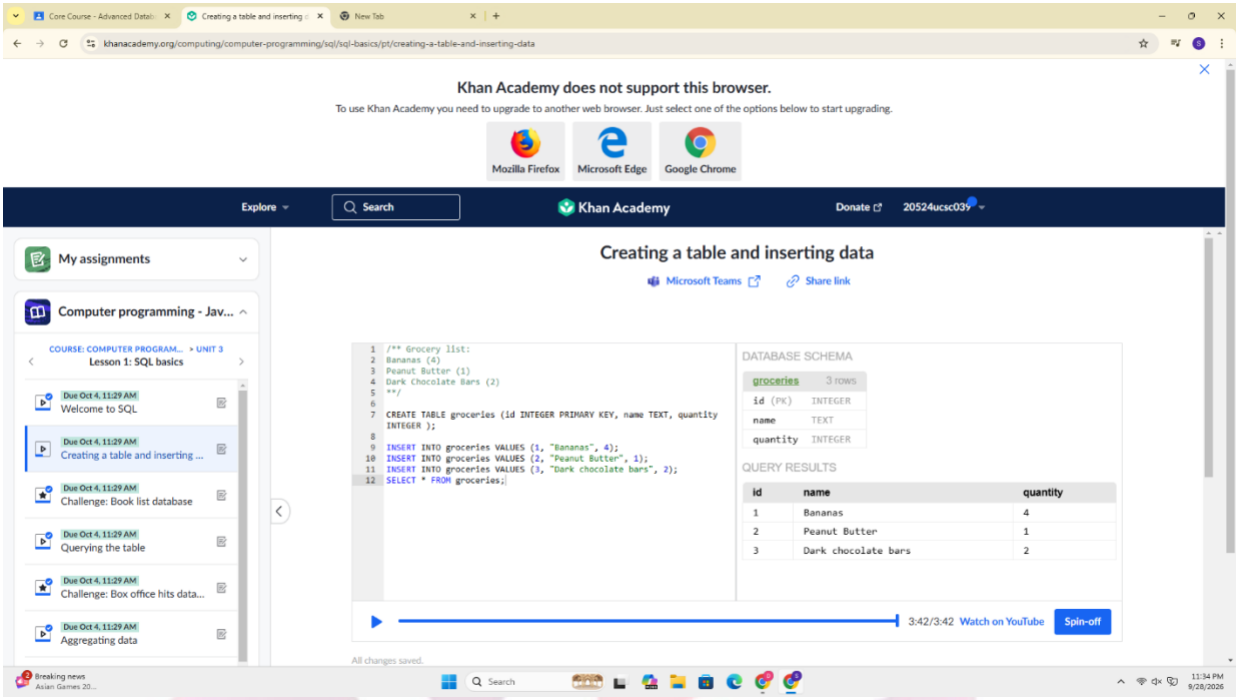
INSERT INTO groceries VALUES (1, "Bananas", 4);

INSERT INTO groceries VALUES (2, "Peanut Butter", 1);

INSERT INTO groceries VALUES (3, "Dark chocolate bars", 2);

SELECT * FROM groceries;
```

OUTPUT:



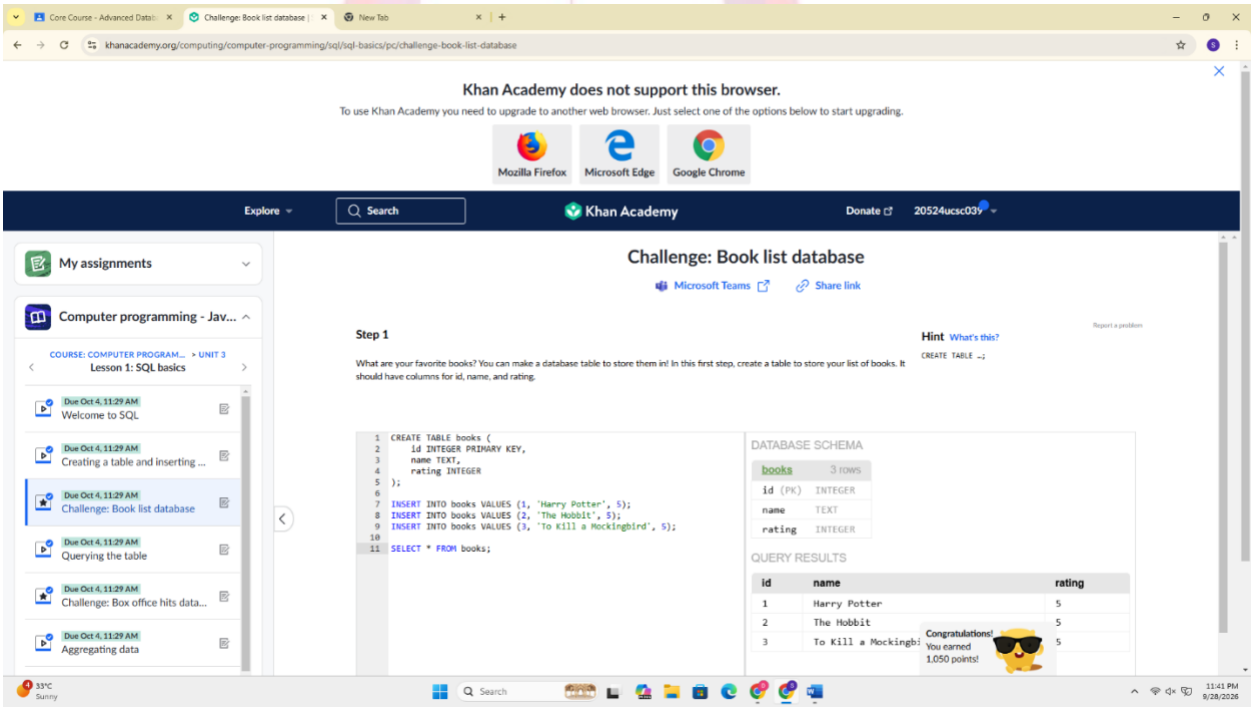
TRUTH-TOIL-TRIUMPH
AUTONOMOUS

2.BOOK LIST DATABASE

PROGRAM:

```
CREATE TABLE books (
    id INTEGER PRIMARY KEY,
    name TEXT,
    rating INTEGER
);
INSERT INTO books VALUES (1, 'Harry Potter', 5);
INSERT INTO books VALUES (2, 'The Hobbit', 5);
INSERT INTO books VALUES (3, 'To Kill a Mockingbird', 5);
SELECT * FROM books;
```

OUTPUT:



3. QUERYING THE TABLE

PROGRAM:

```
CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER, aisle INTEGER);

INSERT INTO groceries VALUES (1, "Bananas", 4, 7);

INSERT INTO groceries VALUES(2, "Peanut Butter", 1, 2);

INSERT INTO groceries VALUES(3, "Dark Chocolate Bars", 2, 2);

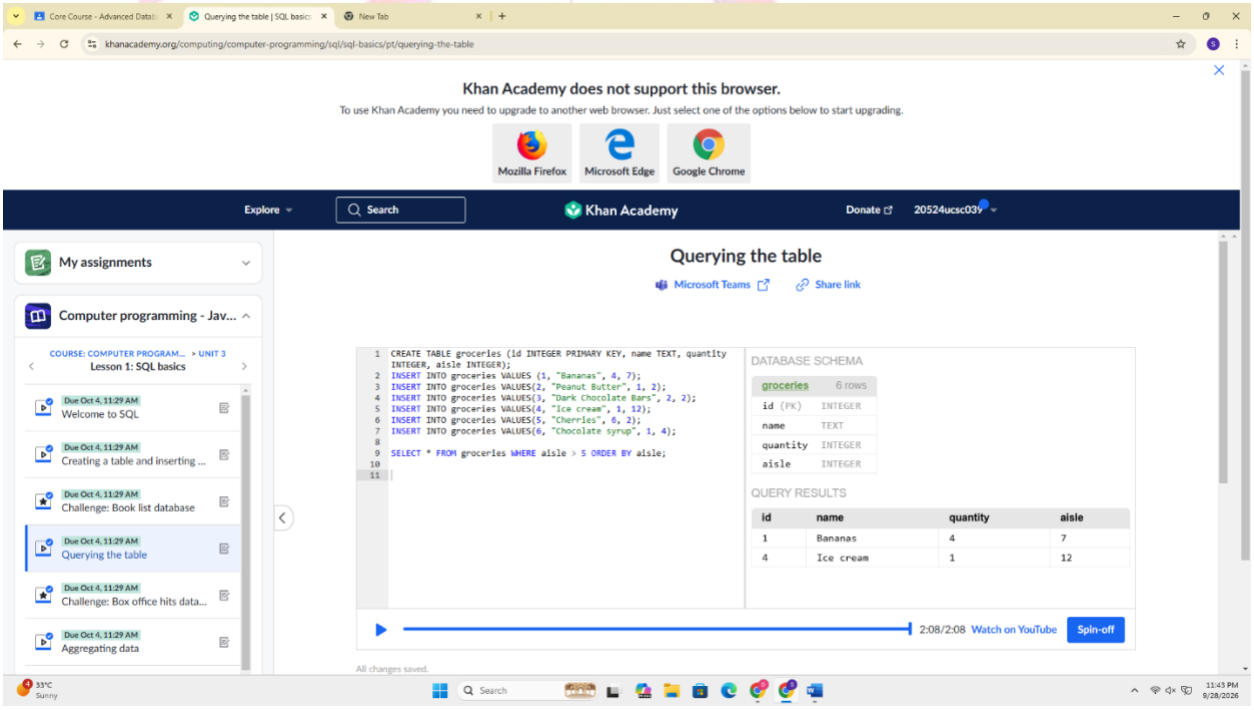
INSERT INTO groceries VALUES(4, "Ice cream", 1, 12);

INSERT INTO groceries VALUES(5, "Cherries", 6, 2);

INSERT INTO groceries VALUES(6, "Chocolate syrup", 1, 4);

SELECT * FROM groceries WHERE aisle > 5 ORDER BY aisle;
```

OUTPUT:



4. BOX OFFICE HITS DATABASE

PROGRAM:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT,
release_year INTEGER);

INSERT INTO movies VALUES (1, "Avatar", 2009);

INSERT INTO movies VALUES (2, "Titanic", 1997);

INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope",
1977);

INSERT INTO movies VALUES (4, "Shrek 2", 2004);

INSERT INTO movies VALUES (5, "The Lion King", 1994);

INSERT INTO movies VALUES (6, "Disney's Up", 2009);

SELECT * FROM movies;

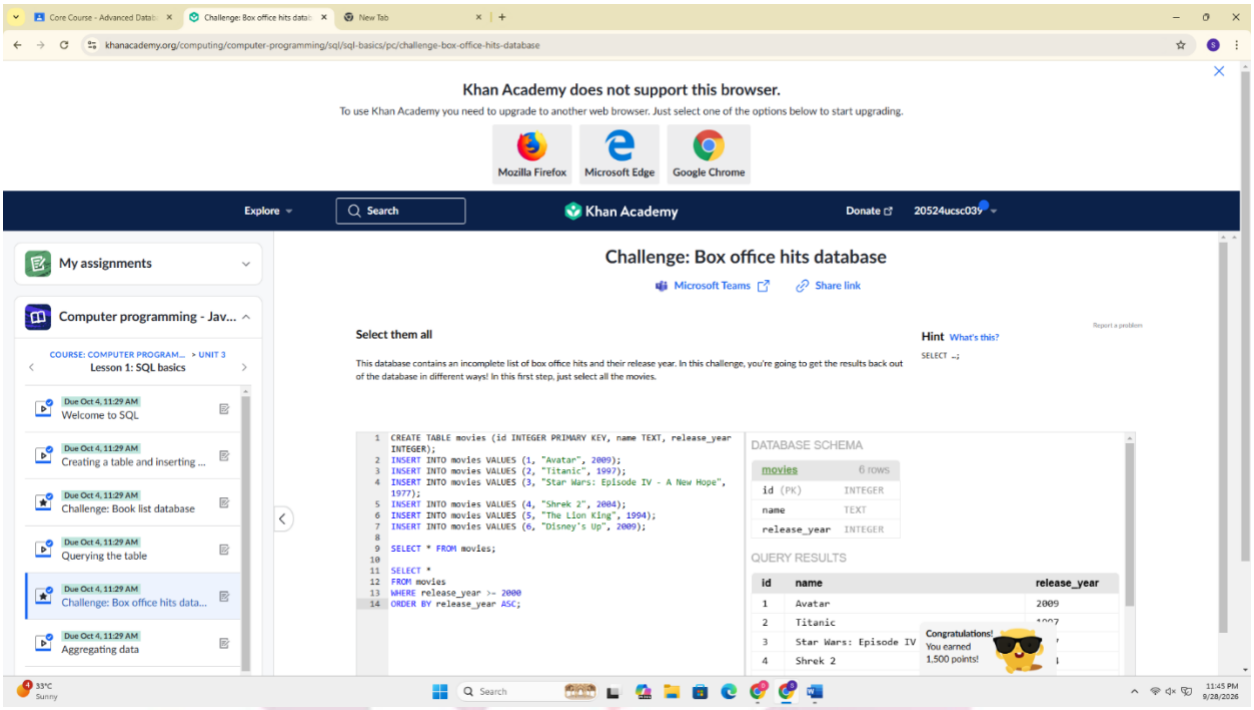
SELECT *

FROM movies

WHERE release_year >= 2000

ORDER BY release_year ASC;
```

OUTPUT:


Khan Academy does not support this browser.
To use Khan Academy you need to upgrade to another web browser. Just select one of the options below to start upgrading.
Mozilla Firefox
Microsoft Edge
Google Chrome
Explore
Search
Khan Academy
Donate
My assignments
Computer programming - Jav...
Lesson 1: SQL basics
Welcome to SQL
Creating a table and inserting ...
Challenge: Book list database
Querying the table
Challenge: Box office hits data...
Aggregating data
Challenge: Box office hits database
Microsoft Teams
Share link
Select them all
This database contains an incomplete list of box office hits and their release year. In this challenge, you're going to get the results back out of the database in different ways! In this first step, just select all the movies.
Hint
What's this?
DATABASE SCHEMA
movies
6 rows
id (PK) INTEGER
name TEXT
release_year INTEGER
QUERY RESULTS
id
name
release_year
1 Avatar 2009
2 Titanic
3 Star Wars: Episode IV
4 Shrek 2
Congratulations!
You earned
1,500 points!

5. AGGREGATING DATA

PROGRAM:

```
CREATE TABLE groceries (id INTEGER PRIMARY KEY, name TEXT, quantity INTEGER, aisle INTEGER);

INSERT INTO groceries VALUES (1, "Bananas", 56, 7);

INSERT INTO groceries VALUES(2, "Peanut Butter", 1, 2);

INSERT INTO groceries VALUES(3, "Dark Chocolate Bars", 2, 2);

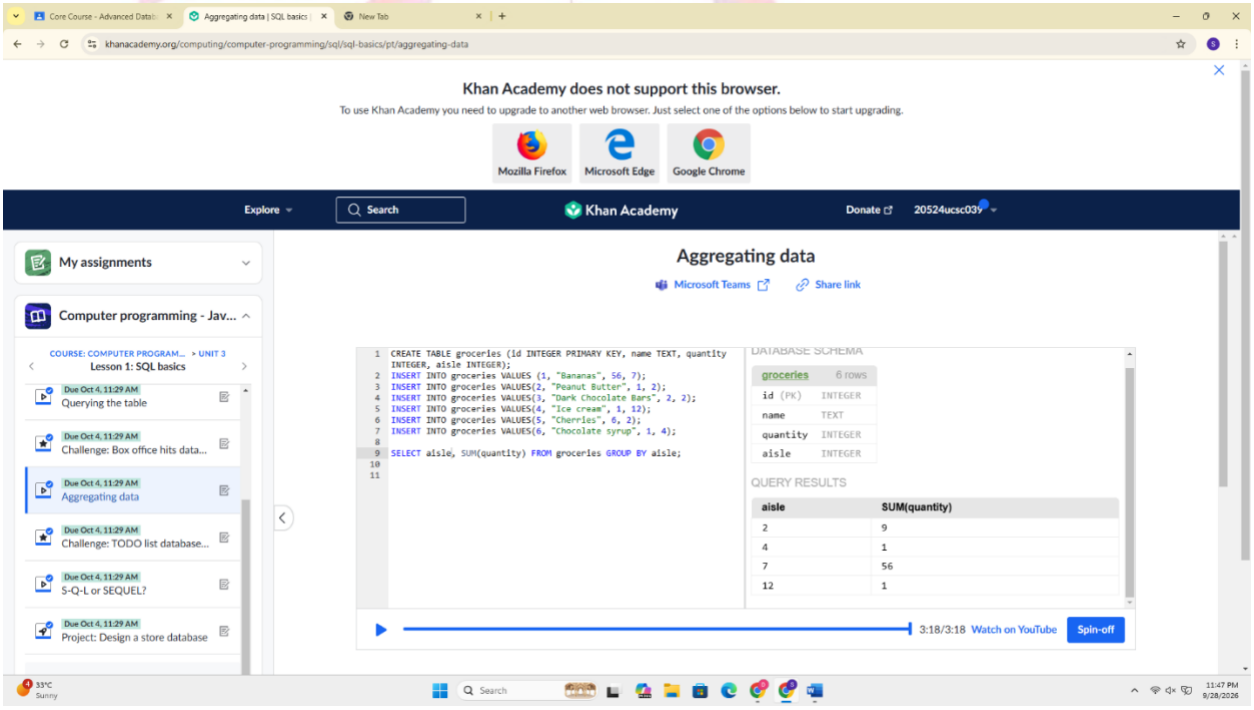
INSERT INTO groceries VALUES(4, "Ice cream", 1, 12);

INSERT INTO groceries VALUES(5, "Cherries", 6, 2);

INSERT INTO groceries VALUES(6, "Chocolate syrup", 1, 4);

SELECT aisle, SUM(quantity) FROM groceries GROUP BY aisle;
```

OUTPUT:



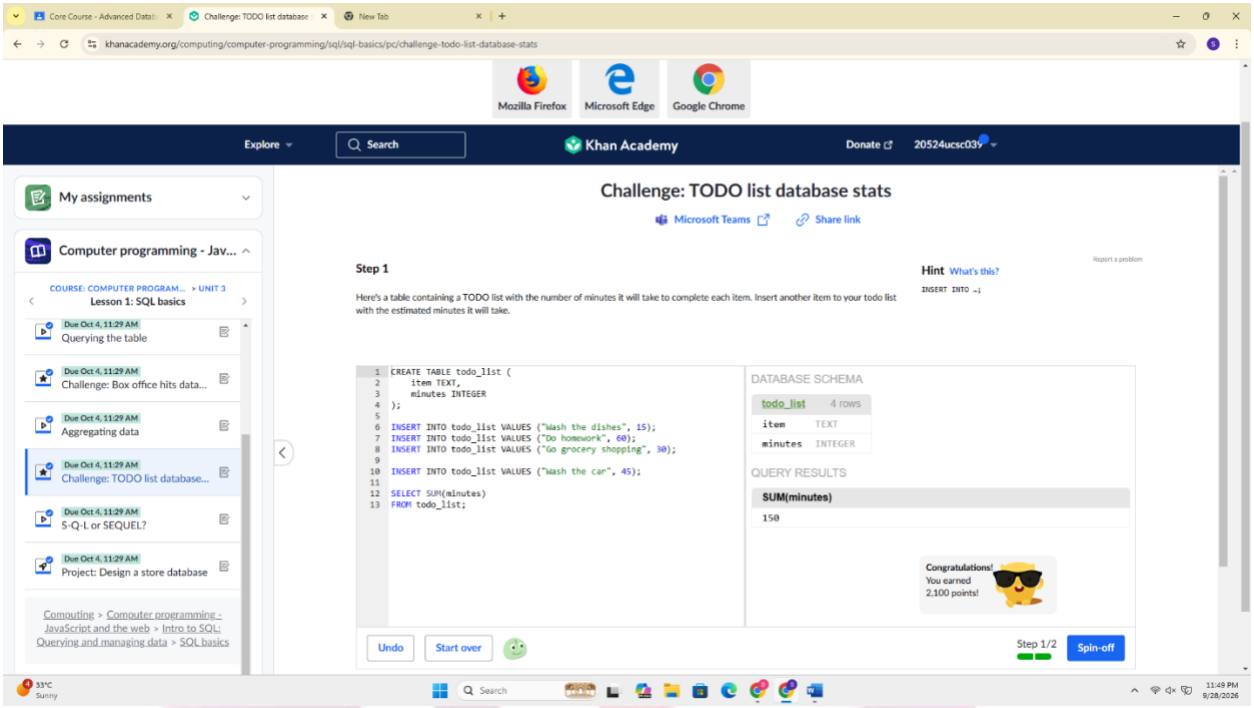
6. TODO LIST DATABASE STATS

PROGRAM:

```
CREATE TABLE todo_list (
   item TEXT,
   minutes INTEGER
);
INSERT INTO todo_list VALUES ("Wash the dishes", 15);
INSERT INTO todo_list VALUES ("Do homework", 60);
INSERT INTO todo_list VALUES ("Go grocery shopping", 30);
INSERT INTO todo_list VALUES ("Wash the car", 45);
SELECT SUM(minutes)
FROM todo_list;
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

OUTPUT:



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

7. DESIGN A STORE DATABASE

PROGRAM:

```
CREATE TABLE store (
   id INTEGER PRIMARY KEY,
   name TEXT,
   category TEXT,
   price REAL,
   quantity INTEGER,
   brand TEXT
);
INSERT INTO store VALUES (1, "Laptop", "Electronics", 65000, 10, "Dell");
INSERT INTO store VALUES (2, "Smartphone", "Electronics", 30000, 20, "Samsung");
INSERT INTO store VALUES (3, "Tablet", "Electronics", 25000, 15, "Apple");
INSERT INTO store VALUES (4, "Smartwatch", "Electronics", 12000, 18, "Noise");
INSERT INTO store VALUES (5, "Headphones", "Accessories", 2500, 30, "Sony");
INSERT INTO store VALUES (6, "Bluetooth Speaker", "Accessories", 3500, 25, "JBL");
INSERT INTO store VALUES (7, "Keyboard", "Accessories", 1500, 40, "Logitech");
INSERT INTO store VALUES (8, "Mouse", "Accessories", 800, 50, "HP");
```

```
INSERT INTO store VALUES (9, "Monitor", "Electronics", 15000, 12, "LG");

INSERT INTO store VALUES (10, "Printer", "Electronics", 9000, 8, "Canon");

INSERT INTO store VALUES (11, "Webcam", "Accessories", 2000, 22,
"Logitech");

INSERT INTO store VALUES (12, "Power Bank", "Accessories", 1800, 35,
"Mi");

INSERT INTO store VALUES (13, "USB Drive", "Accessories", 700, 60,
"SanDisk");

INSERT INTO store VALUES (14, "External Hard Drive", "Accessories", 5500,
14, "Seagate");

INSERT INTO store VALUES (15, "Router", "Networking", 2800, 16, "TP-
Link");

-- Order items by price

SELECT * FROM store

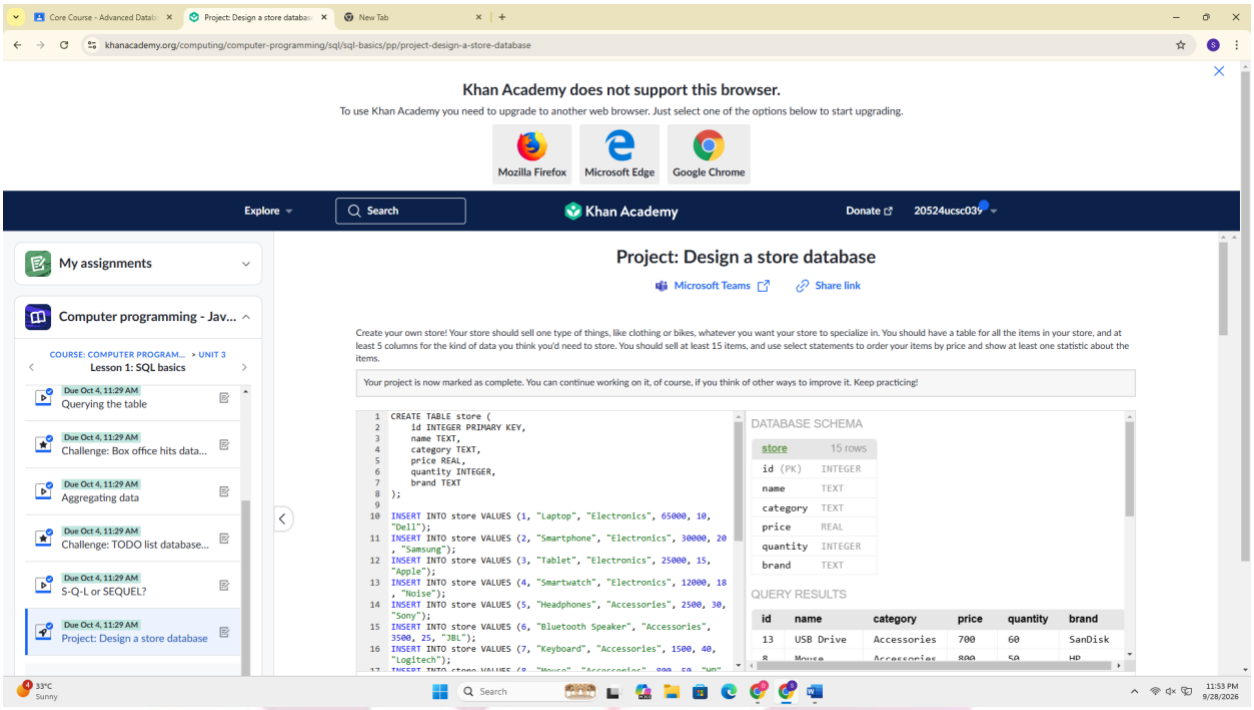
ORDER BY price ASC;

-- Show the average price of all items

SELECT AVG(price) AS average_price

FROM store;
```

OUTPUT:

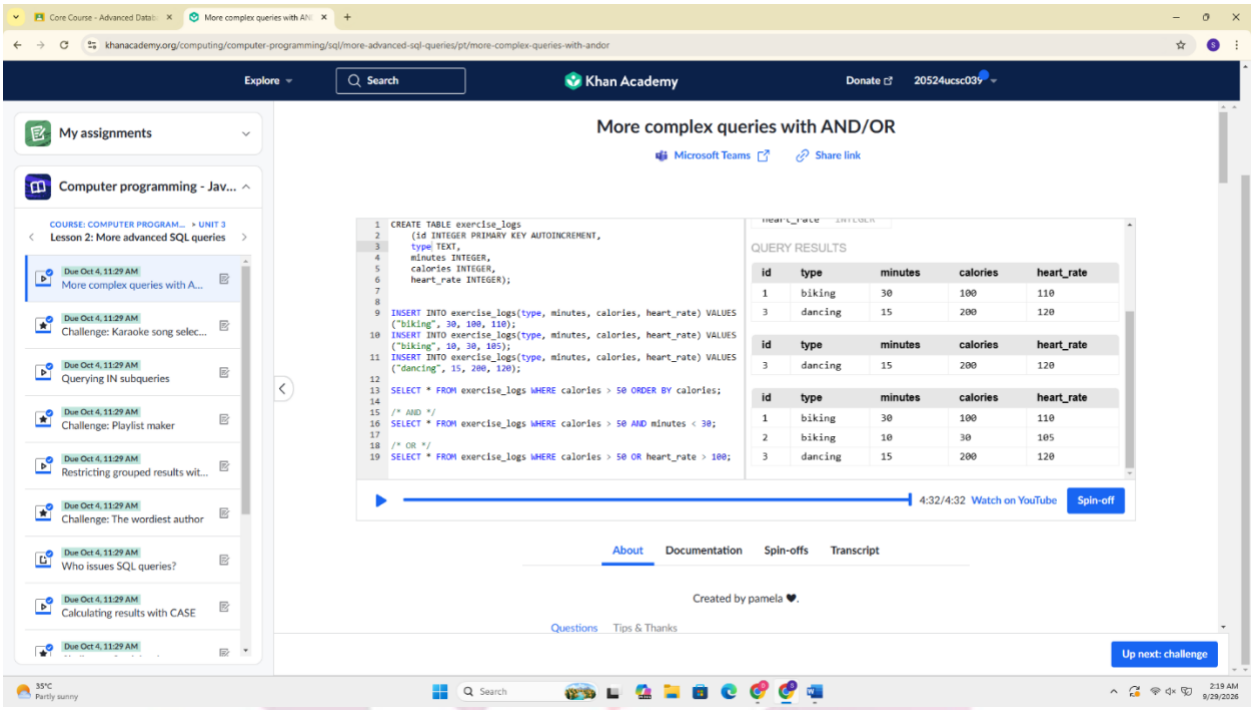

Khan Academy does not support this browser.
To use Khan Academy you need to upgrade to another web browser. Just select one of the options below to start upgrading.
Mozilla Firefox
Microsoft Edge
Google Chrome
Explore
Search
Khan Academy
Donate
20524ucsc039
My assignments
Computer programming - Jav...
COURSE: COMPUTER PROGRAM... > UNIT 3
Lesson 1: SQL basics
Querying the table
Challenge: Box office hits data...
Aggregating data
Challenge: TODO list database...
S-Q-L or SEQUEL?
Project: Design a store database
Project: Design a store database
Microsoft Teams
Share link
Create your own store! Your store should sell one type of things, like clothing or bikes, whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.
Your project is now marked as complete. You can continue working on it, of course, if you think of other ways to improve it. Keep practicing!
CREATE TABLE store (
id INTEGER PRIMARY KEY,
name TEXT,
category TEXT,
price REAL,
quantity INTEGER,
brand TEXT
);
INSERT INTO store VALUES (1, "Laptop", "Electronics", 65000, 10, "Dell");
INSERT INTO store VALUES (2, "Smartphone", "Electronics", 30000, 20, "Samsung");
INSERT INTO store VALUES (3, "Tablet", "Electronics", 25000, 15, "Apple");
INSERT INTO store VALUES (4, "Smartwatch", "Electronics", 12000, 18, "Noise");
INSERT INTO store VALUES (5, "Headphones", "Accessories", 2500, 30, "Sony");
INSERT INTO store VALUES (6, "Bluetooth Speaker", "Accessories", 3500, 25, "JBL");
INSERT INTO store VALUES (7, "Keyboard", "Accessories", 1500, 40, "Logitech");
DATABASE SCHEMA
store 15 rows
id (PK) INTEGER
name TEXT
category TEXT
price REAL
quantity INTEGER
brand TEXT
QUERY RESULTS
id name category price quantity brand
13 USB Drive Accessories 700 60 SanDisk

8. MORE COMPLEX QUERIES WITH AND/OR

PROGRAM:

```
CREATE TABLE exercise_logs
   (id INTEGER PRIMARY KEY AUTOINCREMENT,
   type TEXT,
   minutes INTEGER,
   calories INTEGER,
   heart_rate INTEGER);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("biking", 30, 100, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("biking", 10, 30, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("dancing", 15, 200, 120);
SELECT * FROM exercise_logs WHERE calories > 50 ORDER BY calories;
/* AND */
SELECT * FROM exercise_logs WHERE calories > 50 AND minutes < 30;
/* OR */
SELECT * FROM exercise_logs WHERE calories > 50 OR heart_rate > 100;
```

OUTPUT:



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

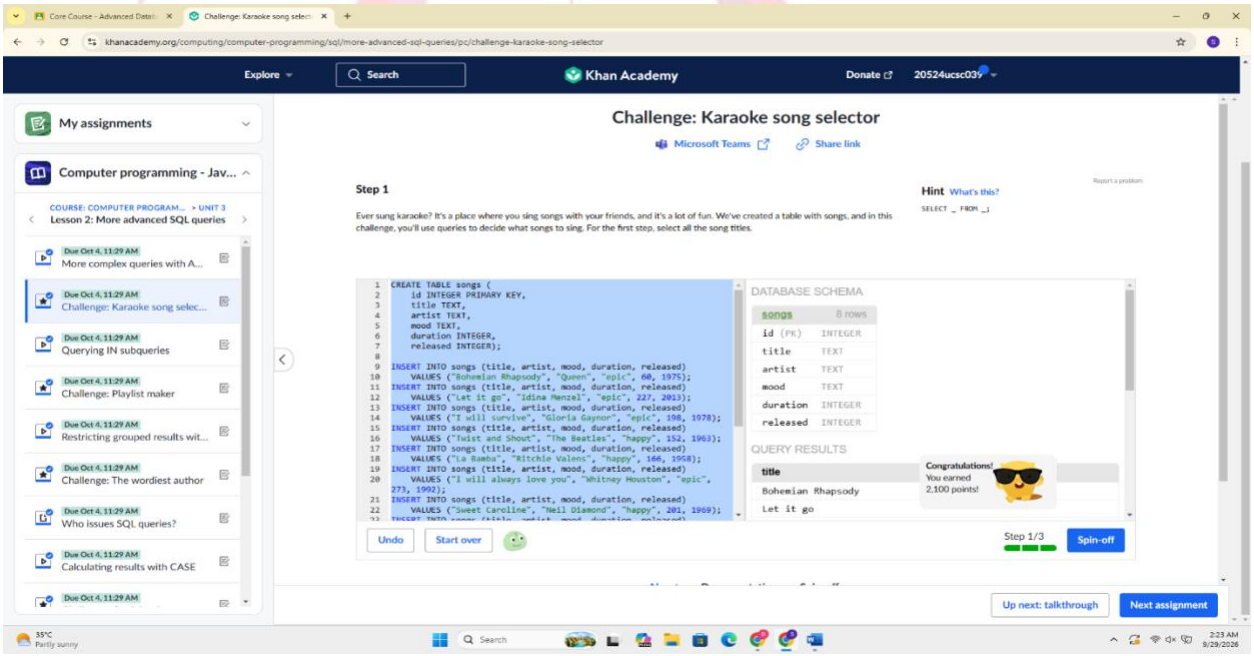
9. KARAOKE SONG SELECTOR

PROGRAM:

```
CREATE TABLE songs (
    id INTEGER PRIMARY KEY,
    title TEXT,
    artist TEXT,
    mood TEXT,
    duration INTEGER,
    released INTEGER);
    INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Let it go", "Idina Menzel", "epic", 227, 2013);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will survive", "Gloria Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Twist and Shout", "The Beatles", "happy", 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("La Bamba", "Ritchie Valens", "happy", 166, 1958);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992);
INSERT INTO songs (title, artist, mood, duration, released)
```

```
 VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);
    SELECT title FROM songs;
SELECT title
FROM songs
WHERE mood = "epic" OR released > 1990;
SELECT title
FROM songs
WHERE mood = "epic"
  AND released > 1990
  AND duration < 240;
```

OUTPUT:



10. QUERYING IN SUBQUERIES

PROGRAM:

```
CREATE TABLE exercise_logs
   (id INTEGER PRIMARY KEY AUTOINCREMENT,
   type TEXT,
   minutes INTEGER,
   calories INTEGER,
   heart_rate INTEGER);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("biking", 30, 100, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("biking", 10, 30, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("dancing", 15, 200, 120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree
climbing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree
climbing", 25, 72, 80);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("rowing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("hiking", 60, 80, 85);
SELECT * FROM exercise_logs WHERE type = "biking" OR type = "hiking" OR
type = "tree climbing" OR type = "rowing";
```

```
/* IN */

SELECT * FROM exercise_logs WHERE type IN ("biking", "hiking", "tree climbing", "rowing");

CREATE TABLE drs_favorites

   (id INTEGER PRIMARY KEY,

   type TEXT,

   reason TEXT);

INSERT INTO drs_favorites(type, reason) VALUES ("biking", "Improves endurance and flexibility.");

INSERT INTO drs_favorites(type, reason) VALUES ("hiking", "Increases cardiovascular health.");

SELECT type FROM drs_favorites;

SELECT * FROM exercise_logs WHERE type IN (

   SELECT type FROM drs_favorites);

   SELECT * FROM exercise_logs WHERE type IN (

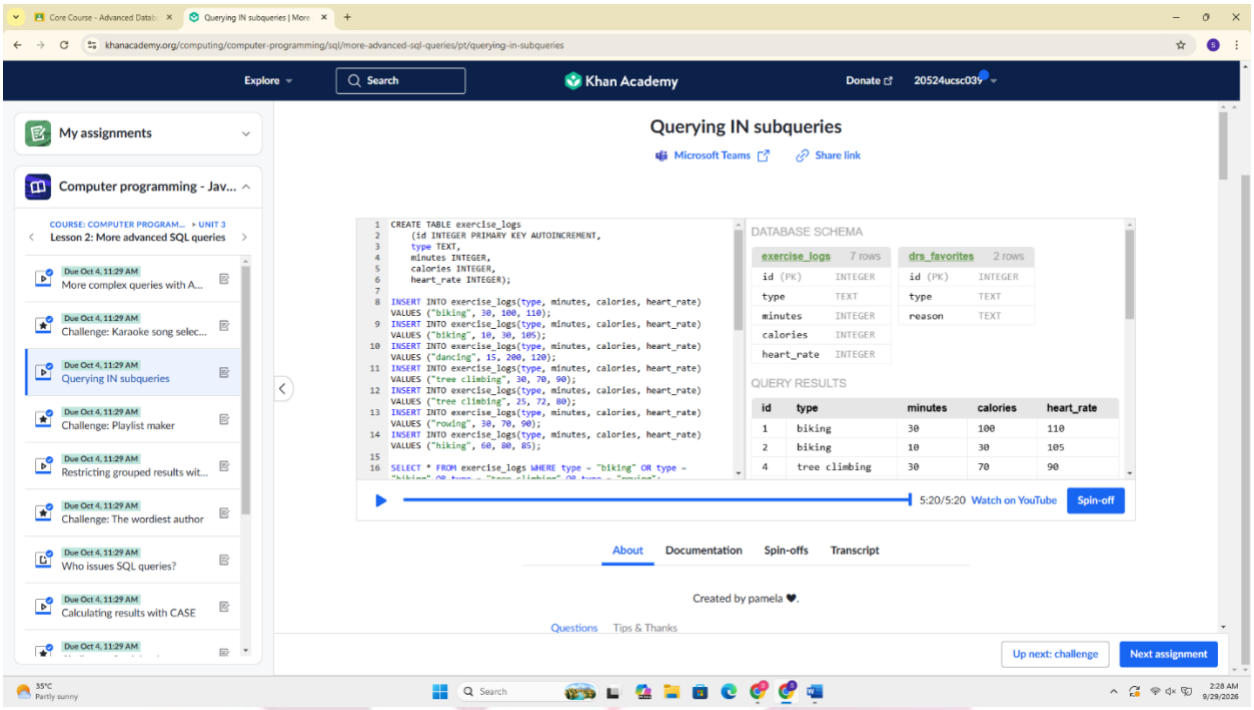
   SELECT type FROM drs_favorites WHERE reason = "Increases cardiovascular health");

   /* LIKE */

SELECT * FROM exercise_logs WHERE type IN (

   SELECT type FROM drs_favorites WHERE reason LIKE "%cardiovascular%");
```

OUTPUT:



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

11. PLAYLIST MAKER

PROGRAM:

```
CREATE TABLE artists (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT,
   country TEXT,
   genre TEXT);
INSERT INTO artists (name, country, genre)
   VALUES ("Taylor Swift", "US", "Pop");
INSERT INTO artists (name, country, genre)
   VALUES ("Led Zeppelin", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
   VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
   VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre)
   VALUES ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre)
   VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
   VALUES ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre)
```

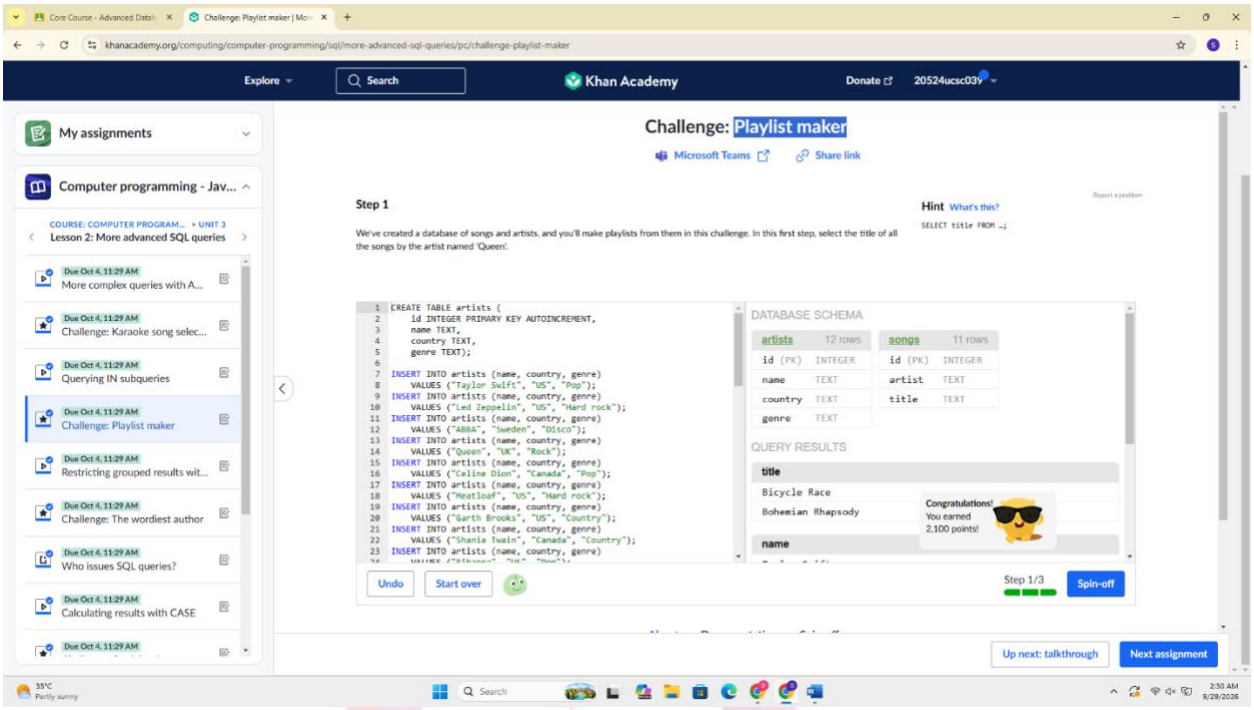
```
    VALUES ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Rihanna", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Guns N' Roses", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Gloria Estefan", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Bob Marley", "Jamaica", "Reggae");
CREATE TABLE songs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    artist TEXT,
    title TEXT);
INSERT INTO songs (artist, title)
    VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title)
    VALUES ("Rihanna", "Stay");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "My heart will go on");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "A new day has come");
INSERT INTO songs (artist, title)
    VALUES ("Shania Twain", "Party for two");
```

TRUTH-TOIL-TRIUMPH AUTONOMOUS

```
INSERT INTO songs (artist, title)
    VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
    VALUES ("Led Zeppelin", "Stairway to heaven");
INSERT INTO songs (artist, title)
    VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bohemian Rhapsody");
INSERT INTO songs (artist, title)
    VALUES ("Guns N' Roses", "Don't cry");
SELECT title
FROM songs
WHERE artist = "Queen";
SELECT name
FROM artists
WHERE genre = "Pop";
SELECT title
FROM songs
WHERE artist IN (
    SELECT name
    FROM artists
```

```
    WHERE genre = "Pop"
);
```

OUTPUT:



12. RESTRICTING GROUPED RESULTS WITH HAVING

PROGRAM:

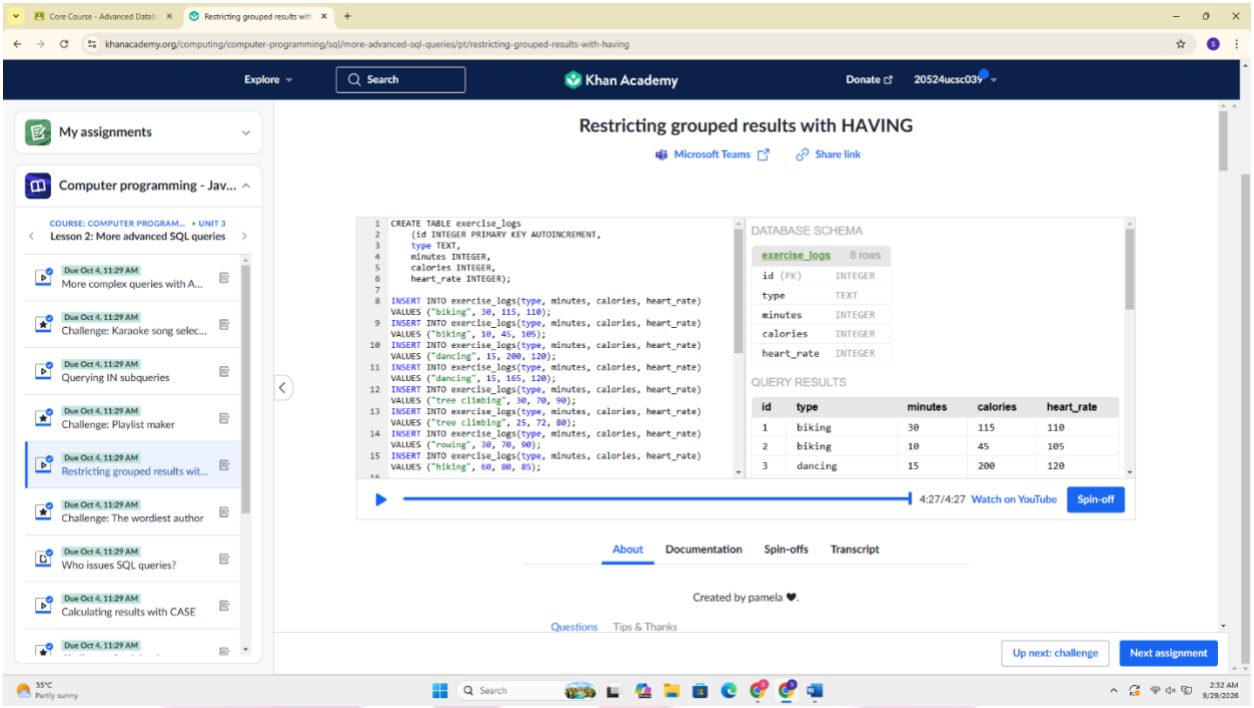
```
CREATE TABLE exercise_logs
   (id INTEGER PRIMARY KEY AUTOINCREMENT,
   type TEXT,
   minutes INTEGER,
   calories INTEGER,
   heart_rate INTEGER);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("biking", 30, 115, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("biking", 10, 45, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("dancing", 15, 200, 120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("dancing", 15, 165, 120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree
climbing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree
climbing", 25, 72, 80);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("rowing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("hiking", 60, 80, 85);
```

```
SELECT * FROM exercise_logs;


SELECT type, SUM(calories) AS total_calories FROM exercise_logs GROUP BY
type;
SELECT type, SUM(calories) AS total_calories FROM exercise_logs
   GROUP BY type
   HAVING total_calories > 150
   ;
SELECT type, AVG(calories) AS avg_calories FROM exercise_logs
   GROUP BY type
   HAVING avg_calories > 70
   ;
SELECT type FROM exercise_logs GROUP BY type HAVING COUNT(*)
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

OUTPUT:



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

13. THE WORDIEST AUTHOR

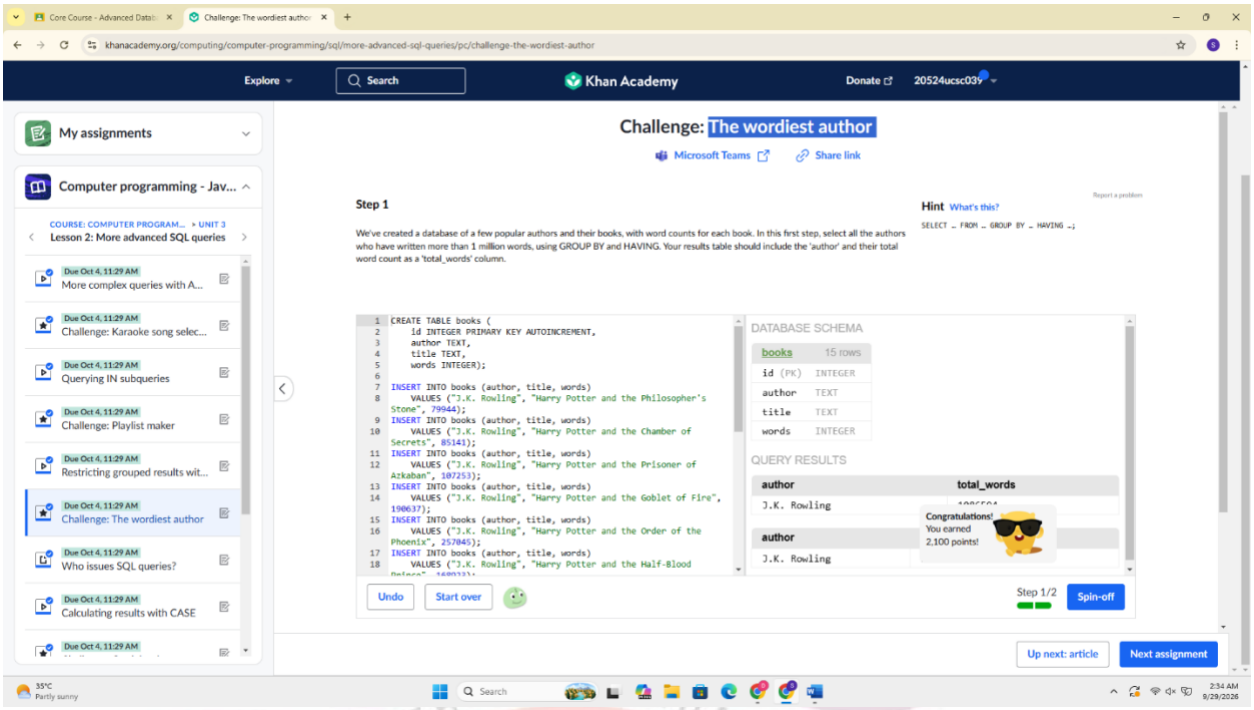
PROGRAM:

```
CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author TEXT,
    title TEXT,
    words INTEGER);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban",
107253);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix",
257045);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);
```

```
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Twilight", 118501);
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Eclipse", 147930);
INSERT INTO books (author, title, words)
   VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
   VALUES ("J.R.R. Tolkien", "Return of the King", 134462);
   SELECT author, SUM(words) AS total_words
FROM books
GROUP BY author
HAVING SUM(words) > 1000000;
SELECT author, AVG(words) AS avg_words
FROM books
GROUP BY author
```

HAVING AVG(words) > 150000;

OUTPUT:



14. CALCULATING RESULTS WITH CASE

PROGRAM:

```
CREATE TABLE exercise_logs
   (id INTEGER PRIMARY KEY AUTOINCREMENT,
   type TEXT,
   minutes INTEGER,
   calories INTEGER,
   heart_rate INTEGER);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("biking", 30, 100, 110);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("biking", 10, 30, 105);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("dancing", 15, 200, 120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("dancing", 15, 165, 120);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree
climbing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES ("tree
climbing", 25, 72, 80);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("rowing", 30, 70, 90);
INSERT INTO exercise_logs(type, minutes, calories, heart_rate) VALUES
("hiking", 60, 80, 85);
```

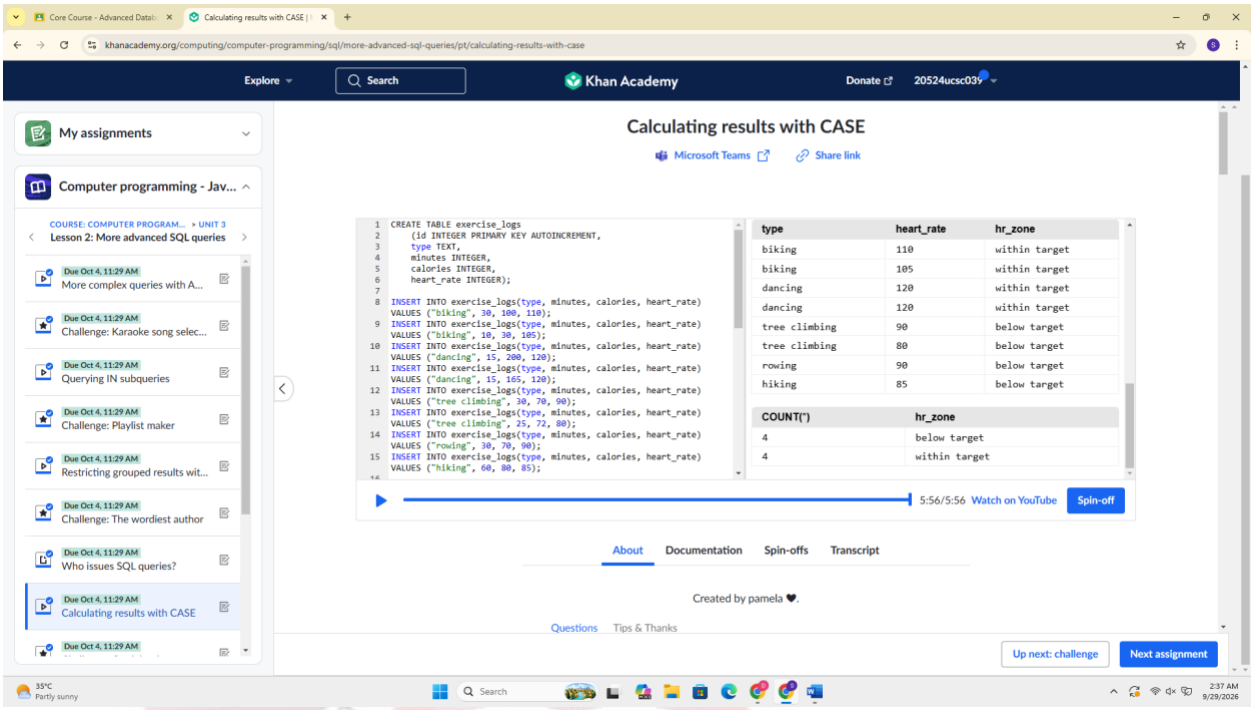
```
SELECT * FROM exercise_logs;


SELECT COUNT(*) FROM exercise_logs WHERE heart_rate > 220 - 30;
/* 50-90% of max*/
SELECT COUNT(*) FROM exercise_logs WHERE
   heart_rate >= ROUND(0.50 * (220-30))
   AND  heart_rate <= ROUND(0.90 * (220-30));
/* CASE */
SELECT type, heart_rate,
   CASE
      WHEN heart_rate > 220-30 THEN "above max"
      WHEN heart_rate > ROUND(0.90 * (220-30)) THEN "above target"
      WHEN heart_rate > ROUND(0.50 * (220-30)) THEN "within target"
      ELSE "below target"
   END as "hr_zone"
FROM exercise_logs;
SELECT COUNT(*),
   CASE
      WHEN heart_rate > 220-30 THEN "above max"
      WHEN heart_rate > ROUND(0.90 * (220-30)) THEN "above target"
      WHEN heart_rate > ROUND(0.50 * (220-30)) THEN "within target"
      ELSE "below target"
   END as "hr_zone"
```

FROM exercise_logs

GROUP BY hr_zone;

OUTPUT:



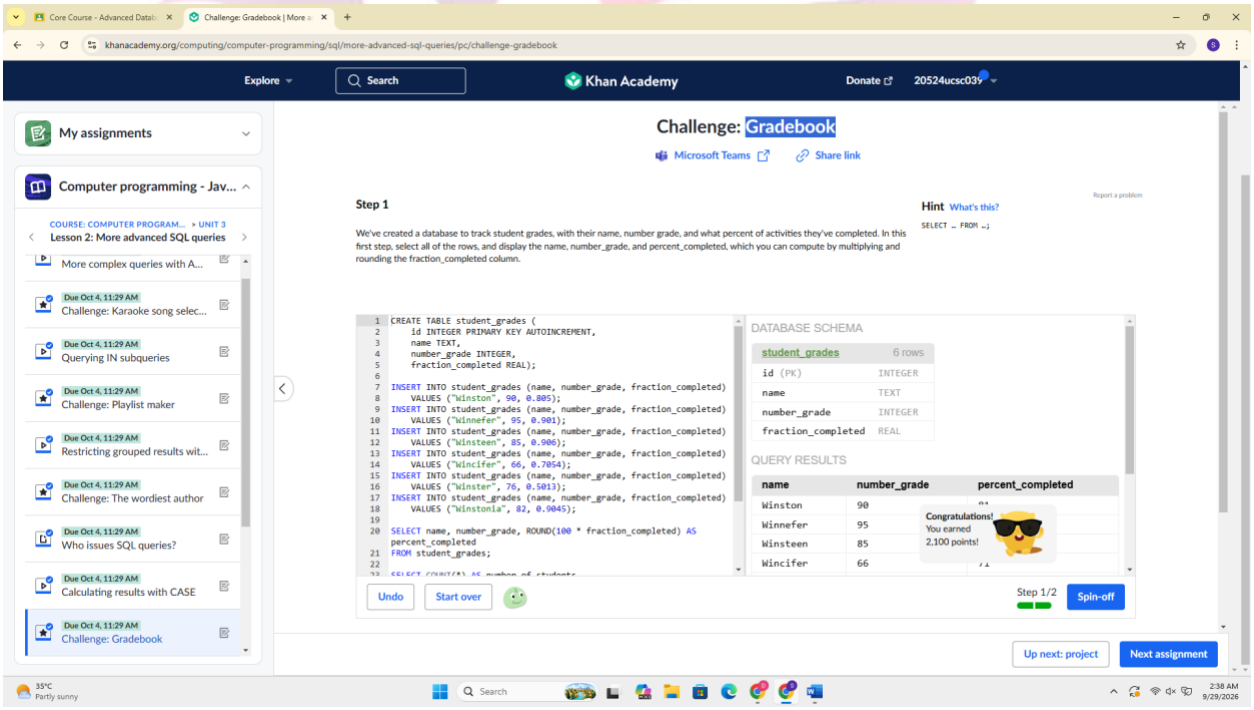
15. GRADEBOOK

PROGRAM:

```
CREATE TABLE student_grades (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    number_grade INTEGER,
    fraction_completed REAL);
    INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winstonia", 82, 0.9045);
    SELECT name, number_grade, ROUND(100 * fraction_completed) AS
percent_completed
FROM student_grades;
```

```
SELECT COUNT(*) AS number_of_students,
CASE
    WHEN number_grade > 90 THEN "A"
    WHEN number_grade > 80 THEN "B"
    WHEN number_grade > 70 THEN "C"
    ELSE "F"
END AS letter_grade
FROM student_grades
GROUP BY letter_grade;
```

OUTPUT:



16. DATA DIG

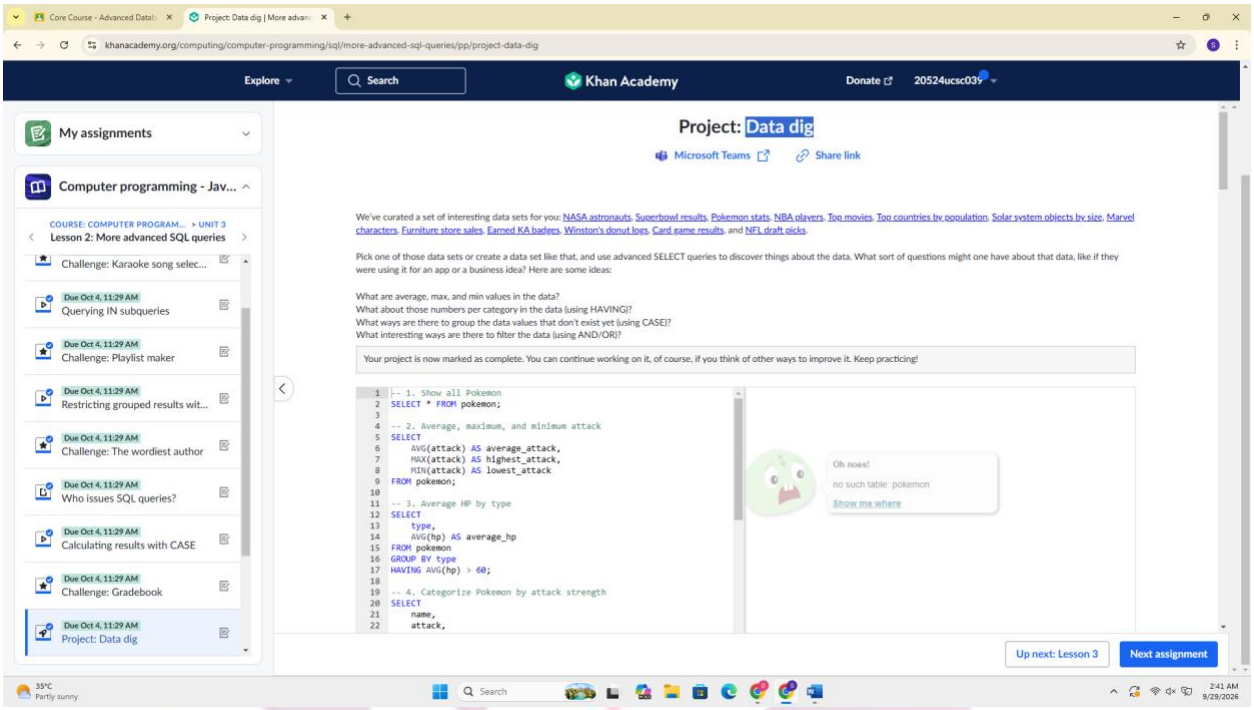
PROGRAM:

```
-- 1. Show all Pokemon
SELECT * FROM pokemon;
-- 2. Average, maximum, and minimum attack
SELECT
    AVG(attack) AS average_attack,
    MAX(attack) AS highest_attack,
    MIN(attack) AS lowest_attack
FROM pokemon;
-- 3. Average HP by type
SELECT
    type,
    AVG(hp) AS average_hp
FROM pokemon
GROUP BY type
HAVING AVG(hp) > 60;
-- 4. Categorize Pokemon by attack strength
SELECT
    name,
    attack,
    CASE
```

```
        WHEN attack >= 100 THEN 'Strong'
        WHEN attack >= 70 THEN 'Medium'
        ELSE 'Weak'
    END AS attack_level
FROM pokemon;
-- 5. Filter Pokemon with high HP and Speed
SELECT
    name,
    hp,
    speed
FROM pokemon
WHERE hp > 70
AND speed > 70;
-- 6. Filter Fire or Water Pokemon
SELECT
    name,
    type
FROM pokemon
WHERE type = 'Fire'
OR type = 'Water';/* Put your data in here and query it! */
```

TRUTH-TOIL-TRIUMPH AUTONOMOUS

OUTPUT:


Explore
Search
Khan Academy
Donate
My assignments
Computer programming - Jav...
Lesson 2: More advanced SQL queries
Challenge: Karaoke song selec...
Querying IN subqueries
Challenge: Playlist maker
Restricting grouped results wit...
Challenge: The wordiest author
Who issues SQL queries?
Calculating results with CASE
Challenge: Gradebook
Project: Data dig
Project: Data dig
Microsoft Teams
Share link
Pick one of those data sets or create a data set like that, and use advanced SELECT queries to discover things about the data. What sort of questions might one have about that data, like if they were using it for an app or a business idea? Here are some ideas:
What are average, max, and min values in the data?
What about those numbers per category in the data (using HAVING)?
What ways are there to group the data values that don't exist yet (using CASE)?
What interesting ways are there to filter the data (using AND/OR)?
Your project is now marked as complete. You can continue working on it, of course, if you think of other ways to improve it. Keep practicing!
Oh noes!
no such table: pokemon
Show me where
Up next: Lesson 3
Next assignment

17. JOINING RELATED TABLES

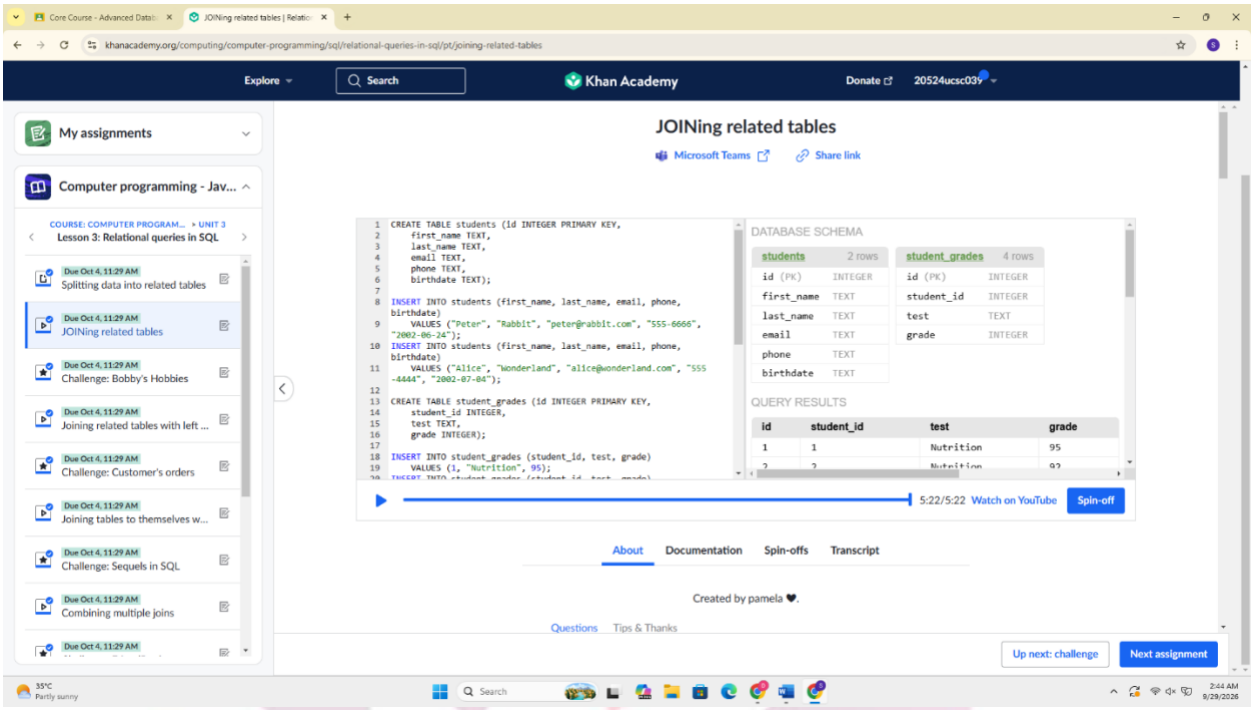
PROGRAM:

```
CREATE TABLE students (id INTEGER PRIMARY KEY,
    first_name TEXT,
    last_name TEXT,
    email TEXT,
    phone TEXT,
    birthdate TEXT);
INSERT INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Peter", "Rabbit", "peter@rabbit.com", "555-6666", "2002-06-24");
INSERT INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Alice", "Wonderland", "alice@wonderland.com", "555-4444",
"2002-07-04");
CREATE TABLE student_grades (id INTEGER PRIMARY KEY,
    student_id INTEGER,
    test TEXT,
    grade INTEGER);
INSERT INTO student_grades (student_id, test, grade)
    VALUES (1, "Nutrition", 95);
INSERT INTO student_grades (student_id, test, grade)
    VALUES (2, "Nutrition", 92);
INSERT INTO student_grades (student_id, test, grade)
```

```
    VALUES (1, "Chemistry", 85);
INSERT INTO student_grades (student_id, test, grade)
    VALUES (2, "Chemistry", 95);
SELECT * FROM student_grades;
/* cross join */
SELECT * FROM student_grades, students;
/* implicit inner join */
SELECT * FROM student_grades, students
    WHERE student_grades.student_id = students.id;
/* explicit inner join - JOIN */
SELECT students.first_name, students.last_name, students.email,
student_grades.test, student_grades.grade FROM students
    JOIN student_grades
    ON students.id = student_grades.student_id
    WHERE grade > 90;
```

AUTONOMOUS

OUTPUT:


JOINing related tables
Microsoft Teams
Share link
My assignments
Computer programming - Jav...
Lesson 3: Relational queries in SQL
Splitting data into related tables
JOINing related tables
Challenge: Bobby's Hobbies
Joining related tables with left ...
Challenge: Customer's orders
Joining tables to themselves w...
Challenge: Sequels in SQL
Combining multiple joins
DATABASE SCHEMA
students
student_grades
QUERY RESULTS
5:22/5:22 Watch on YouTube
Spin-off
About
Documentation
Spin-offs
Transcript
Created by pamela
Questions
Tips & Thanks
Up next: challenge
Next assignment

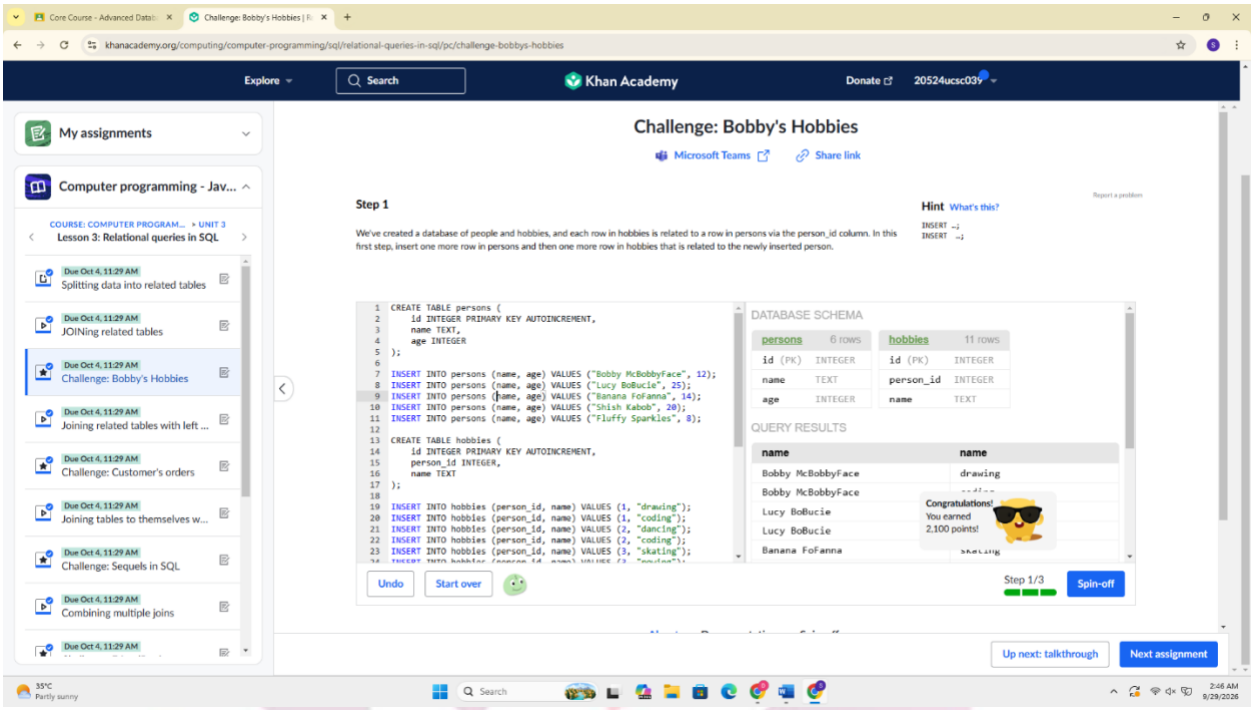
18. BOBBY'S HOBBIES

PROGRAM:

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    age INTEGER
);
INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace", 12);
INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8);
CREATE TABLE hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT
);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
```

```
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
INSERT INTO persons (name, age) VALUES ("Aisha Khan", 20);
INSERT INTO hobbies (person_id, name) VALUES (6, "coding");
SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id
WHERE persons.name = "Bobby McBobbyFace";
```

OUTPUT:


Challenge: Bobby's Hobbies
Step 1
We've created a database of people and hobbies, and each row in hobbies is related to a row in persons via the person_id column. In this first step, insert one more row in persons and then one more row in hobbies that is related to the newly inserted person.
Hint
DATABASE SCHEMA
QUERY RESULTS
Congratulations!
You earned
2,100 points!
Undo
Start over
Step 1/3
Spin-off
Up next: talkthrough
Next assignment

19. JOINING RELATED TABLES WITH LEFT OUTER JOINS

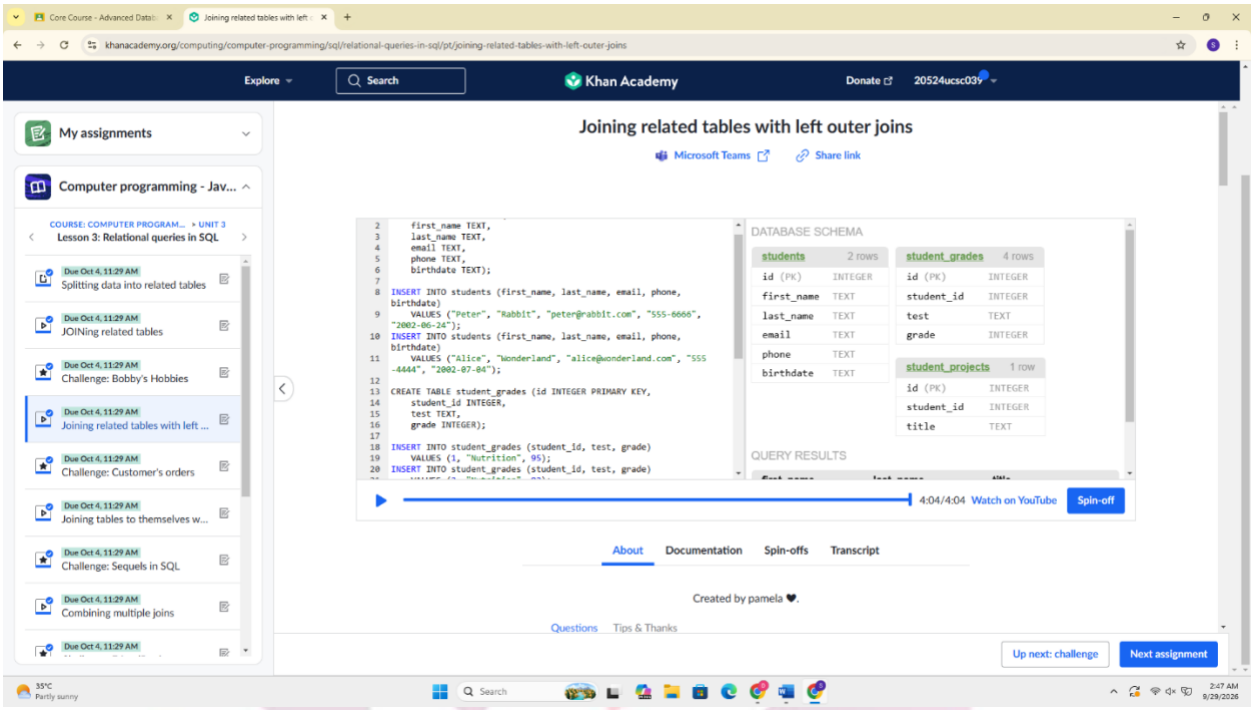
PROGRAM:

```
CREATE TABLE students (id INTEGER PRIMARY KEY,
    first_name TEXT,
    last_name TEXT,
    email TEXT,
    phone TEXT,
    birthdate TEXT);
INSERT INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Peter", "Rabbit", "peter@rabbit.com", "555-6666", "2002-06-24");
INSERT INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Alice", "Wonderland", "alice@wonderland.com", "555-4444",
"2002-07-04");
CREATE TABLE student_grades (id INTEGER PRIMARY KEY,
    student_id INTEGER,
    test TEXT,
    grade INTEGER);
INSERT INTO student_grades (student_id, test, grade)
    VALUES (1, "Nutrition", 95);
INSERT INTO student_grades (student_id, test, grade)
    VALUES (2, "Nutrition", 92);
INSERT INTO student_grades (student_id, test, grade)
```

```
    VALUES (1, "Chemistry", 85);
INSERT INTO student_grades (student_id, test, grade)
    VALUES (2, "Chemistry", 95);
CREATE TABLE student_projects (id INTEGER PRIMARY KEY,
    student_id INTEGER,
    title TEXT);
INSERT INTO student_projects (student_id, title)
    VALUES (1, "Carrotapault");
/* outer join */
SELECT students.first_name, students.last_name, student_projects.title
    FROM students
    LEFT OUTER JOIN student_projects
    ON students.id = student_projects.student_id;
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

OUTPUT:


Joining related tables with left outer joins
Microsoft Teams
Share link
DATABASE SCHEMA
students 2 rows
student_grades 4 rows
student_projects 1 row
QUERY RESULTS
4:04/4:04 Watch on YouTube
Spin-off
About
Documentation
Spin-offs
Transcript
Created by pamela
Questions
Tips & Thanks
Up next: challenge
Next assignment

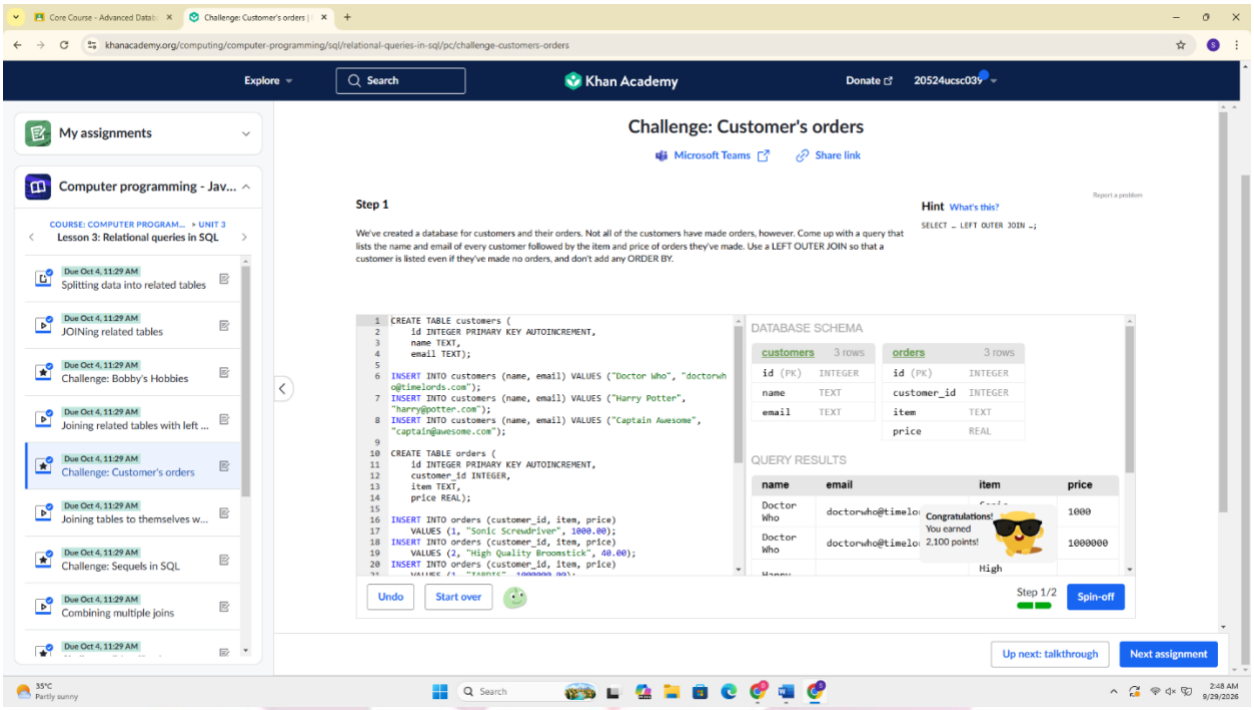
20. CUSTOMER'S ORDERS

PROGRAM:

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT);
INSERT INTO customers (name, email) VALUES ("Doctor Who",
"doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter",
"harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome",
"captain@awesome.com");
CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER,
    item TEXT,
    price REAL);
INSERT INTO orders (customer_id, item, price)
    VALUES (1, "Sonic Screwdriver", 1000.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (2, "High Quality Broomstick", 40.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (1, "TARDIS", 1000000.00);
```

```
    SELECT customers.name,
      customers.email,
      orders.item,
      orders.price
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id;
SELECT
   customers.name,
   customers.email,
   SUM(orders.price) AS total_spent
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id
GROUP BY customers.id
ORDER BY total_spent DESC;
```

OUTPUT:



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

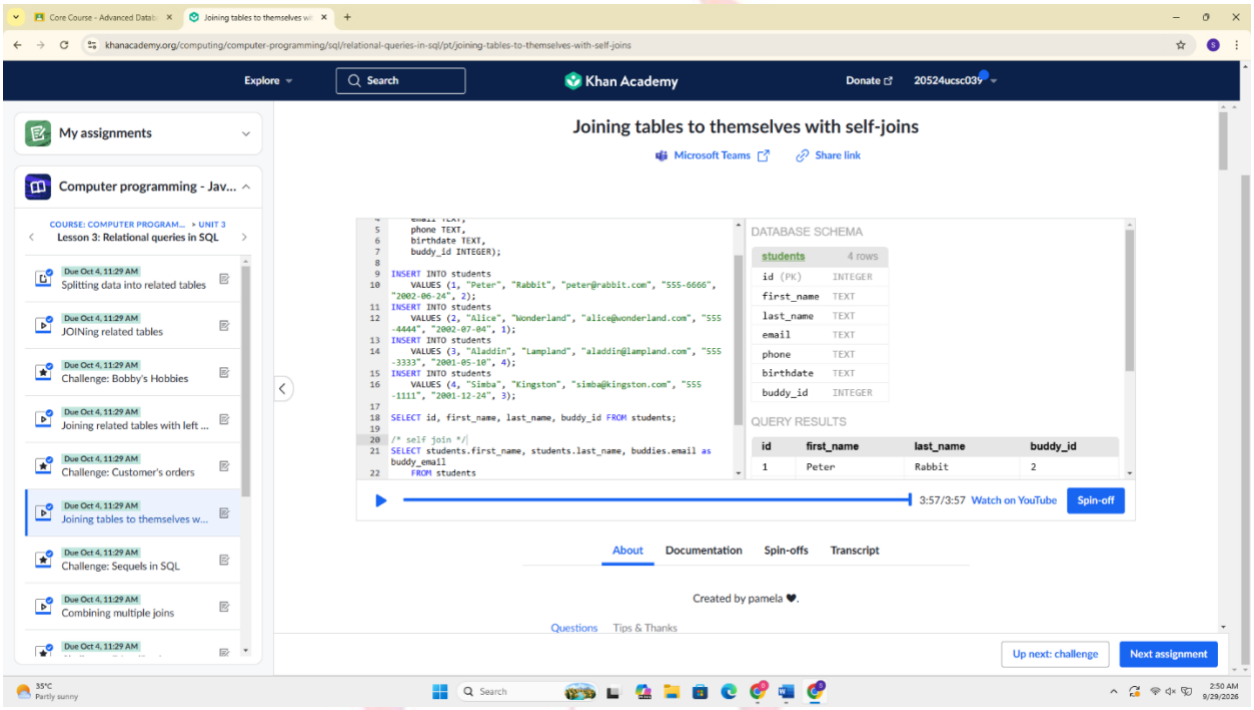
21. JOINING TABLES TO THEMSELVES WITH SELF-JOINS

PROGRAM:

```
CREATE TABLE students (id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT,
    last_name TEXT,
    email TEXT,
    phone TEXT,
    birthdate TEXT,
    buddy_id INTEGER);
INSERT INTO students
    VALUES (1, "Peter", "Rabbit", "peter@rabbit.com", "555-6666", "2002-06-24",
2);
INSERT INTO students
    VALUES (2, "Alice", "Wonderland", "alice@wonderland.com", "555-4444",
"2002-07-04", 1);
INSERT INTO students
    VALUES (3, "Aladdin", "Lampland", "aladdin@lampland.com", "555-3333",
"2001-05-10", 4);
INSERT INTO students
    VALUES (4, "Simba", "Kingston", "simba@kingston.com", "555-1111", "2001-
12-24", 3);
SELECT id, first_name, last_name, buddy_id FROM students;
/* self join */
```

```
SELECT students.first_name, students.last_name, buddies.email as buddy_email
    FROM students
    JOIN students buddies
    ON students.buddy_id = buddies.id;
```

OUTPUT:



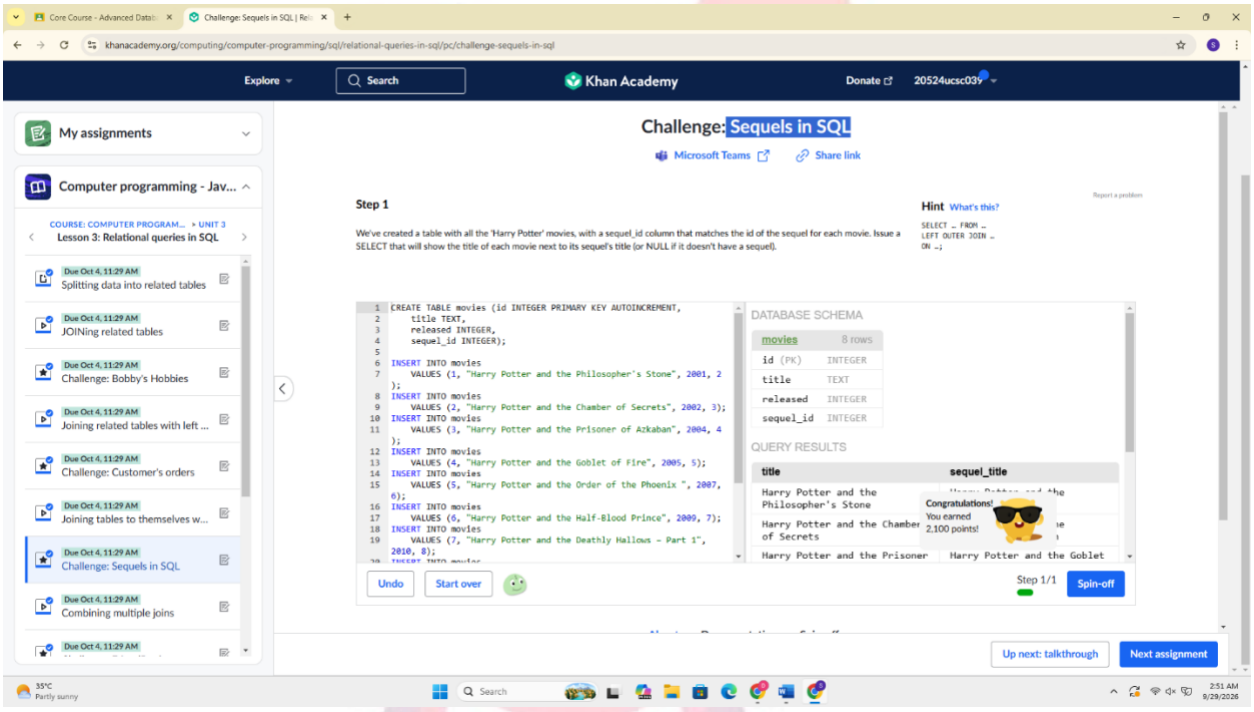
22. SEQUELS IN SQL

PROGRAM:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY AUTOINCREMENT,
   title TEXT,
   released INTEGER,
   sequel_id INTEGER);
INSERT INTO movies
   VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2);
INSERT INTO movies
   VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3);
INSERT INTO movies
   VALUES (3, "Harry Potter and the Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
   VALUES (4, "Harry Potter and the Goblet of Fire", 2005, 5);
INSERT INTO movies
   VALUES (5, "Harry Potter and the Order of the Phoenix ", 2007, 6);
INSERT INTO movies
   VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7);
INSERT INTO movies
   VALUES (7, "Harry Potter and the Deathly Hallows – Part 1", 2010, 8);
INSERT INTO movies
   VALUES (8, "Harry Potter and the Deathly Hallows – Part 2", 2011, NULL);
```

```
    SELECT
    m1.title,
    m2.title AS sequel_title
FROM movies AS m1
LEFT OUTER JOIN movies AS m2
ON m1.sequel_id = m2.id;
```

OUTPUT:



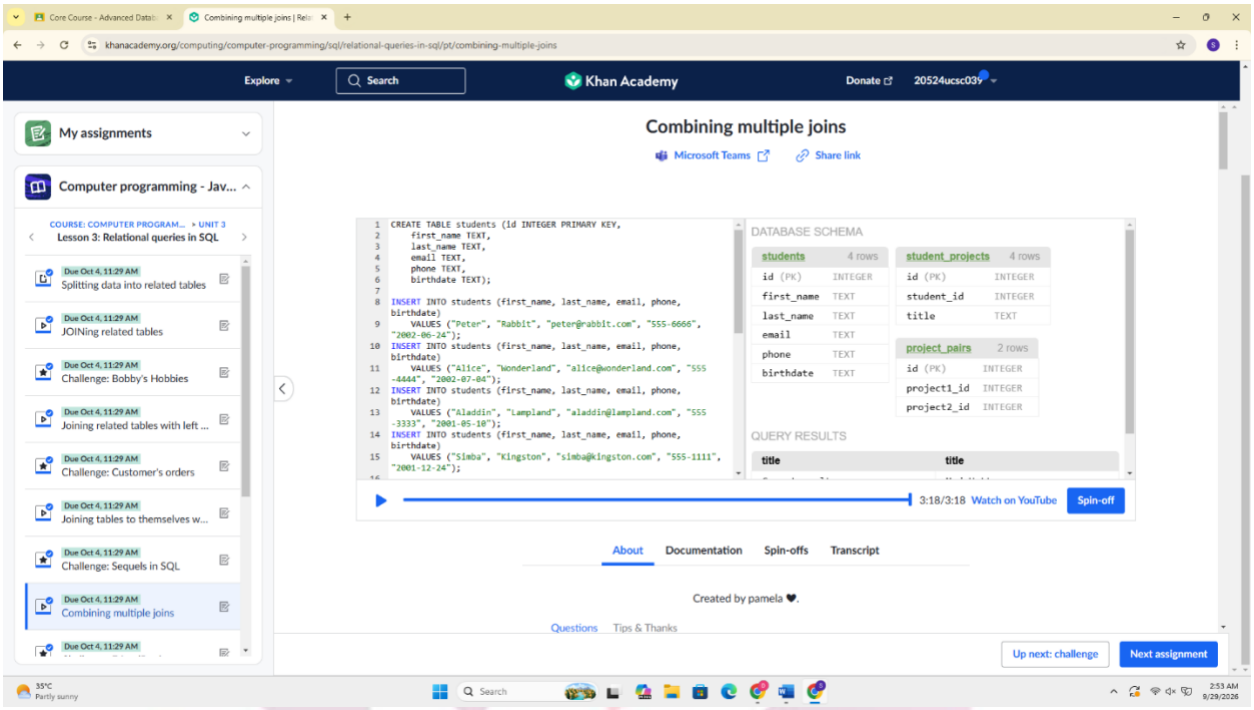
23. COMBINING MULTIPLE JOINS

PROGRAM:

```
CREATE TABLE students (id INTEGER PRIMARY KEY,
    first_name TEXT,
    last_name TEXT,
    email TEXT,
    phone TEXT,
    birthdate TEXT);
INSERT INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Peter", "Rabbit", "peter@rabbit.com", "555-6666", "2002-06-24");
INSERT INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Alice", "Wonderland", "alice@wonderland.com", "555-4444",
"2002-07-04");
INSERT INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Aladdin", "Lampland", "aladdin@lampland.com", "555-3333",
"2001-05-10");
INSERT INTO students (first_name, last_name, email, phone, birthdate)
    VALUES ("Simba", "Kingston", "simba@kingston.com", "555-1111", "2001-
12-24");
CREATE TABLE student_projects (id INTEGER PRIMARY KEY,
    student_id INTEGER,
    title TEXT);
INSERT INTO student_projects (student_id, title)
```

```
    VALUES (1, "Carrotapault");
INSERT INTO student_projects (student_id, title)
    VALUES (2, "Mad Hattery");
INSERT INTO student_projects (student_id, title)
    VALUES (3, "Carpet Physics");
INSERT INTO student_projects (student_id, title)
    VALUES (4, "Hyena Habitats");
CREATE TABLE project_pairs (id INTEGER PRIMARY KEY,
    project1_id INTEGER,
    project2_id INTEGER);
INSERT INTO project_pairs (project1_id, project2_id)
    VALUES(1, 2);
INSERT INTO project_pairs (project1_id, project2_id)
    VALUES(3, 4);
SELECT a.title, b.title FROM project_pairs
    JOIN student_projects a
    ON project_pairs.project1_id = a.id
    JOIN student_projects b
    ON project_pairs.project2_id = b.id;
```

OUTPUT:



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

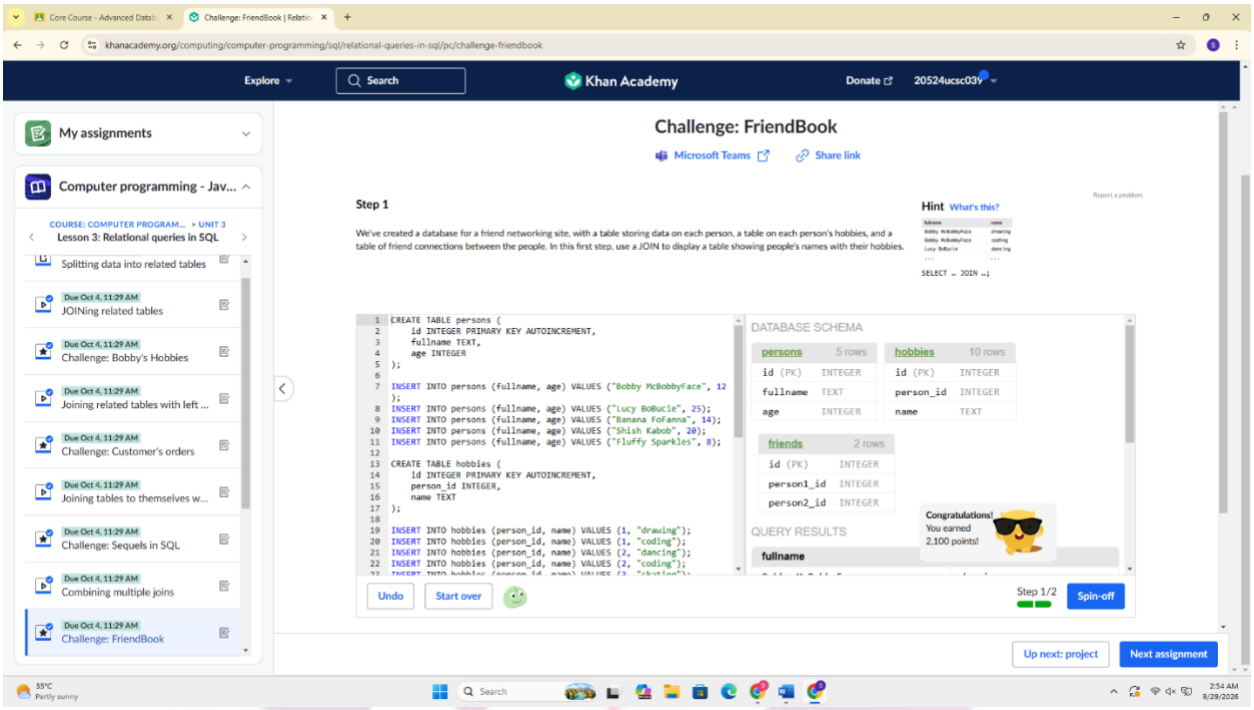
24. FRIENDBOOK

PROGRAM:

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    fullname TEXT,
    age INTEGER
);
INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", 12);
INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", 25);
INSERT INTO persons (fullname, age) VALUES ("Banana FoFanna", 14);
INSERT INTO persons (fullname, age) VALUES ("Shish Kabob", 20);
INSERT INTO persons (fullname, age) VALUES ("Fluffy Sparkles", 8);
CREATE TABLE hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT
);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
```

```
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
CREATE TABLE friends (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person1_id INTEGER,
    person2_id INTEGER
);
INSERT INTO friends (person1_id, person2_id) VALUES (1, 4);
INSERT INTO friends (person1_id, person2_id) VALUES (2, 3);
SELECT persons.fullname, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT a.fullname, b.fullname
FROM friends
JOIN persons a
ON friends.person1_id = a.id
JOIN persons b
ON friends.person2_id = b.id;
```

OUTPUT:


Challenge: FriendBook
Step 1
We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a JOIN to display a table showing people's names with their hobbies.
Hint
DATABASE SCHEMA
persons 5 rows
hobbies 10 rows
friends 2 rows
QUERY RESULTS
fullname
Congratulations!
You earned
2,100 points!
Undo
Start over
Step 1/2
Spin-off
Up next: project
Next assignment

25. FAMOUS PEOPLE

PROGRAM:

```
-- Create the famous_people table
CREATE TABLE famous_people (
    id INTEGER PRIMARY KEY,
    name TEXT,
    nationality TEXT
);
INSERT INTO famous_people VALUES
(1, 'Leonardo DiCaprio', 'American'),
(2, 'Emma Watson', 'British'),
(3, 'Tom Hanks', 'American'),
(4, 'Scarlett Johansson', 'American');
-- Create the movies table
CREATE TABLE movies (
    id INTEGER PRIMARY KEY,
    title TEXT,
    actor_id INTEGER
)
INSERT INTO movies VALUES
(1, 'Titanic', 1),
(2, 'Inception', 1),
```

(3, 'Harry Potter', 2),

(4, 'Forrest Gump', 3),

(5, 'Avengers: Endgame', 4);

-- Display actors and the movies they acted in

SELECT

famous_people.name,

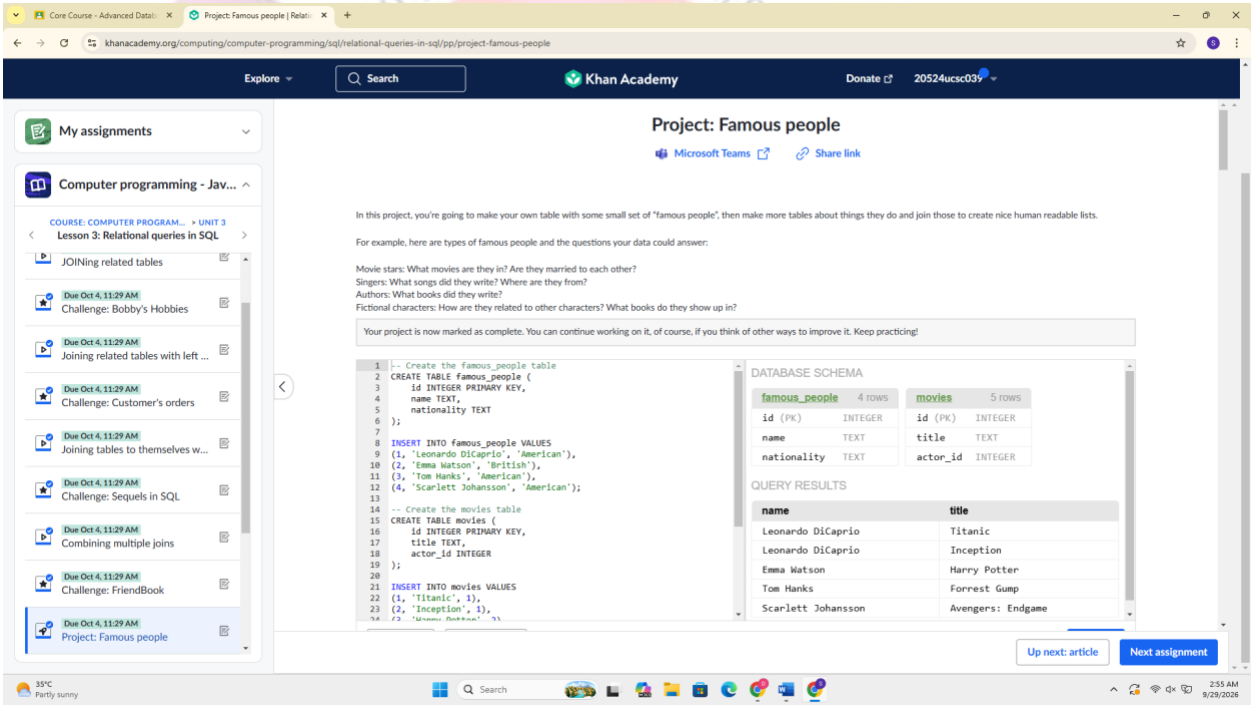
movies.title

FROM famous_people

JOIN movies

ON famous_people.id = movies.actor_id;/* Create table about the people and what they do here */

OUTPUT:



1. 15 DAYS OF LEARNING SQL

PROGRAM

```
WITH RECURSIVE dates AS (
    SELECT DATE('2016-03-01') AS submission_date
    UNION ALL
    SELECT submission_date + INTERVAL 1 DAY
    FROM dates
    WHERE submission_date < '2016-03-15'
),

daily_hackers AS (
    SELECT
        s.submission_date,
        s.hacker_id,
        COUNT(*) AS submissions_count
    FROM Submissions s
    WHERE s.submission_date BETWEEN '2016-03-01' AND '2016-03-15'
    GROUP BY s.submission_date, s.hacker_id
),

consistent_hackers AS (
    SELECT
        submission_date,
        COUNT(DISTINCT hacker_id) AS hacker_count
```

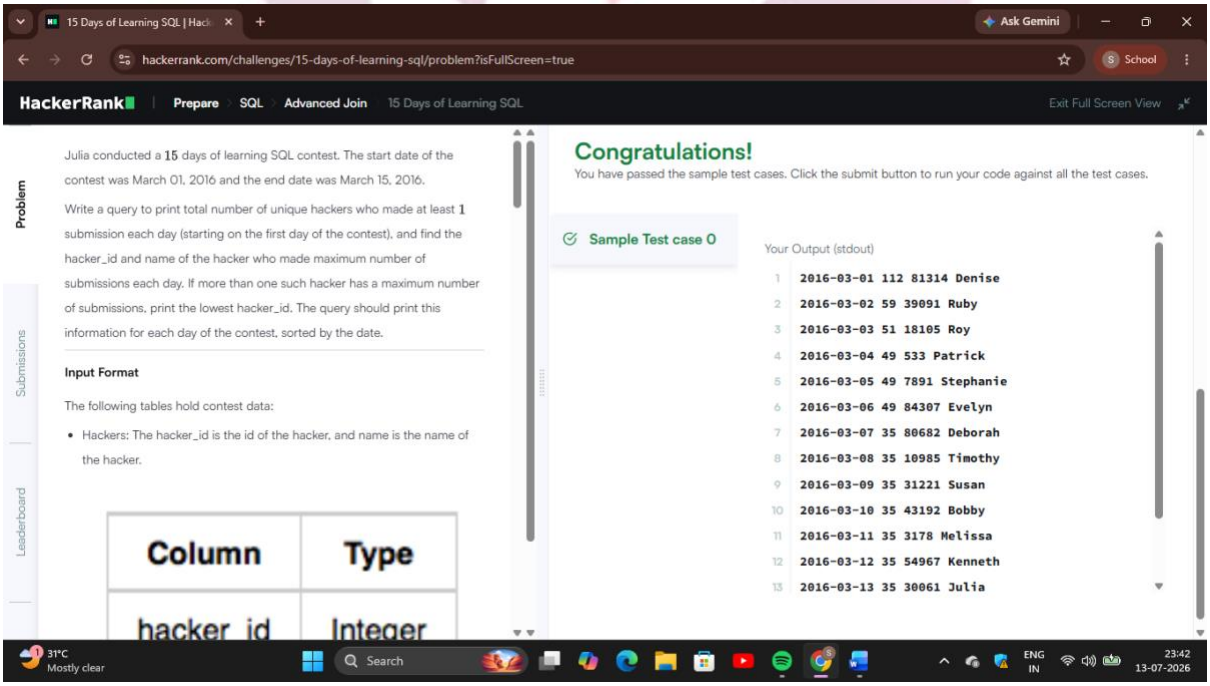
```
    FROM daily_hackers dh1
    WHERE (
        SELECT COUNT(DISTINCT dh2.submission_date)
        FROM daily_hackers dh2
        WHERE dh2.hacker_id = dh1.hacker_id
        AND dh2.submission_date <= dh1.submission_date
    ) = DATEDIFF(dh1.submission_date, '2016-03-01') + 1
    GROUP BY submission_date
),

max_submissions AS (
    SELECT
        submission_date,
        hacker_id,
        submissions_count,
        ROW_NUMBER() OVER (
            PARTITION BY submission_date
            ORDER BY submissions_count DESC, hacker_id
        ) AS rn
    FROM daily_hackers
)

SELECT
    d.submission_date,
    COALESCE(c.hacker_count, 0),
    m.hacker_id,
```

```
    h.name
FROM dates d
LEFT JOIN consistent_hackers c
    ON d.submission_date = c.submission_date
JOIN max_submissions m
    ON d.submission_date = m.submission_date
    AND m.rn = 1
JOIN Hackers h
    ON m.hacker_id = h.hacker_id
ORDER BY d.submission_date;
```

OUTPUT:

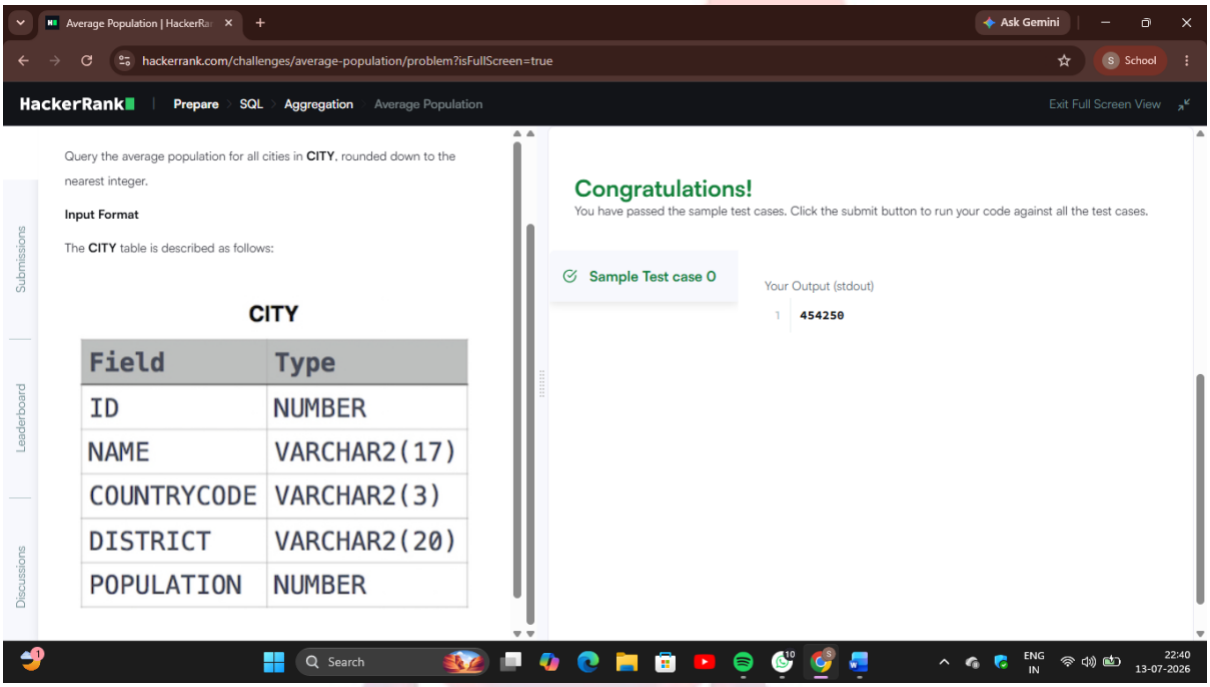


2.AVERAGE POPULATION

PROGRAM

```
SELECT FLOOR(AVG(POPULATION))
FROM CITY;
```

OUTPUT:

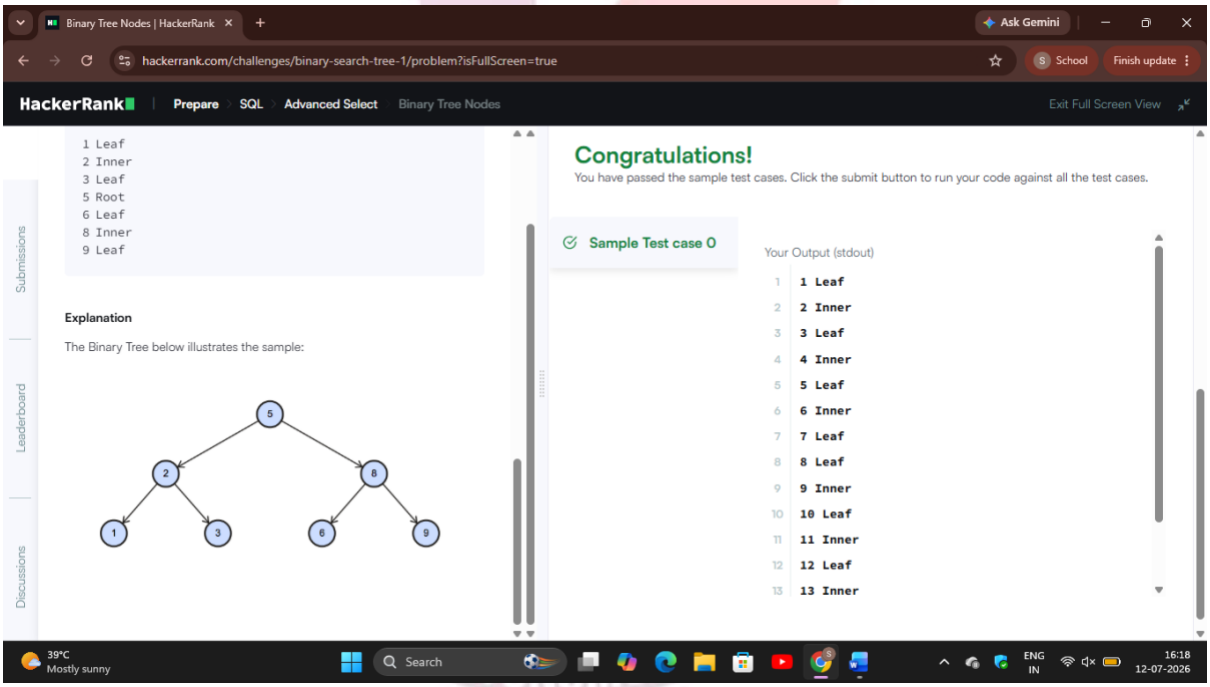


3. BINARY TREE NODES

PROGRAM

```
SELECT
   N,
   CASE
      WHEN P IS NULL THEN 'Root'
      WHEN N NOT IN (SELECT P FROM BST WHERE P IS NOT NULL) THEN 'Leaf'
      ELSE 'Inner'
   END AS Node_Type
FROM BST
ORDER BY N;
```

OUTPUT:



4.CHALLENGES

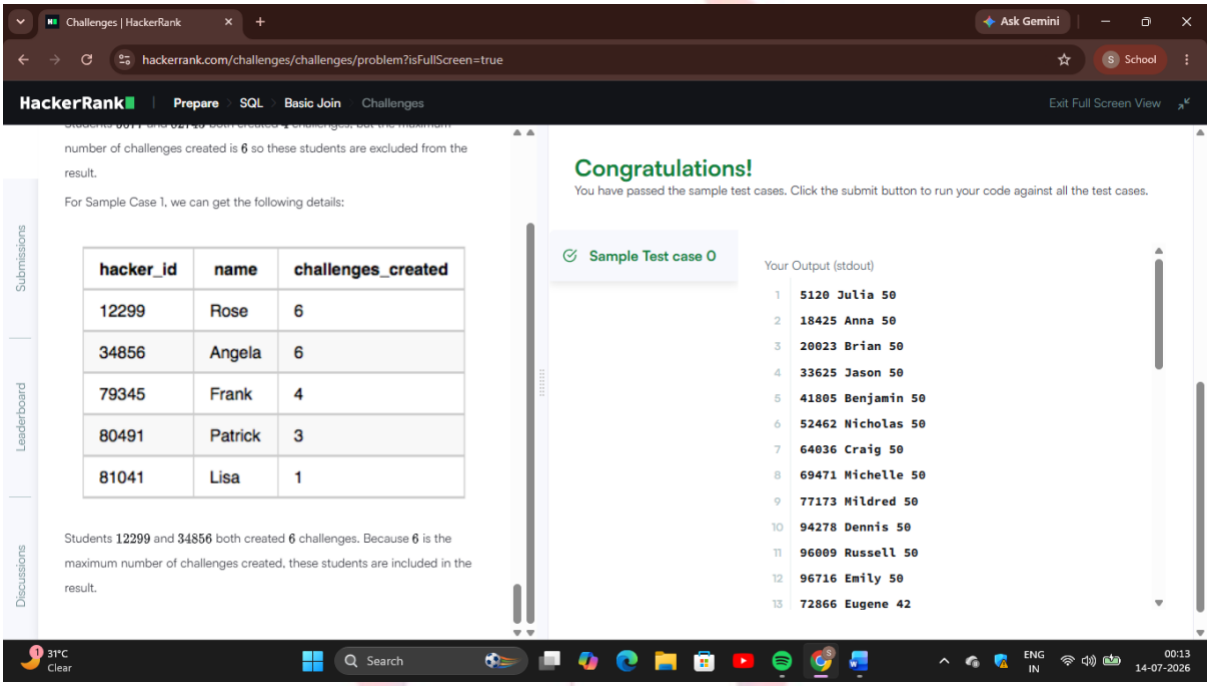
PROGRAM

```
SELECT h.hacker_id,
       h.name,
       COUNT(c.challenge_id) AS total_challenges
FROM Hackers h
JOIN Challenges c
ON h.hacker_id = c.hacker_id
GROUP BY h.hacker_id, h.name
HAVING total_challenges = (
    SELECT MAX(challenge_count)
    FROM (
        SELECT COUNT(challenge_id) AS challenge_count
        FROM Challenges
        GROUP BY hacker_id
    ) AS counts
)
OR total_challenges IN (
    SELECT challenge_count
    FROM (
        SELECT COUNT(challenge_id) AS challenge_count
        FROM Challenges
        GROUP BY hacker_id
    ) AS counts
    GROUP BY challenge_count
    HAVING COUNT(*) = 1
```

)

ORDER BY total_challenges DESC, h.hacker_id;

OUTPUT:

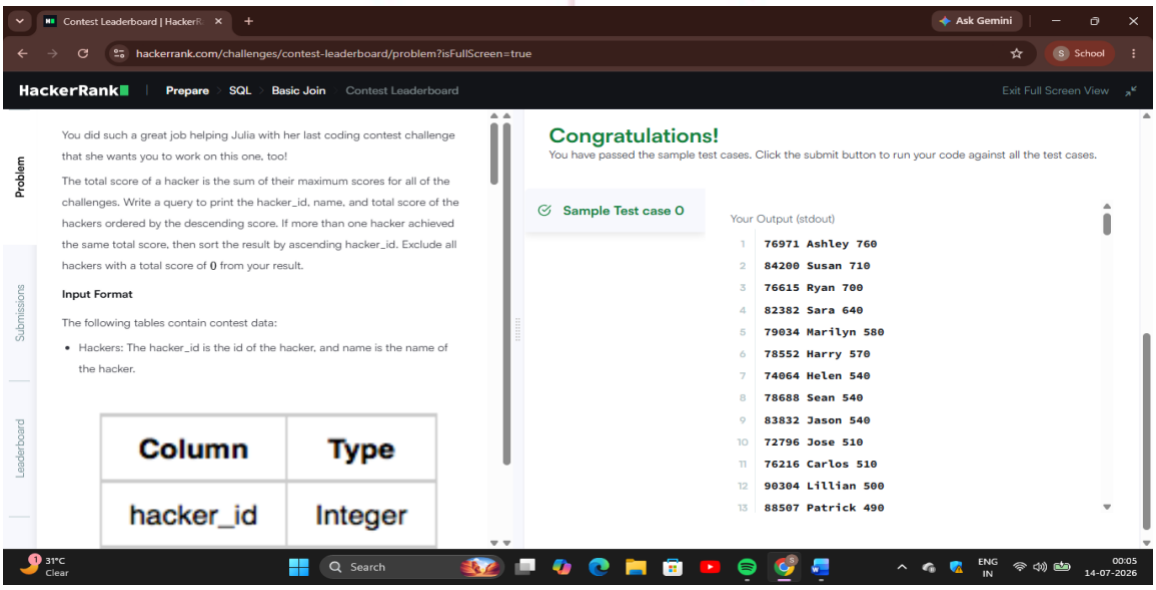


5.CONTEST LEADERBOARD

PROGRAM

```
SELECT h.hacker_id, h.name, SUM(ms.max_score) AS total_score
FROM Hackers h
JOIN (
    SELECT hacker_id, challenge_id, MAX(score) AS max_score
    FROM Submissions
    GROUP BY hacker_id, challenge_id
) ms
ON h.hacker_id = ms.hacker_id
GROUP BY h.hacker_id, h.name
HAVING total_score > 0
ORDER BY total_score DESC, h.hacker_id ASC;
```

OUTPUT:

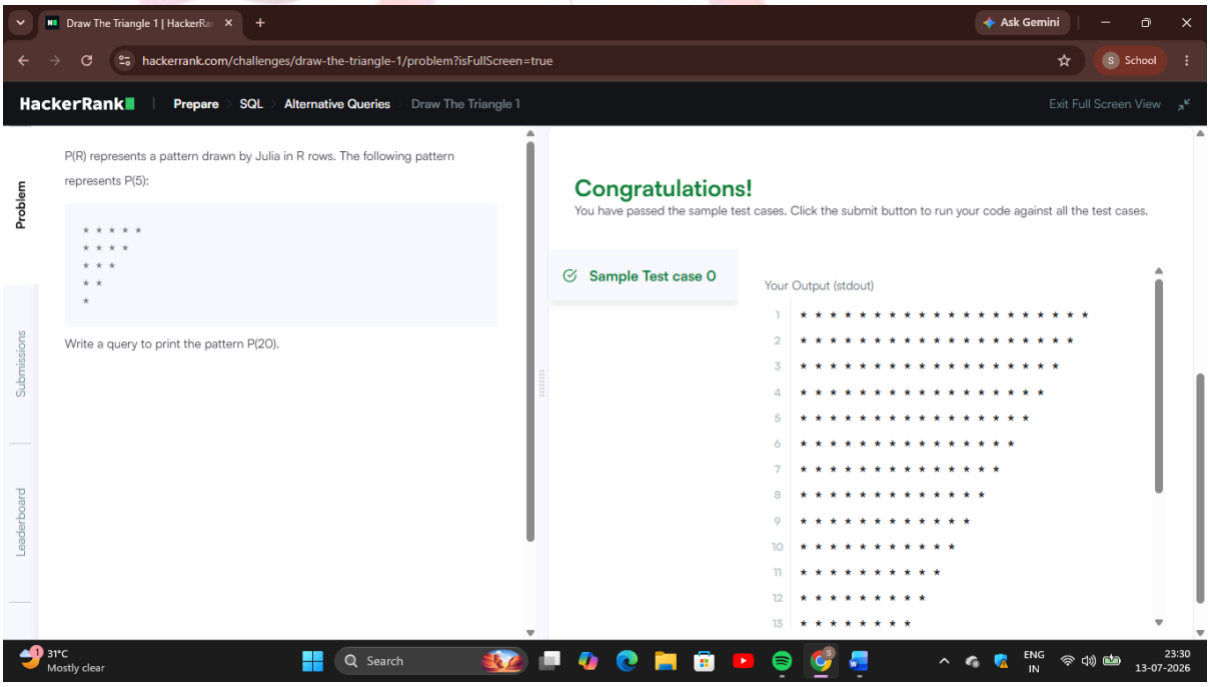


6.DRAW THE TRIANGLE 1

PROGRAM

```
WITH RECURSIVE pattern AS (
    SELECT 20 AS n
    UNION ALL
    SELECT n - 1
    FROM pattern
    WHERE n > 1
)
SELECT REPEAT('* ', n)
FROM pattern;
```

OUTPUT

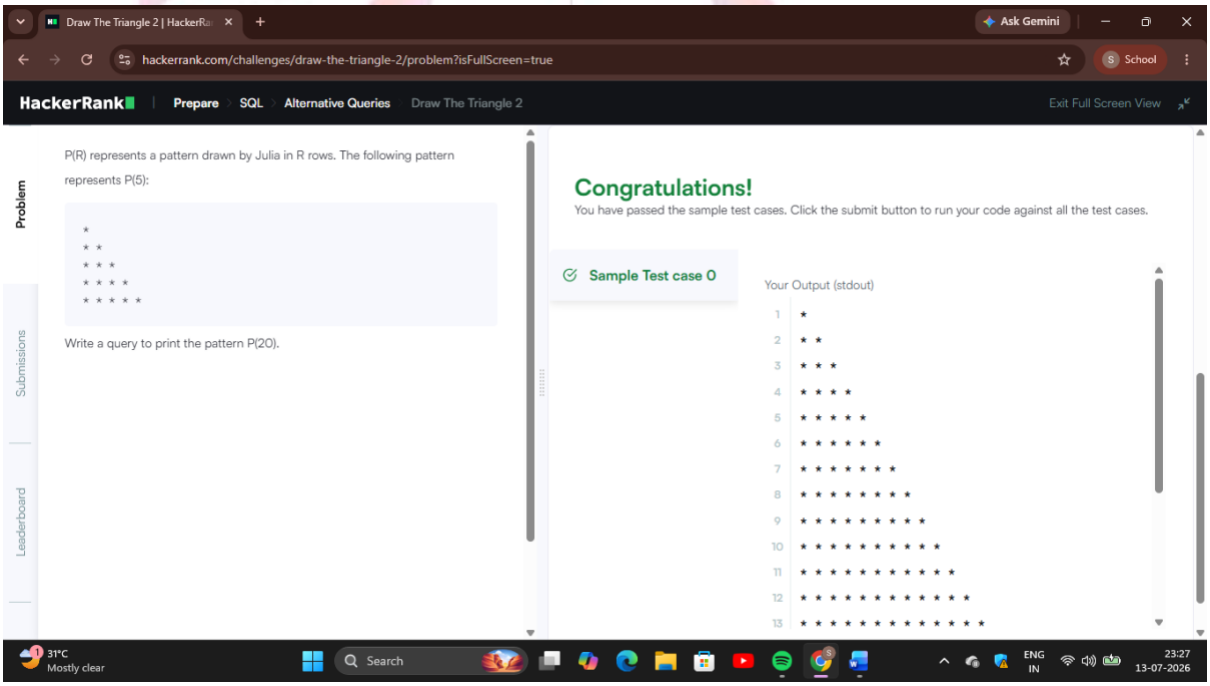


7.DRAW THE TRIANGLE 2

PROGRAM

```
WITH RECURSIVE pattern AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1
    FROM pattern
    WHERE n < 20
)
SELECT REPEAT('* ', n)
FROM pattern;
```

OUTPUT

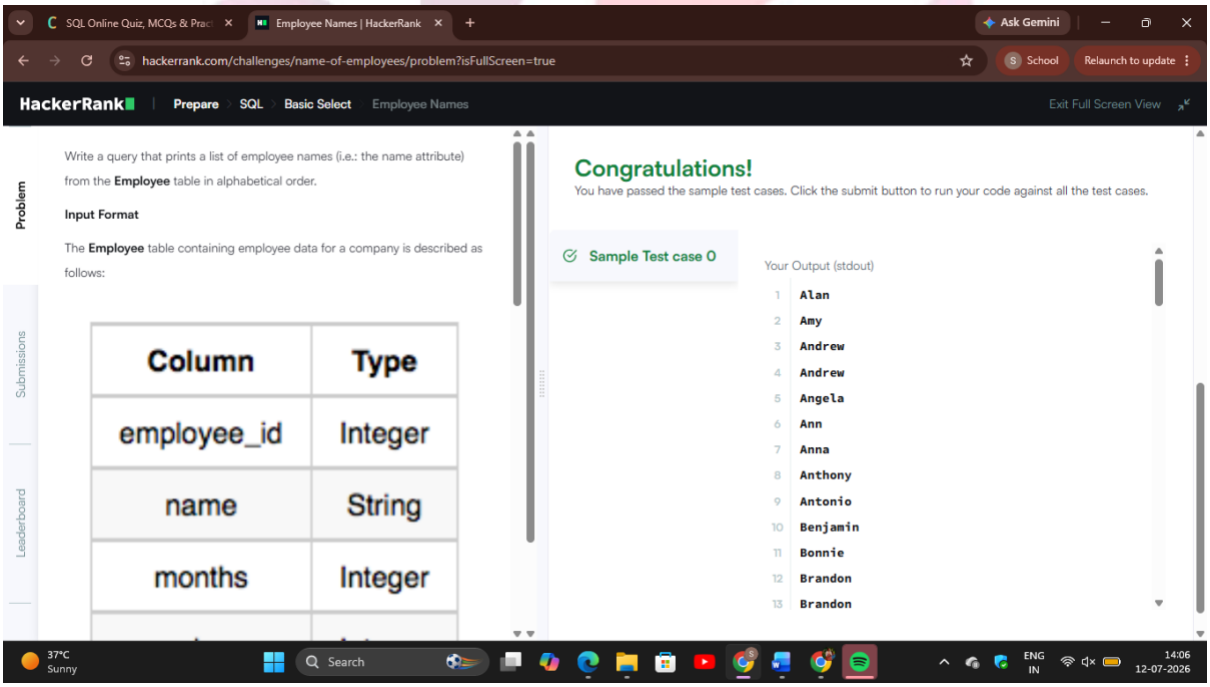


8.EMPLOYEE NAMES

PROGRAM

```
SELECT name
FROM Employee
ORDER BY name ASC;
```

OUTPUT:

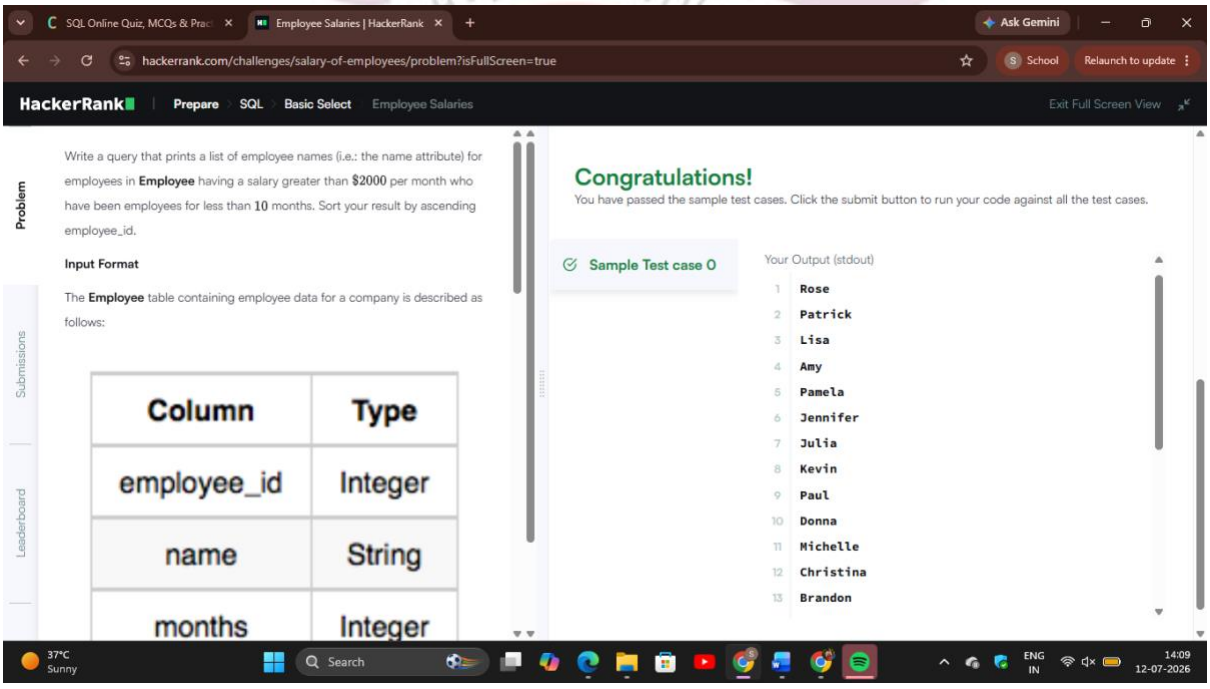


9.EMPLOYEE SALARIES

PROGRAM

```
SELECT name
FROM Employee
WHERE salary > 2000
  AND months < 10
ORDER BY employee_id ASC;
```

OUTPUT:

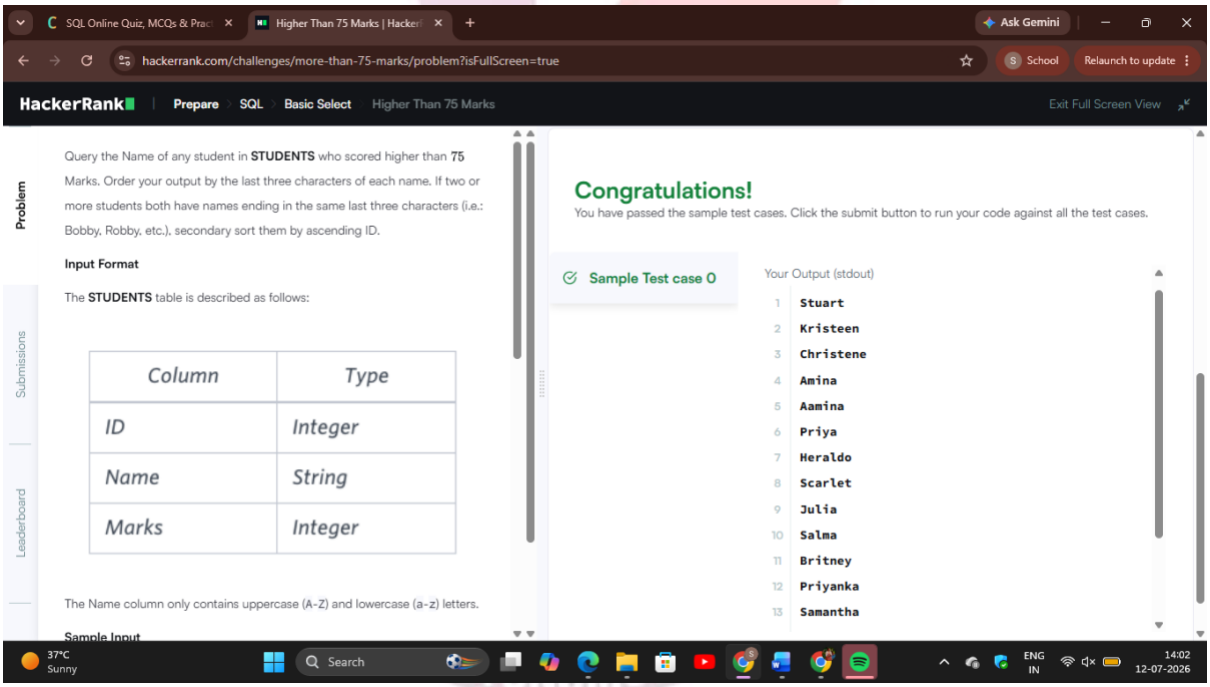


10.HIGHER THAN 75 MARKS

PROGRAM

```
SELECT Name
FROM STUDENTS
WHERE Marks > 75
ORDER BY RIGHT(Name, 3), ID;
```

OUTPUT:



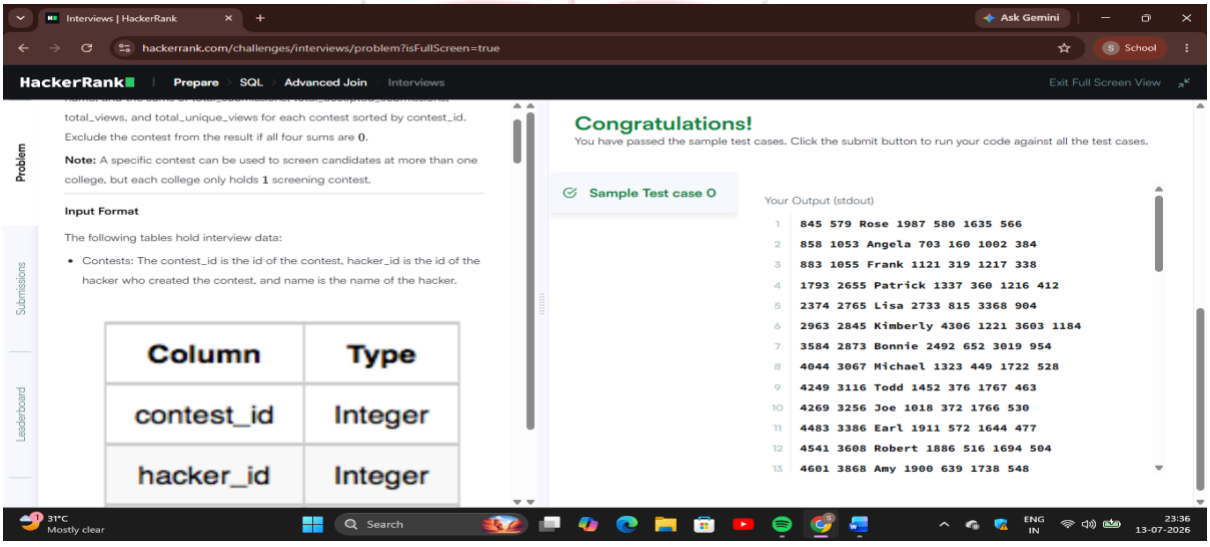
11.INTERVIEWS

PROGRAM

```
SELECT
    c.contest_id,
    c.hacker_id,
    c.name,
    SUM(COALESCE(ss.total_submissions, 0)) AS total_submissions,
    SUM(COALESCE(ss.total_accepted_submissions, 0)) AS total_accepted_submissions,
    SUM(COALESCE(vs.total_views, 0)) AS total_views,
    SUM(COALESCE(vs.total_unique_views, 0)) AS total_unique_views
FROM Contests c
JOIN Colleges col
    ON c.contest_id = col.contest_id
JOIN Challenges ch
    ON col.college_id = ch.college_id
LEFT JOIN (
    SELECT
        challenge_id,
        SUM(total_submissions) AS total_submissions,
        SUM(total_accepted_submissions) AS total_accepted_submissions
    FROM Submission_Stats
    GROUP BY challenge_id
) ss
    ON ch.challenge_id = ss.challenge_id
LEFT JOIN (
```

```
    SELECT
        challenge_id,
        SUM(total_views) AS total_views,
        SUM(total_unique_views) AS total_unique_views
    FROM View_Stats
    GROUP BY challenge_id
) vs
    ON ch.challenge_id = vs.challenge_id
GROUP BY
    c.contest_id,
    c.hacker_id,
    c.name
HAVING
    total_submissions + total_accepted_submissions + total_views + total_unique_views > 0
ORDER BY
    c.contest_id;
```

OUTPUT:



12.JAPAN POPULATION

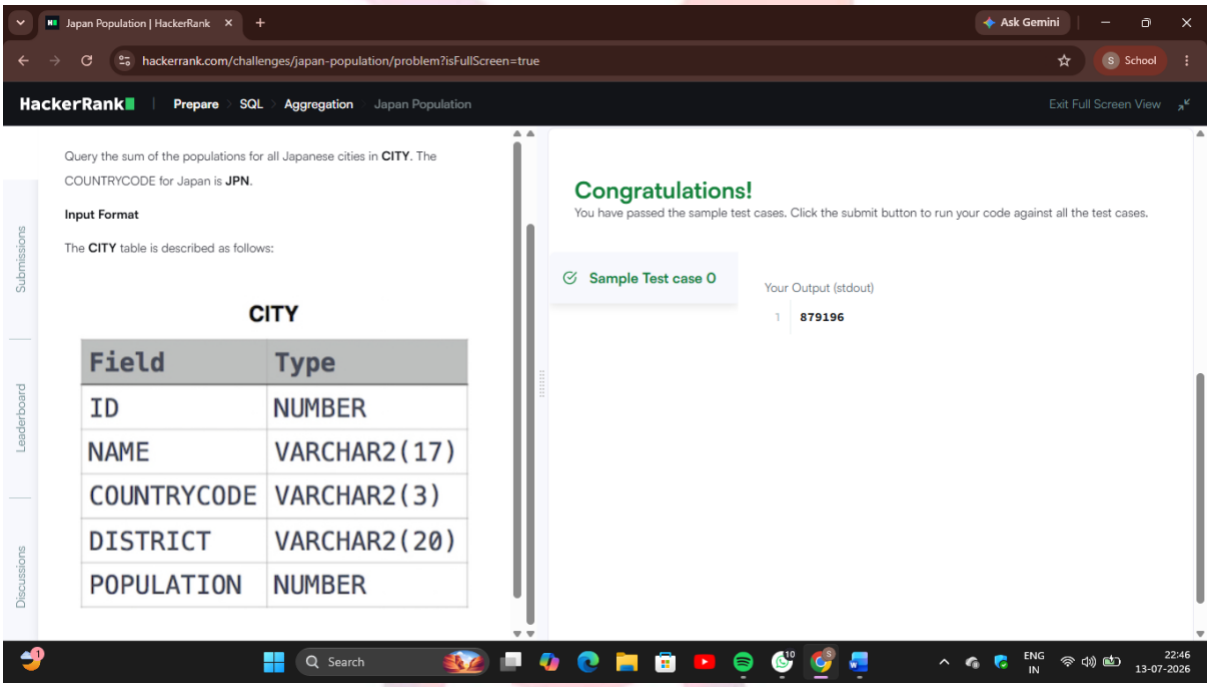
PROGRAM

```
SELECT SUM(POPULATION)

FROM CITY

WHERE COUNTRYCODE = 'JPN';
```

OUTPUT



13.JAPANESE CITIES' ATTRIBUTES

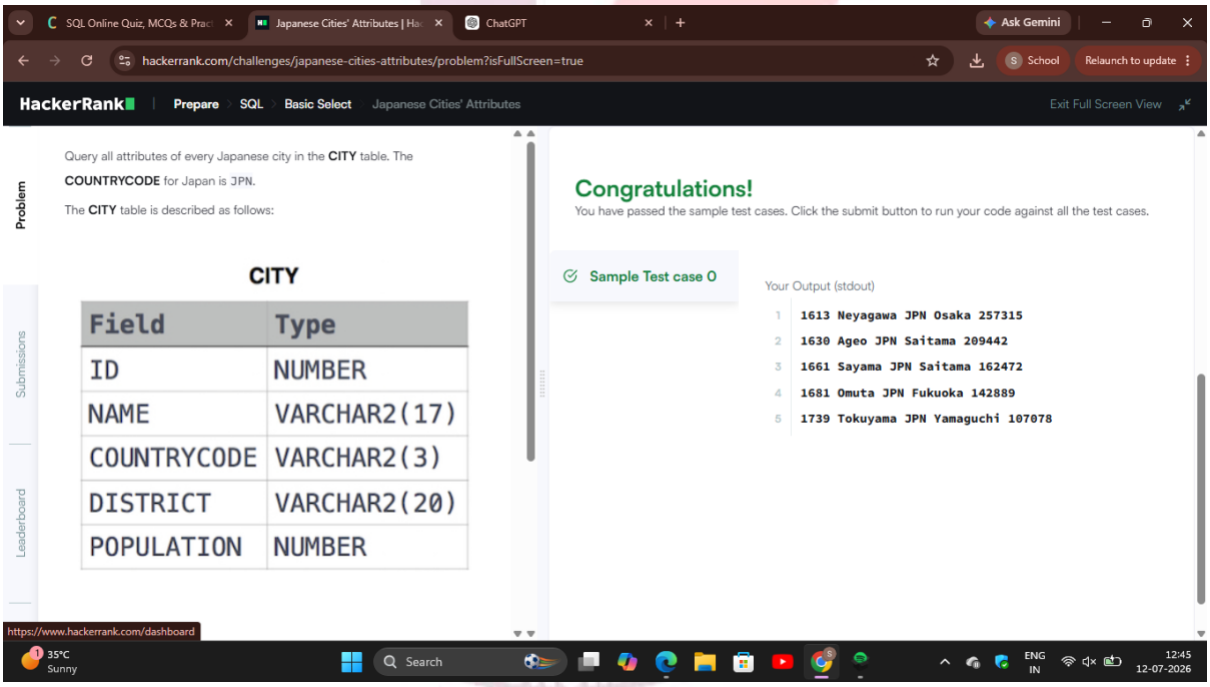
PROGRAM

```
SELECT *

FROM CITY

WHERE COUNTRYCODE = 'JPN';
```

OUTPUT

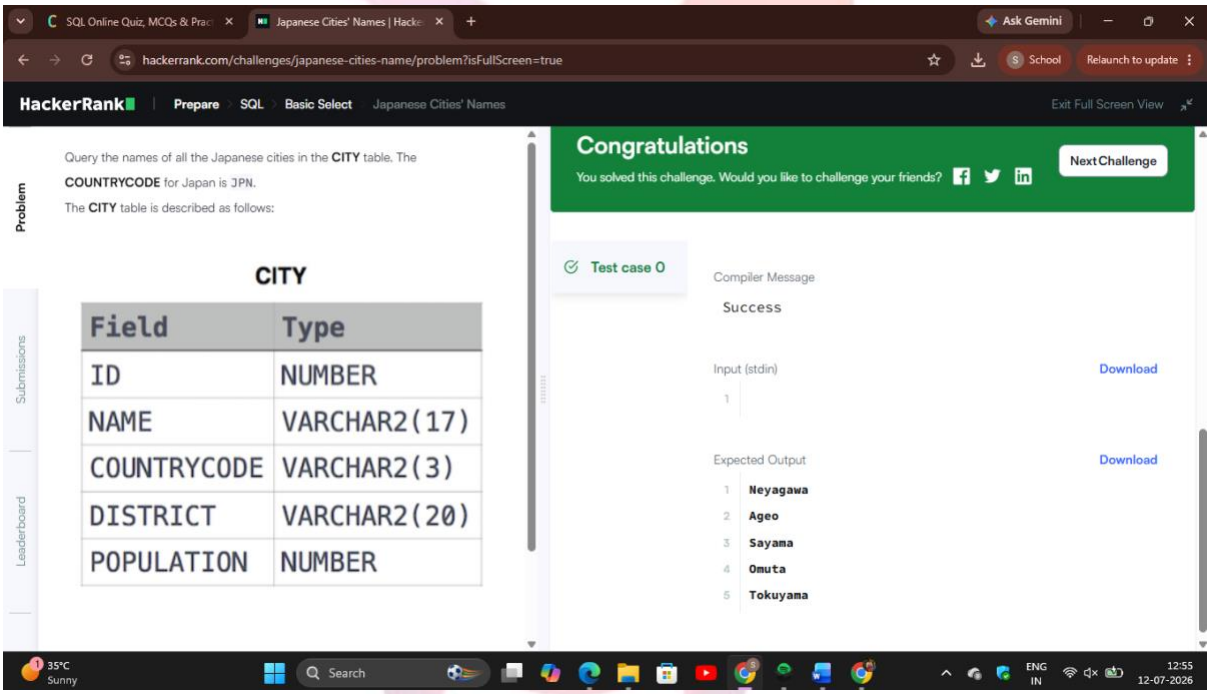


14.JAPANESE CITIES' NAMES

PROGRAM

```
SELECT NAME
FROM CITY
WHERE COUNTRYCODE = 'JPN';
```

OUTPUT

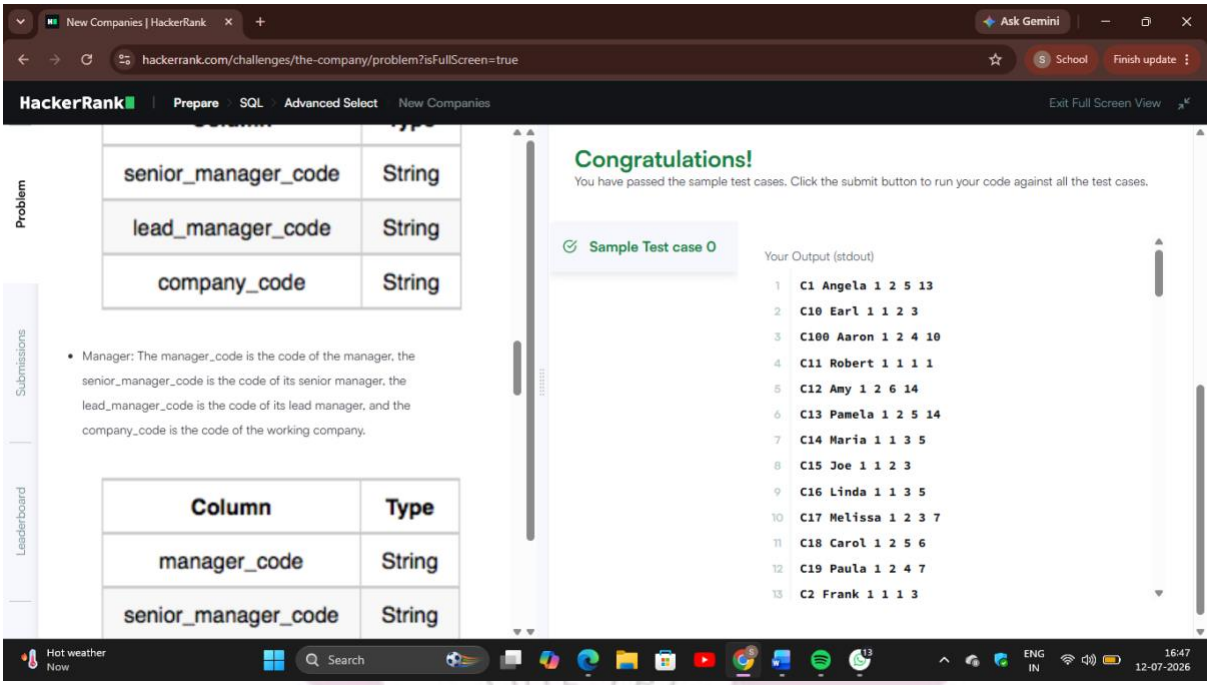


15.NEW COMPANIES

PROGRAM

```
SELECT
    c.company_code,
    c.founder,
    COUNT(DISTINCT lm.lead_manager_code) AS total_lead_managers,
    COUNT(DISTINCT sm.senior_manager_code) AS total_senior_managers,
    COUNT(DISTINCT m.manager_code) AS total_managers,
    COUNT(DISTINCT e.employee_code) AS total_employees
FROM Company c
LEFT JOIN Lead_Manager lm
    ON c.company_code = lm.company_code
LEFT JOIN Senior_Manager sm
    ON c.company_code = sm.company_code
LEFT JOIN Manager m
    ON c.company_code = m.company_code
LEFT JOIN Employee e
    ON c.company_code = e.company_code
GROUP BY
    c.company_code,
    c.founder
ORDER BY
    c.company_code;
```

OUTPUT

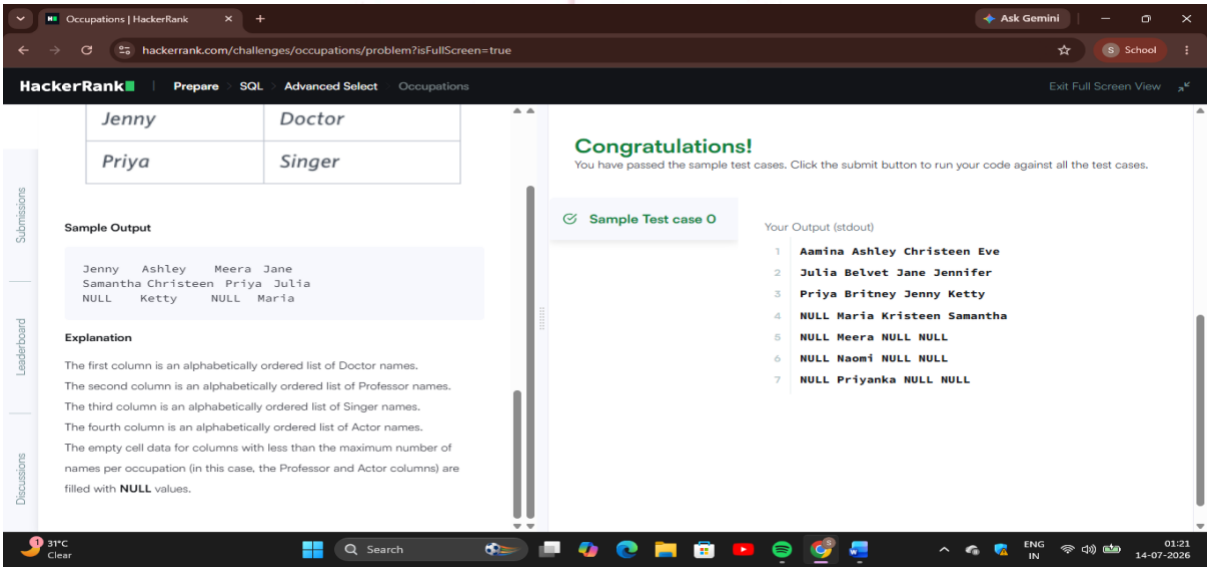

New Companies | HackerRank
hackerrank.com/challenges/the-company/problem?isFullScreen=true
HackerRank
Prepare
SQL
Advanced Select
New Companies
Exit Full Screen View
Problem
Submissions
Leaderboard
senior_manager_code String
lead_manager_code String
company_code String
Manager: The manager_code is the code of the manager, the senior_manager_code is the code of its senior manager, the lead_manager_code is the code of its lead manager, and the company_code is the code of the working company.
Column Type
manager_code String
senior_manager_code String
Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.
Sample Test case 0
Your Output (stdout)
C1 Angela 1 2 5 13
C10 Earl 1 1 2 3
C100 Aaron 1 2 4 10
C11 Robert 1 1 1 1
C12 Amy 1 2 6 14
C13 Pamela 1 2 5 14
C14 Maria 1 1 3 5
C15 Joe 1 1 2 3
C16 Linda 1 1 3 5
C17 Melissa 1 2 3 7
C18 Carol 1 2 5 6
C19 Paula 1 2 4 7
C2 Frank 1 1 1 3

16.OCCUPATIONS

PROGRAM

```
SELECT
    MAX(CASE WHEN Occupation = 'Doctor' THEN Name END) AS Doctor,
    MAX(CASE WHEN Occupation = 'Professor' THEN Name END) AS Professor,
    MAX(CASE WHEN Occupation = 'Singer' THEN Name END) AS Singer,
    MAX(CASE WHEN Occupation = 'Actor' THEN Name END) AS Actor
FROM (
    SELECT
        Name,
        Occupation,
        ROW_NUMBER() OVER (PARTITION BY Occupation ORDER BY Name) AS rn
    FROM OCCUPATIONS
) t
GROUP BY rn
ORDER BY rn;
```

OUTPUT

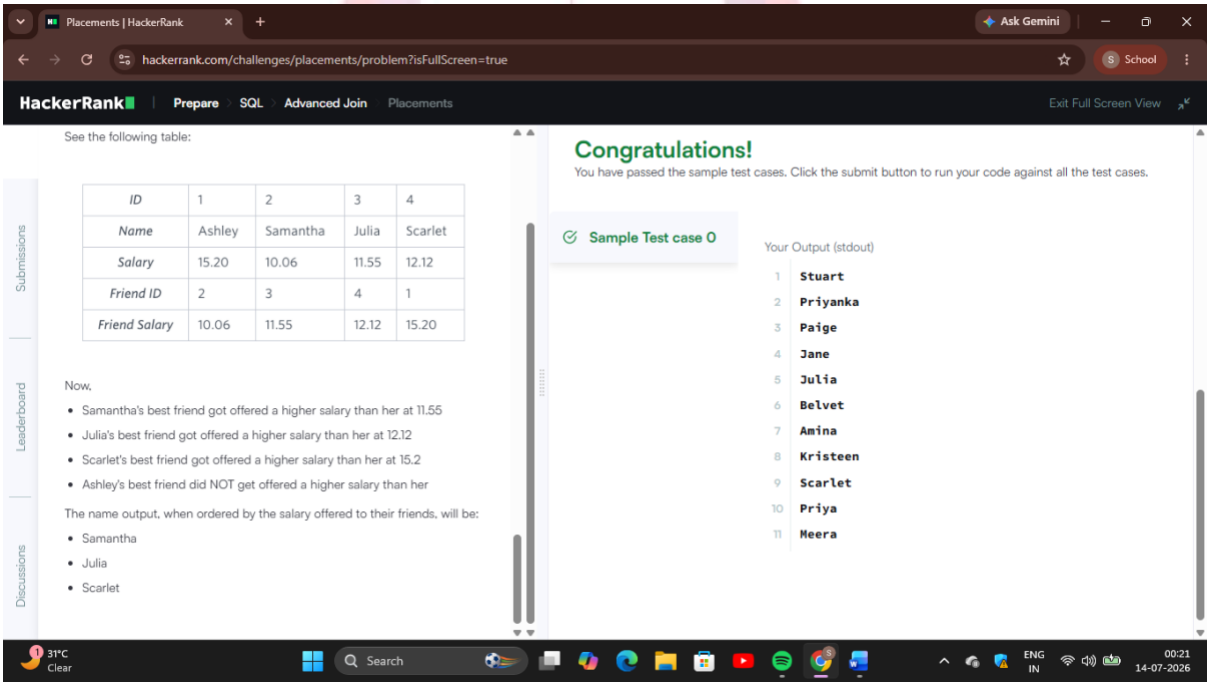


17.PLACEMENTS

PROGRAM

```
SELECT s.Name
FROM Students s
JOIN Friends f
    ON s.ID = f.ID
JOIN Packages p1
    ON s.ID = p1.ID
JOIN Packages p2
    ON f.Friend_ID = p2.ID
WHERE p2.Salary > p1.Salary
ORDER BY p2.Salary;
```

OUTPUT

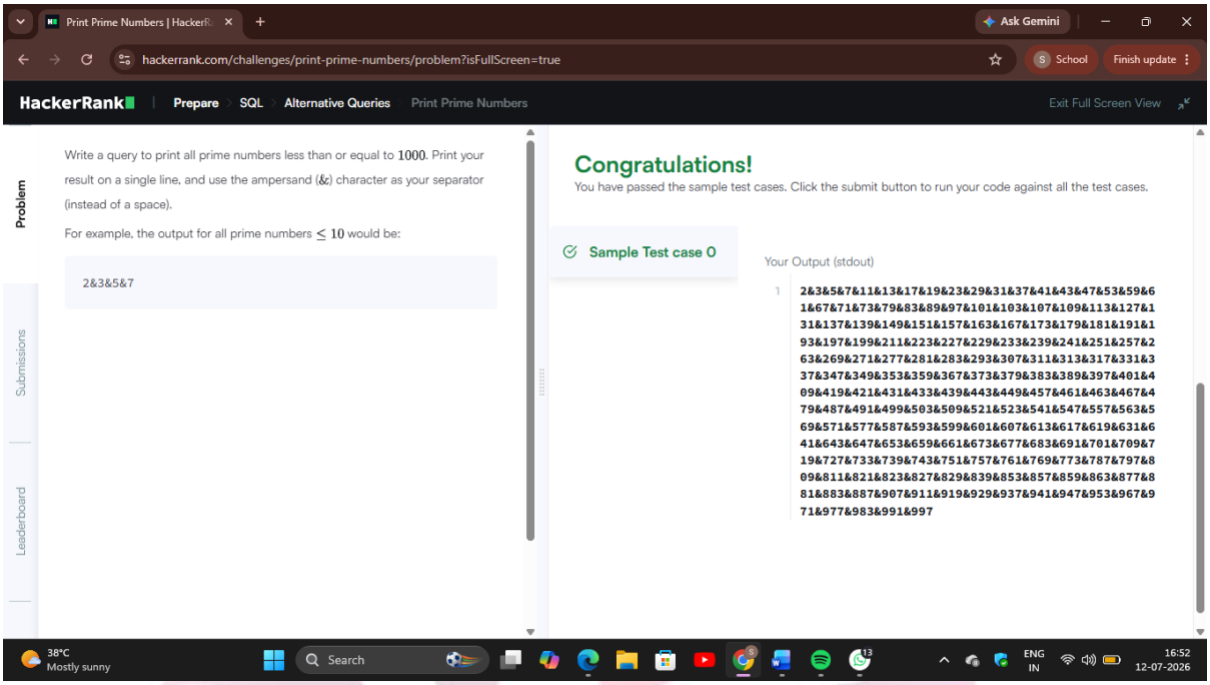


18.PRINT PRIME NUMBERS

PROGRAM

```
WITH nums (n) AS (
   SELECT 2
   FROM SYSIBM.SYSDUMMY1
   UNION ALL
   SELECT n + 1
   FROM nums
   WHERE n < 1000
),
primes (n) AS (
   SELECT n
   FROM nums a
   WHERE NOT EXISTS (
      SELECT 1
      FROM nums b
      WHERE b.n < a.n
        AND b.n * b.n <= a.n
        AND MOD(a.n, b.n) = 0
   )
)
SELECT LISTAGG(VARCHAR(n), '&') WITHIN GROUP (ORDER BY n)
FROM primes;
```

OUTPUT



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

19.REVISING AGGREGATIONS - THE COUNT FUNCTION

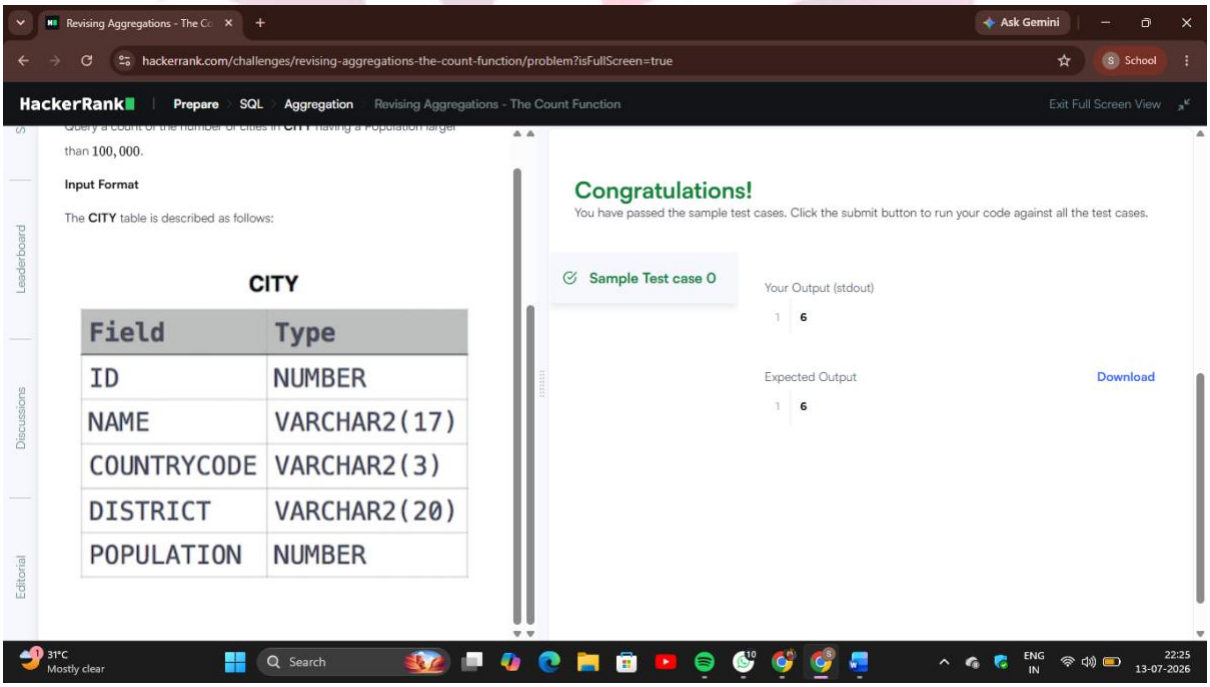
PROGRAM

```
SELECT COUNT(*)

FROM CITY

WHERE Population > 100000;
```

OUTPUT



20.REVISING AGGREGATIONS - THE SUM FUNCTION

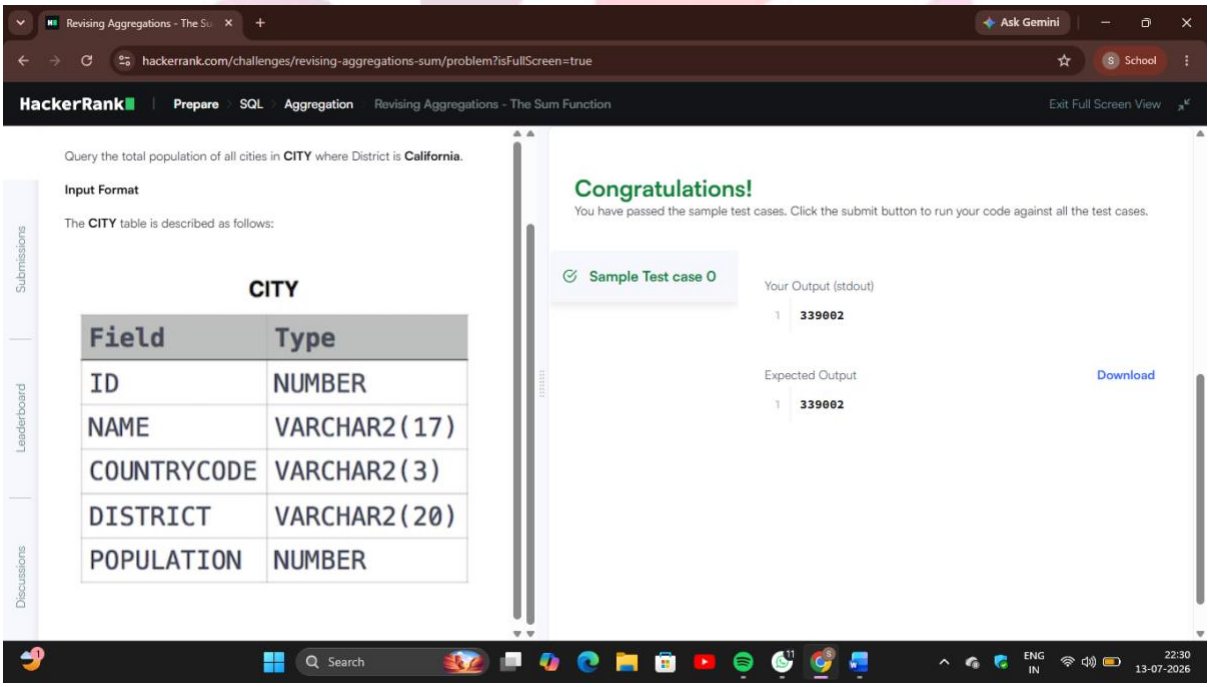
PROGRAM

```
SELECT SUM(Population)

FROM CITY

WHERE District = 'California';
```

OUTPUT

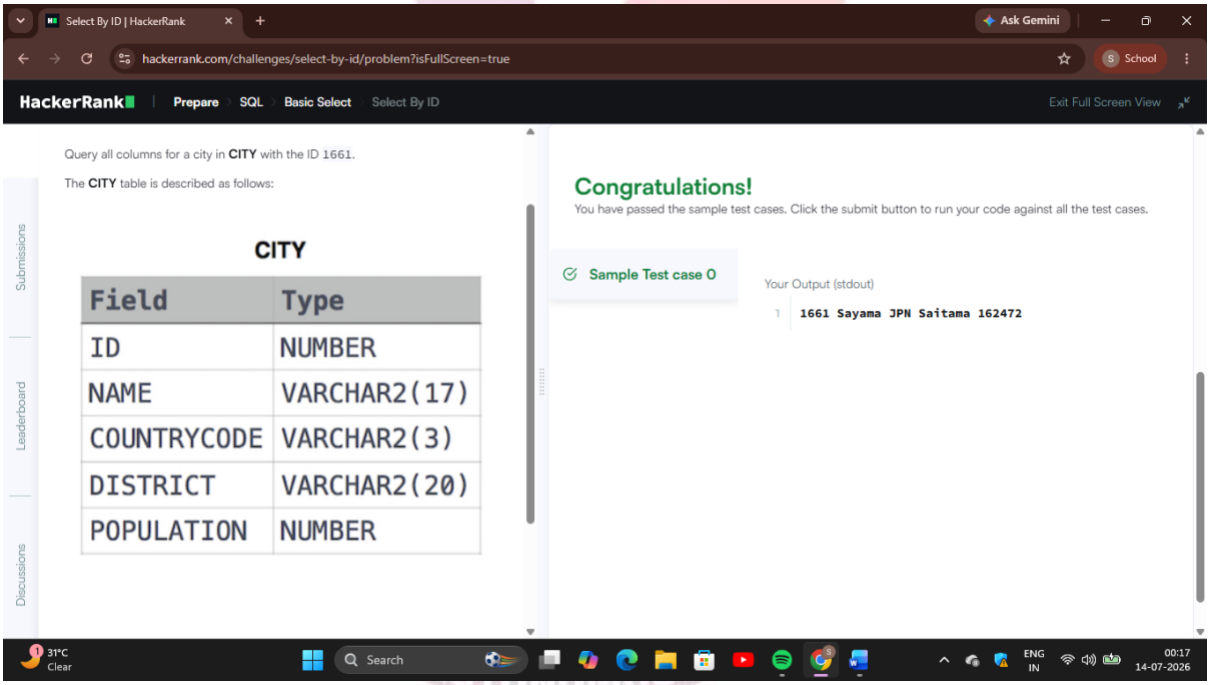


21.SELECT BY ID

PROGRAM

```
SELECT *
FROM CITY
WHERE ID = 1661;
```

OUTPUT



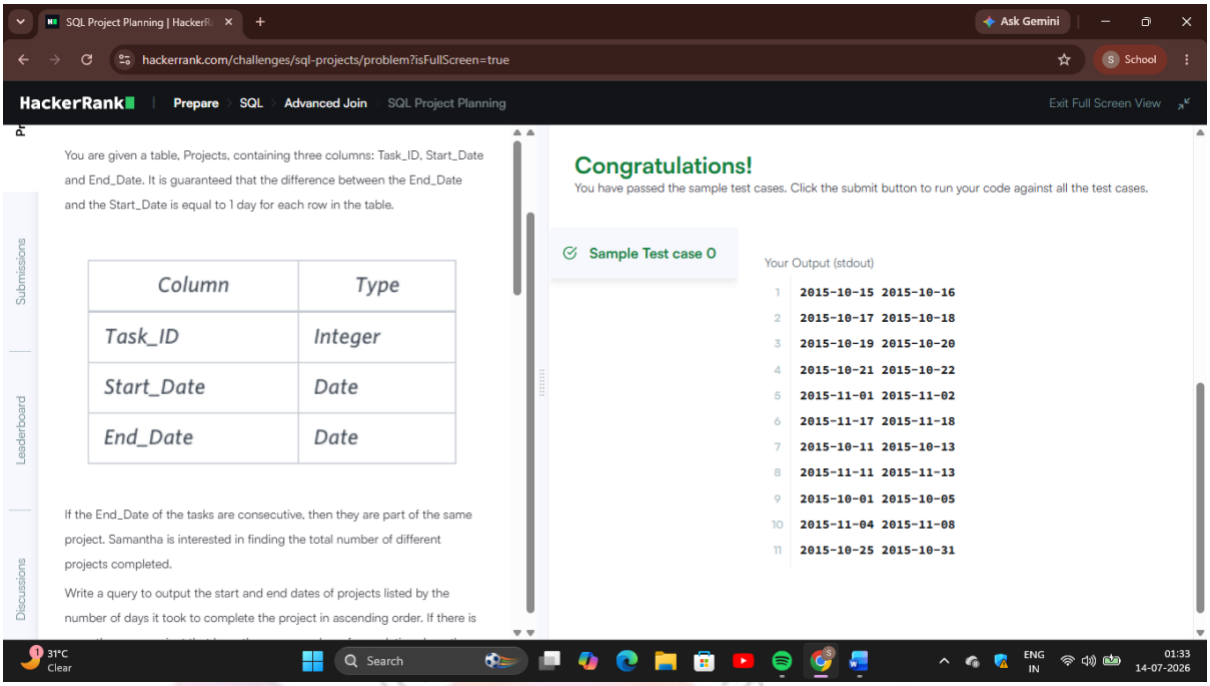
22.SQL PROJECT PLANNING

PROGRAM

```
SELECT
    MIN(Start_Date) AS Start_Date,
    MAX(End_Date) AS End_Date
FROM (
    SELECT
        p1.Start_Date,
        p1.End_Date,
        (
            SELECT COUNT(*)
            FROM Projects p2
            WHERE p2.End_Date < p1.Start_Date
            AND NOT EXISTS (
                SELECT 1
                FROM Projects p3
                WHERE p3.End_Date = DATE_ADD(p2.End_Date, INTERVAL 1 DAY)
            )
        ) AS project_id
    FROM Projects p1
) AS Projects_Grouped
GROUP BY project_id
ORDER BY
    DATEDIFF(MAX(End_Date), MIN(Start_Date)),
```

MIN(Start_Date);

OUTPUT

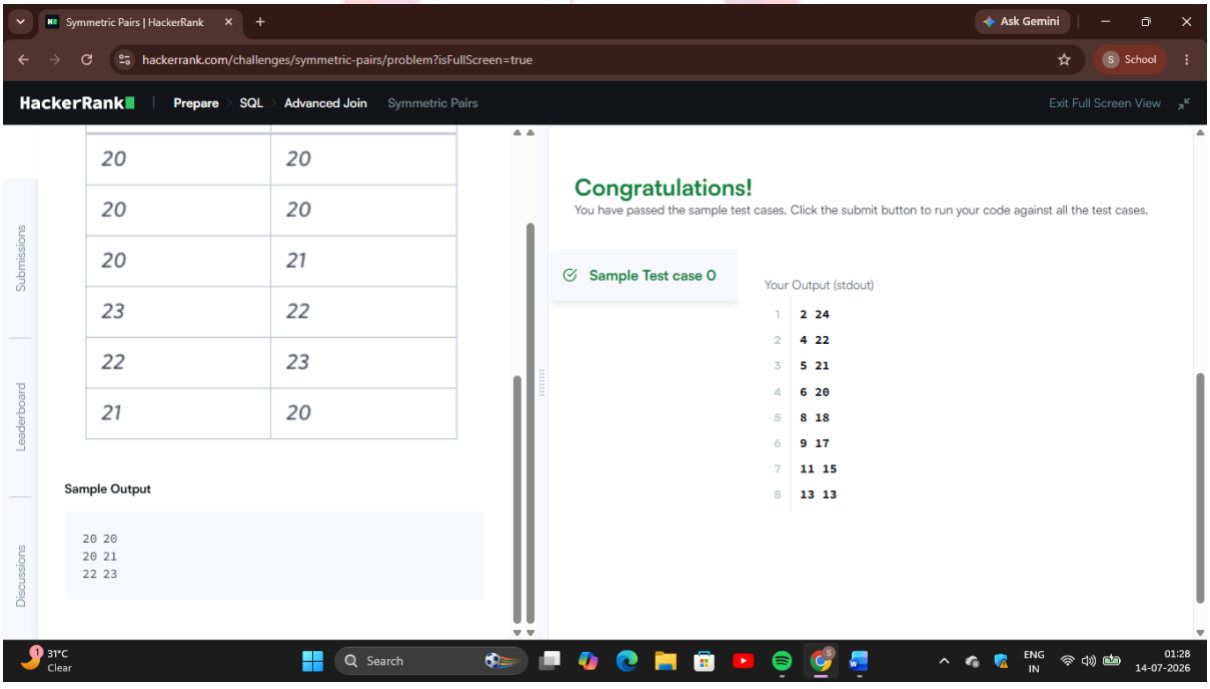


23.SYMMETRIC PAIRS

PROGRAM

```
SELECT f1.X, f1.Y
FROM Functions f1
JOIN Functions f2
    ON f1.X = f2.Y
    AND f1.Y = f2.X
WHERE f1.X <= f1.Y
GROUP BY f1.X, f1.Y
HAVING COUNT(*) > 1 OR f1.X < f1.Y
ORDER BY f1.X;
```

OUTPUT



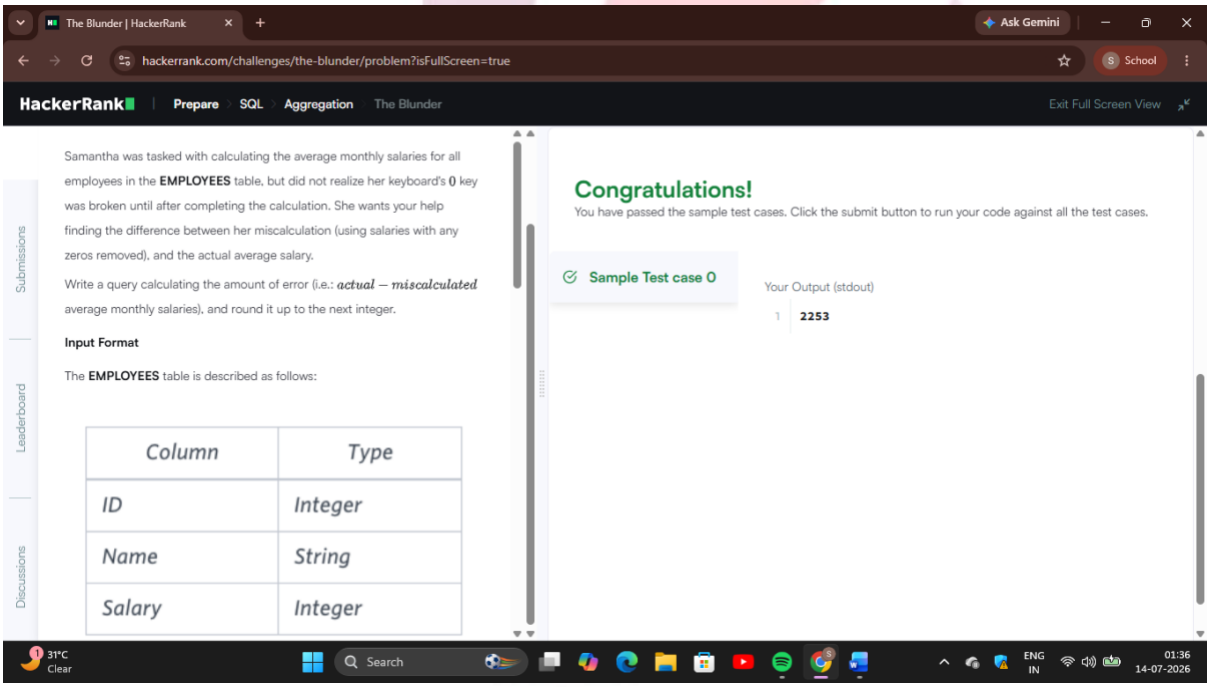
24.THE BLUNDER

PROGRAM

```
SELECT CEIL(AVG(Salary) - AVG(CAST(REPLACE(CAST(Salary AS CHAR), '0', '') AS
UNSIGNED)))

FROM EMPLOYEES;
```

OUTPUT



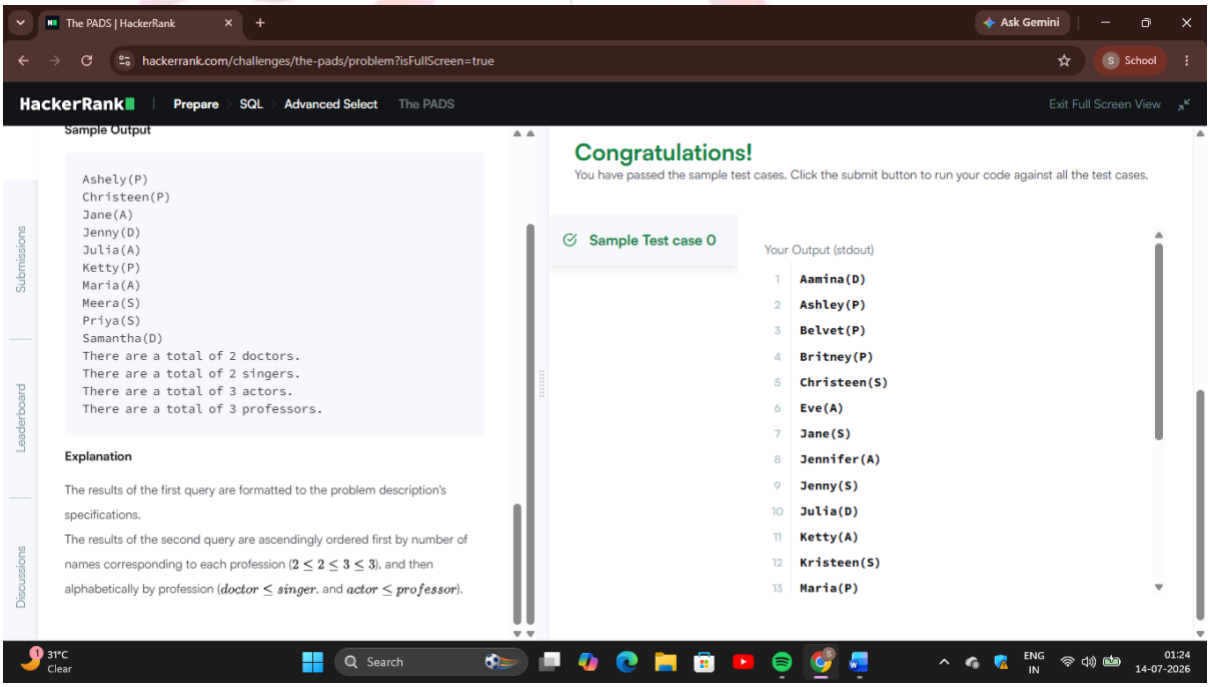
25.THE PADS

PROGRAM

```
SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')')
FROM OCCUPATIONS
ORDER BY Name;

SELECT CONCAT('There are a total of ', COUNT(*), ' ', LOWER(Occupation), 's.')
FROM OCCUPATIONS
GROUP BY Occupation
ORDER BY COUNT(*), Occupation;
```

OUTPUT

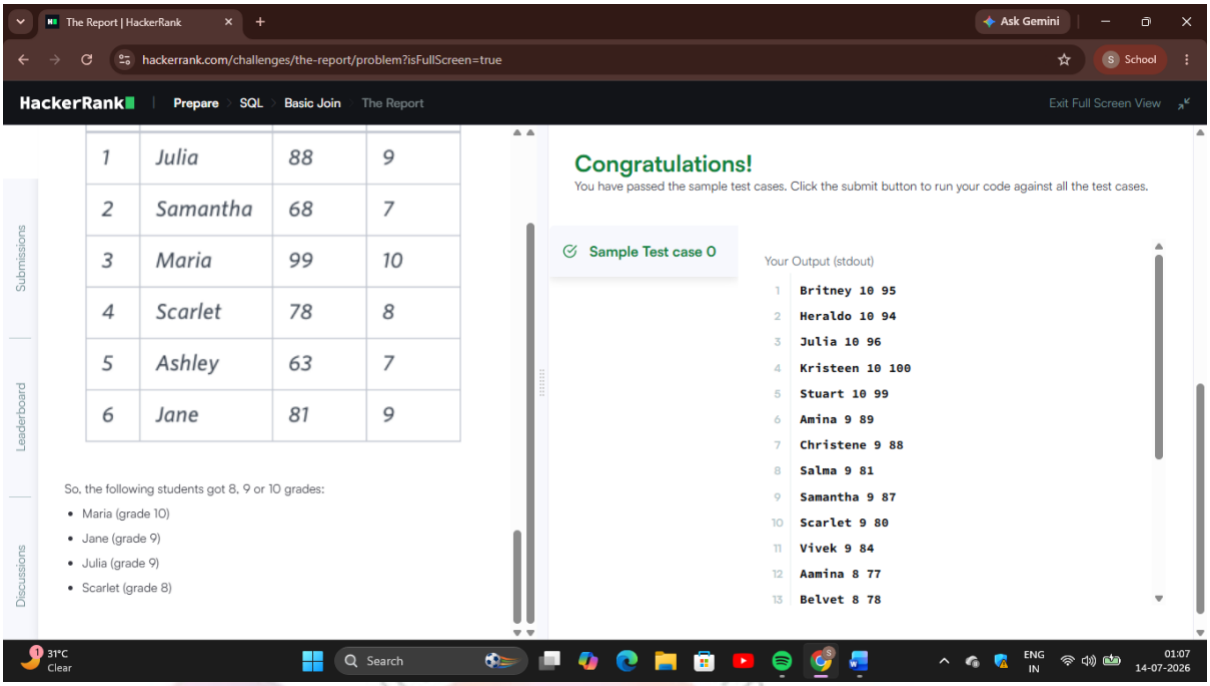


26.THE REPORT

PROGRAM

```
SELECT
   CASE
      WHEN g.Grade < 8 THEN NULL
      ELSE s.Name
   END AS Name,
   g.Grade,
   s.Marks
FROM Students s
JOIN Grades g
   ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
ORDER BY
   g.Grade DESC,
   CASE
      WHEN g.Grade >= 8 THEN s.Name
   END ASC,
   CASE
      WHEN g.Grade < 8 THEN s.Marks
   END ASC;
```

OUTPUT



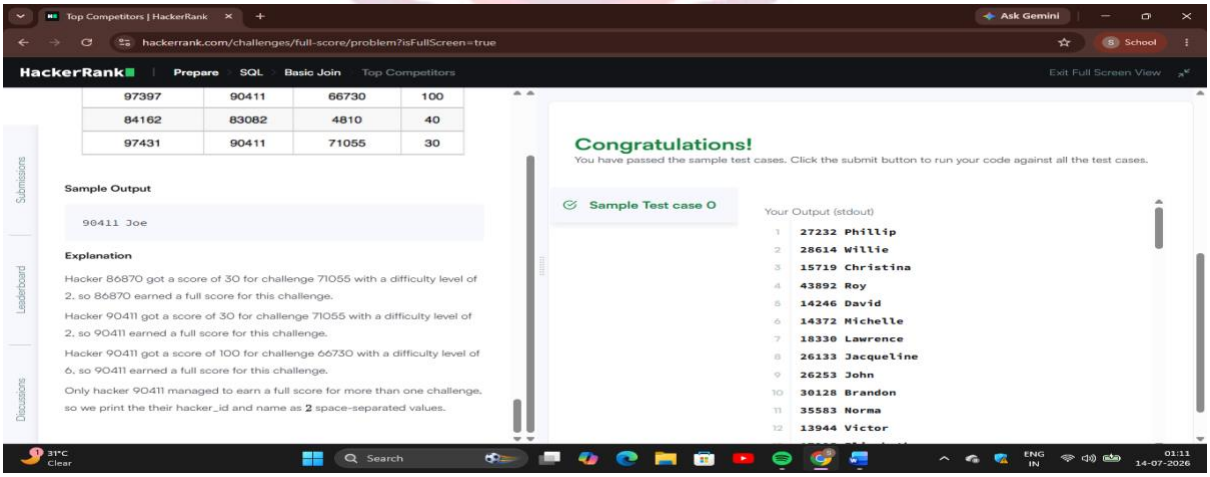
AUTONOMOUS

27.TOP COMPETITORS

PROGRAM

```
SELECT h.hacker_id, h.name
FROM Hackers h
JOIN Submissions s
    ON h.hacker_id = s.hacker_id
JOIN Challenges c
    ON s.challenge_id = c.challenge_id
JOIN Difficulty d
    ON c.difficulty_level = d.difficulty_level
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(DISTINCT s.challenge_id) > 1
ORDER BY COUNT(DISTINCT s.challenge_id) DESC, h.hacker_id ASC;
```

OUTPUT

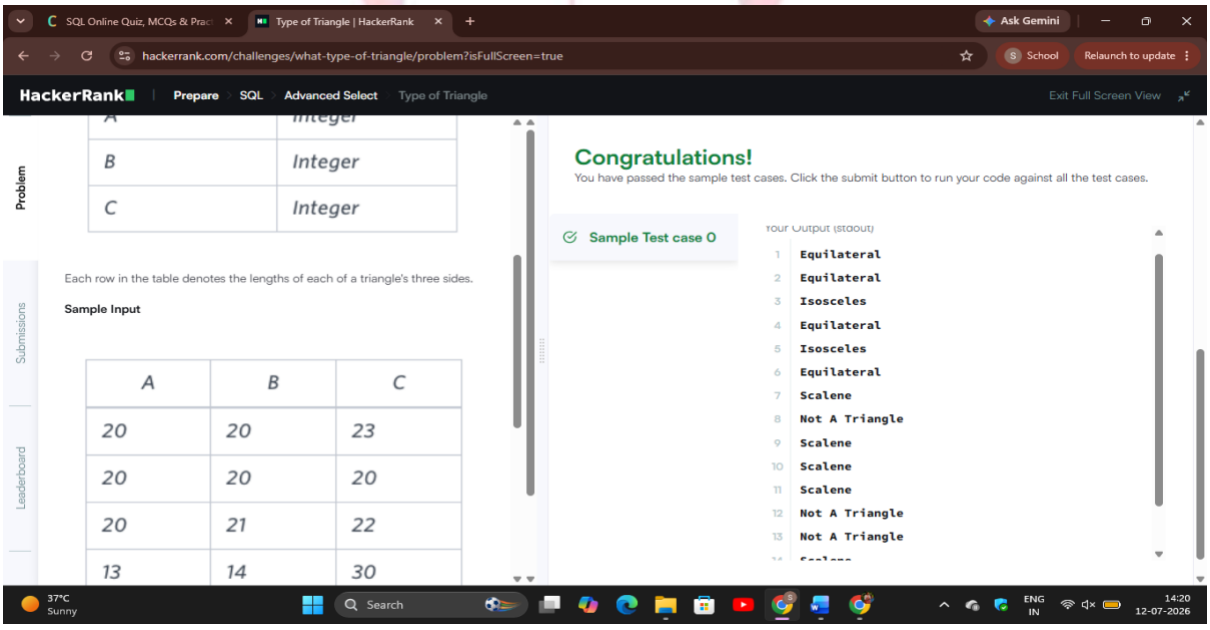


28.TYPE OF TRIANGLE

PROGRAM

```
SELECT
   CASE
      WHEN A + B <= C OR A + C <= B OR B + C <= A
         THEN 'Not A Triangle'
      WHEN A = B AND B = C
         THEN 'Equilateral'
      WHEN A = B OR A = C OR B = C
         THEN 'Isosceles'
      ELSE 'Scalene'
   END AS Triangle_Type
FROM TRIANGLES;
```

OUTPUT

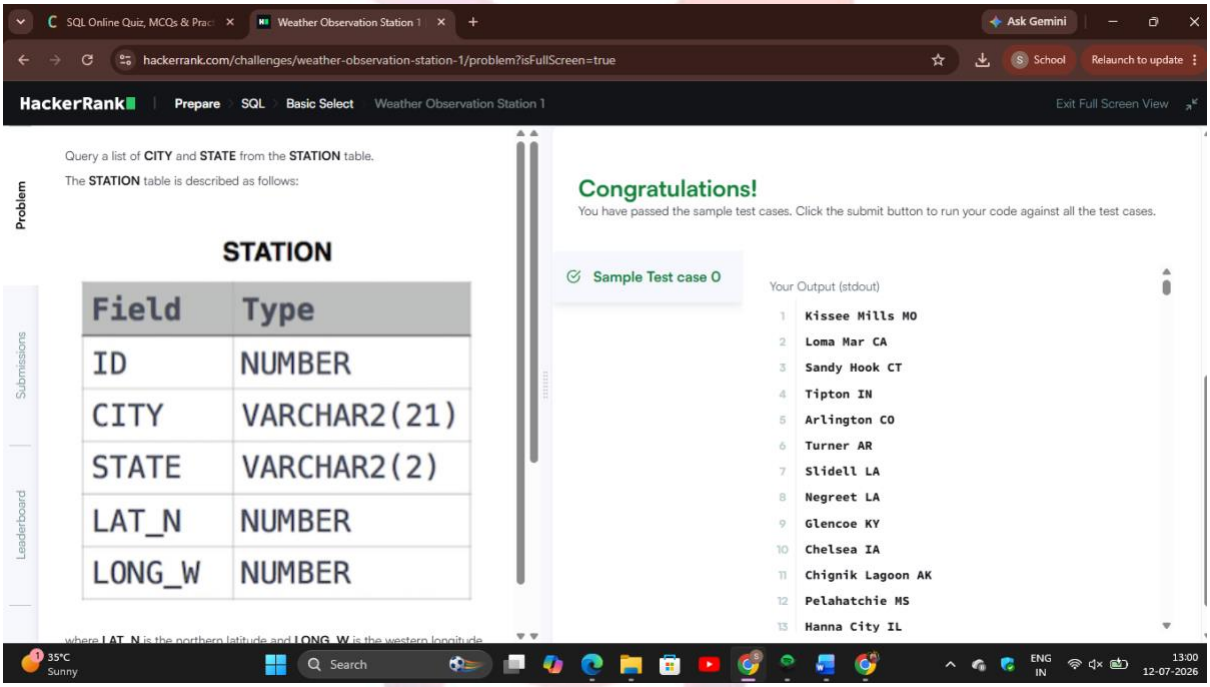


29.WEATHER OBSERVATION STATION 1

PROGRAM

```
SELECT CITY, STATE
FROM STATION;
```

OUTPUT

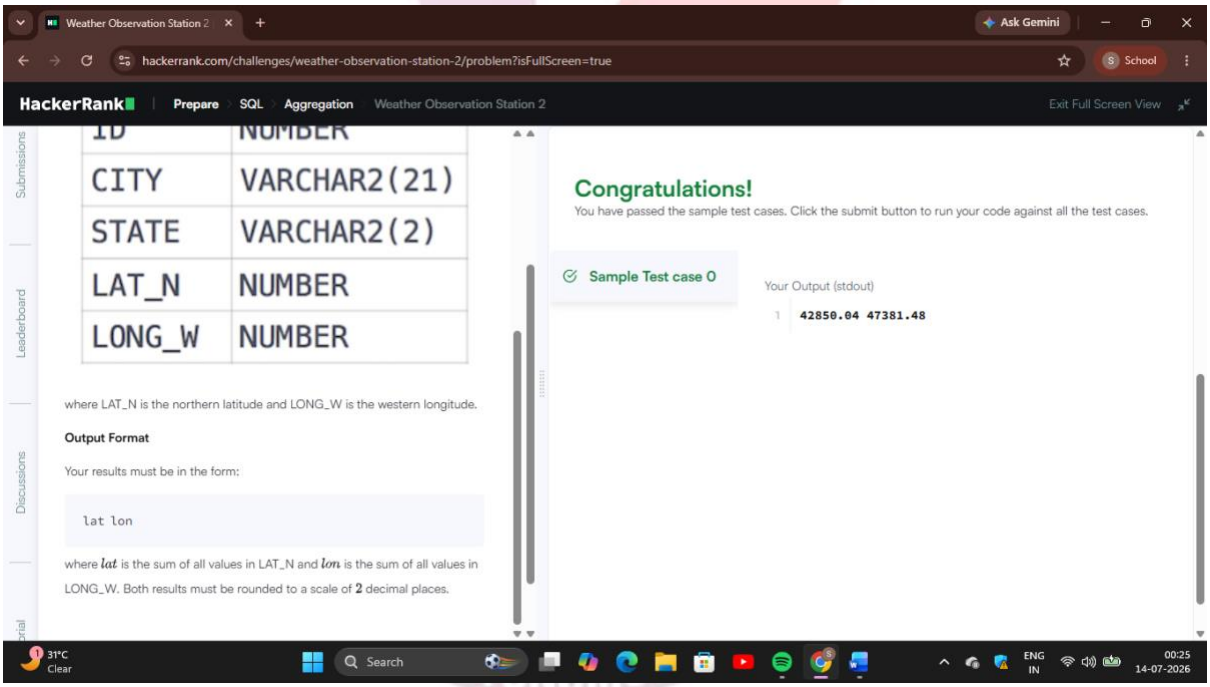


30.WEATHER OBSERVATION STATION 2

PROGRAM

```
SELECT
   ROUND(SUM(LAT_N), 2),
   ROUND(SUM(LONG_W), 2)
FROM STATION;
```

OUTPUT

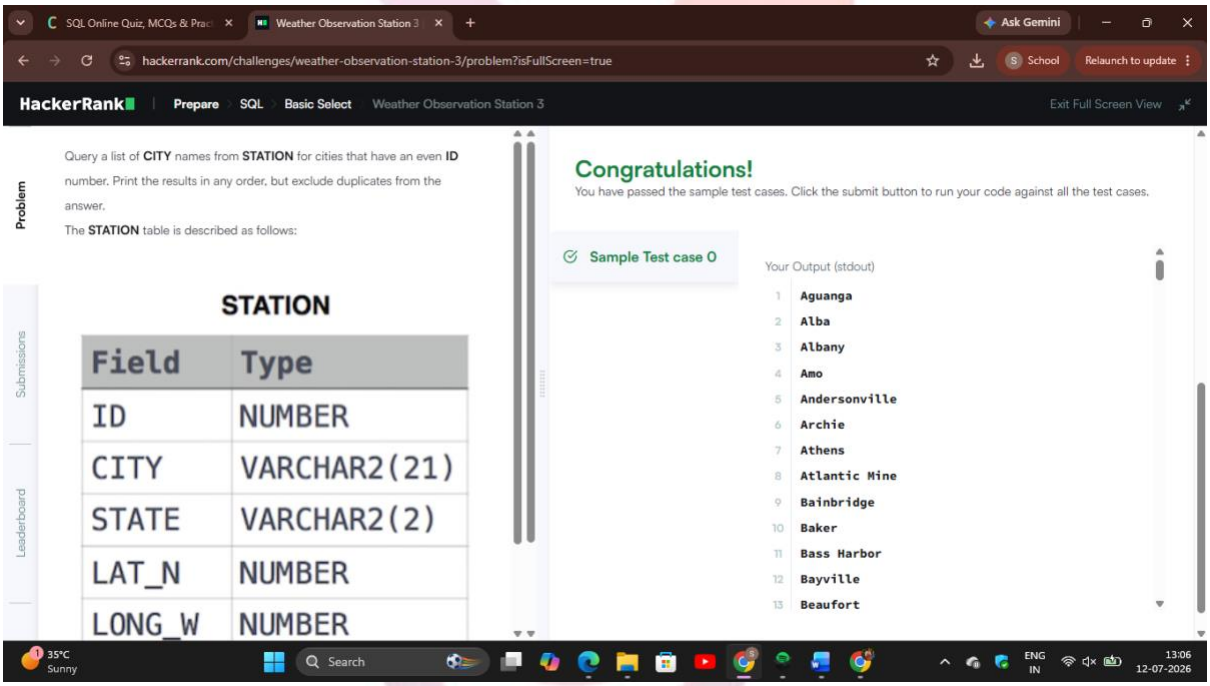


31.WEATHER OBSERVATION STATION 3

PROGRAM

```
SELECT DISTINCT CITY
FROM STATION
WHERE MOD(ID,2) = 0;
```

OUTPUT

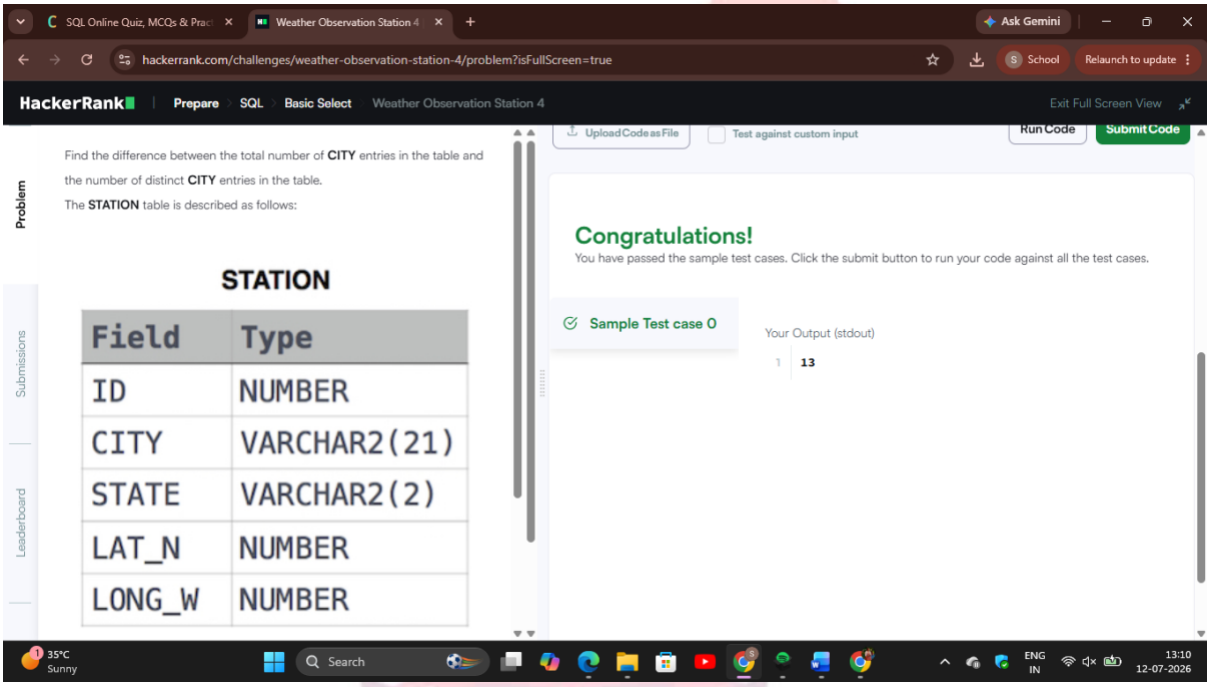


32.WEATHER OBSERVATION STATION 4

PROGRAM

```
SELECT COUNT(CITY) - COUNT(DISTINCT CITY)
FROM STATION;
```

OUTPUT

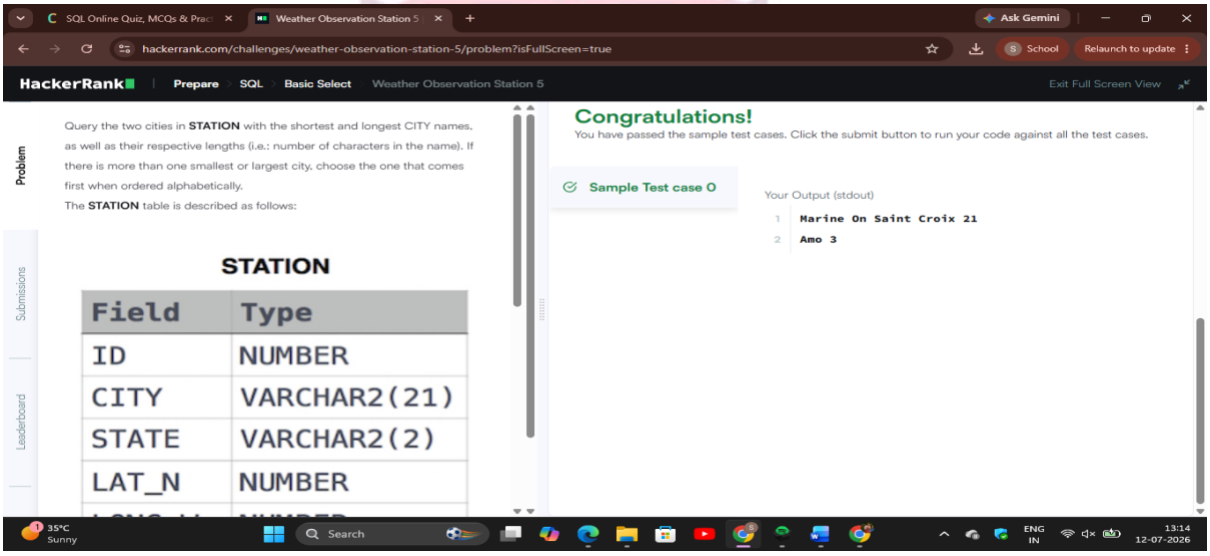


33.WEATHER OBSERVATION STATION 5

PROGRAM

```
(
   SELECT CITY, LENGTH(CITY) AS name_length
   FROM STATION
   ORDER BY LENGTH(CITY), CITY
   LIMIT 1
)
UNION ALL
(
   SELECT CITY, LENGTH(CITY) AS name_length
   FROM STATION
   ORDER BY LENGTH(CITY) DESC, CITY
   LIMIT 1
);
```

OUTPUT

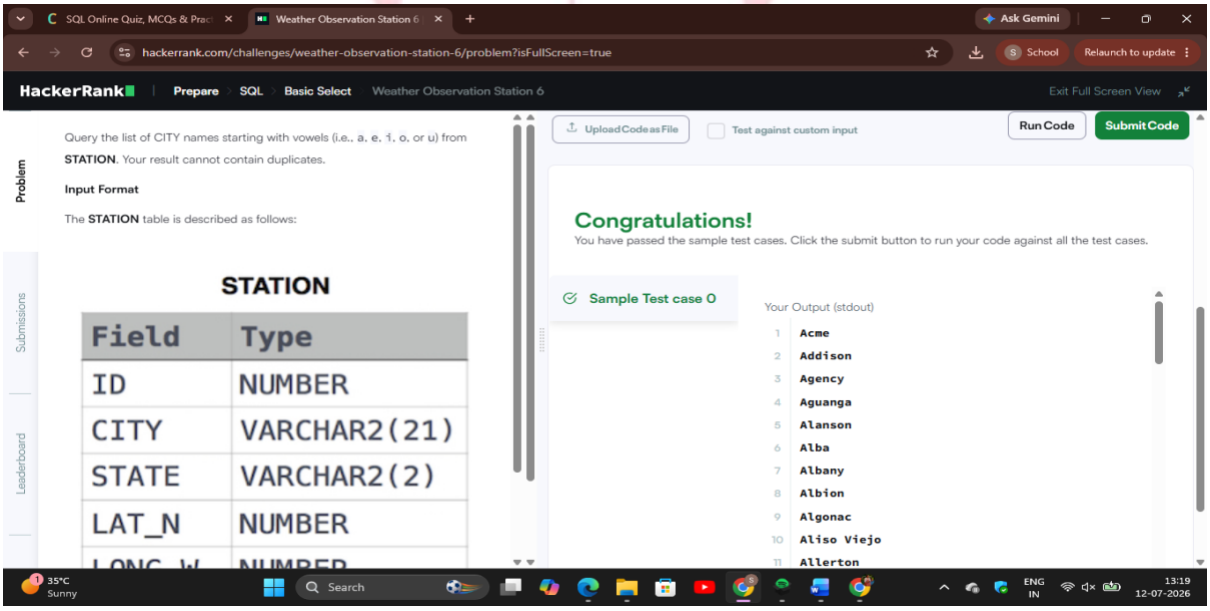


34.WEATHER OBSERVATION STATION 6

PROGRAM

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY LIKE 'A%'
  OR CITY LIKE 'E%'
  OR CITY LIKE 'I%'
  OR CITY LIKE 'O%'
  OR CITY LIKE 'U%'
  OR CITY LIKE 'a%'
  OR CITY LIKE 'e%'
  OR CITY LIKE 'i%'
  OR CITY LIKE 'o%'
  OR CITY LIKE 'u%';
```

OUTPUT

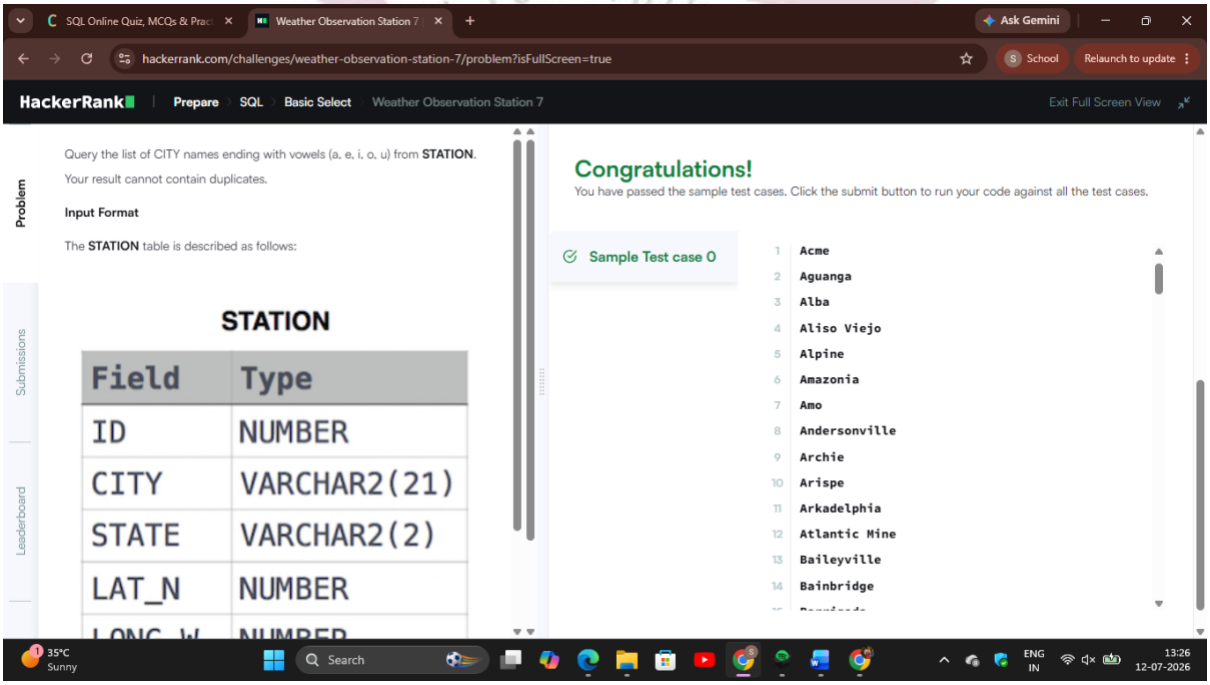


35.WEATHER OBSERVATION STATION 7

PROGRAM

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY LIKE '%a'
  OR CITY LIKE '%e'
  OR CITY LIKE '%i'
  OR CITY LIKE '%o'
  OR CITY LIKE '%u';
```

OUTPUT



36.WEATHER OBSERVATION STATION 8

PROGRAM

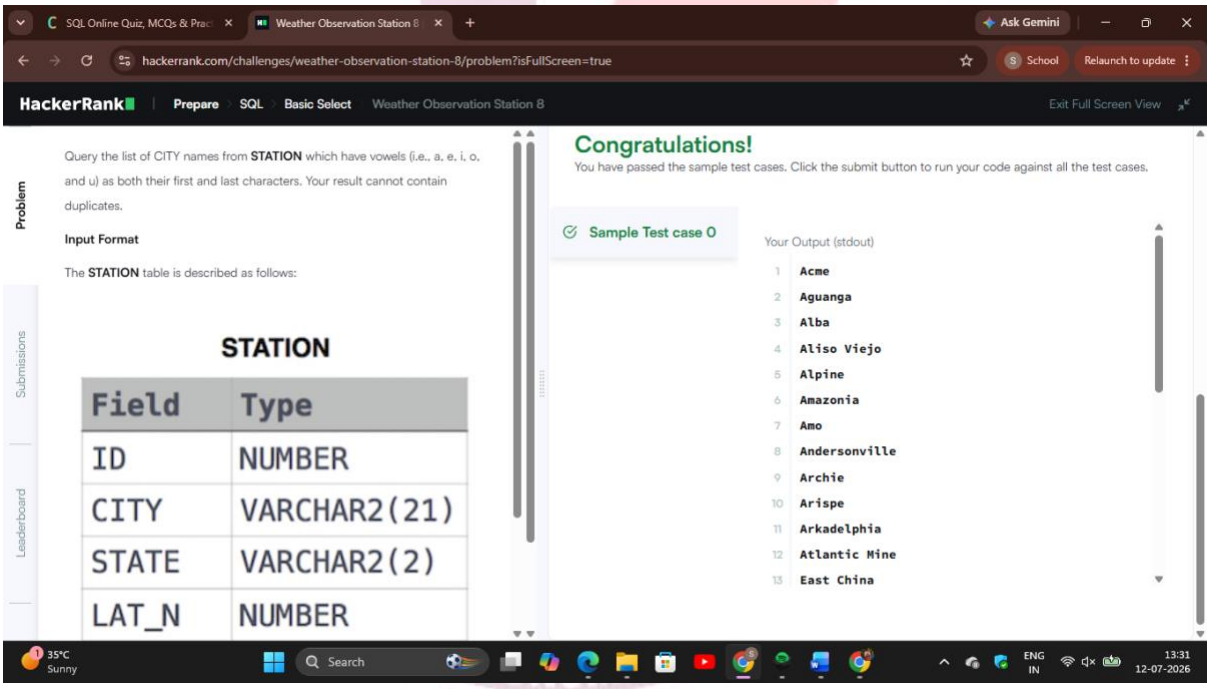
```
SELECT DISTINCT CITY

FROM STATION

WHERE LOWER(LEFT(CITY, 1)) IN ('a', 'e', 'i', 'o', 'u')

  AND LOWER(RIGHT(CITY, 1)) IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT



37.WEATHER OBSERVATION STATION 9

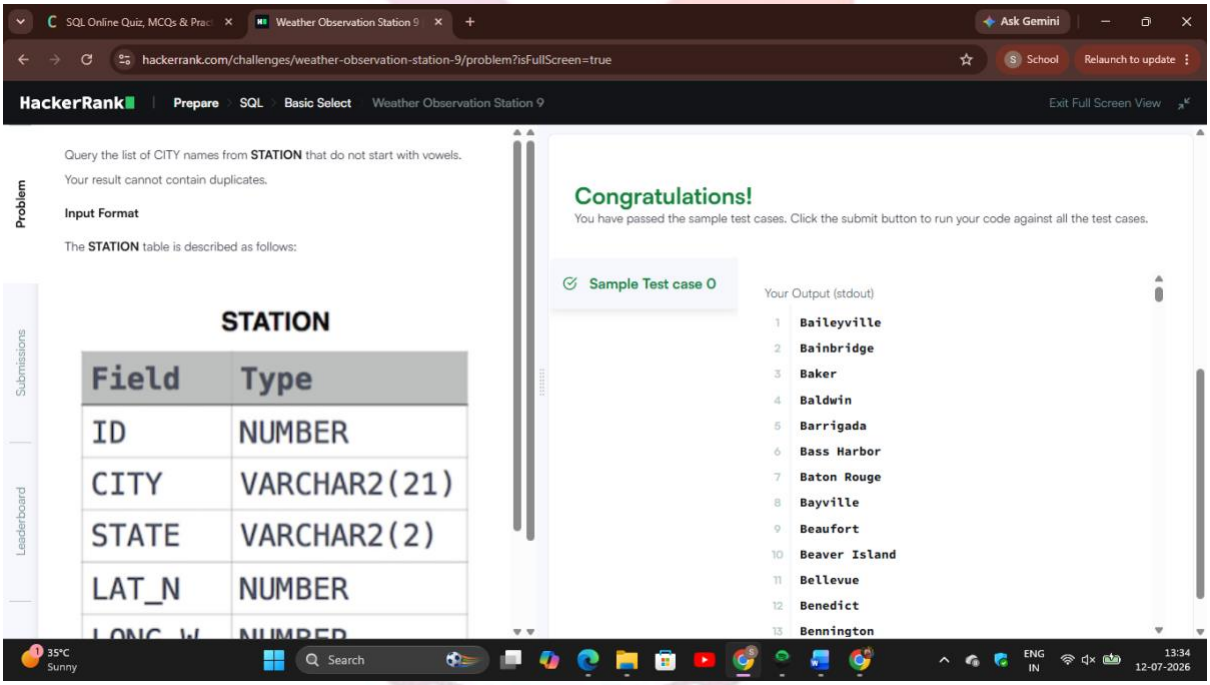
PROGRAM

```
SELECT DISTINCT CITY

FROM STATION

WHERE UPPER(LEFT(CITY, 1)) NOT IN ('A', 'E', 'I', 'O', 'U');
```

OUTPUT



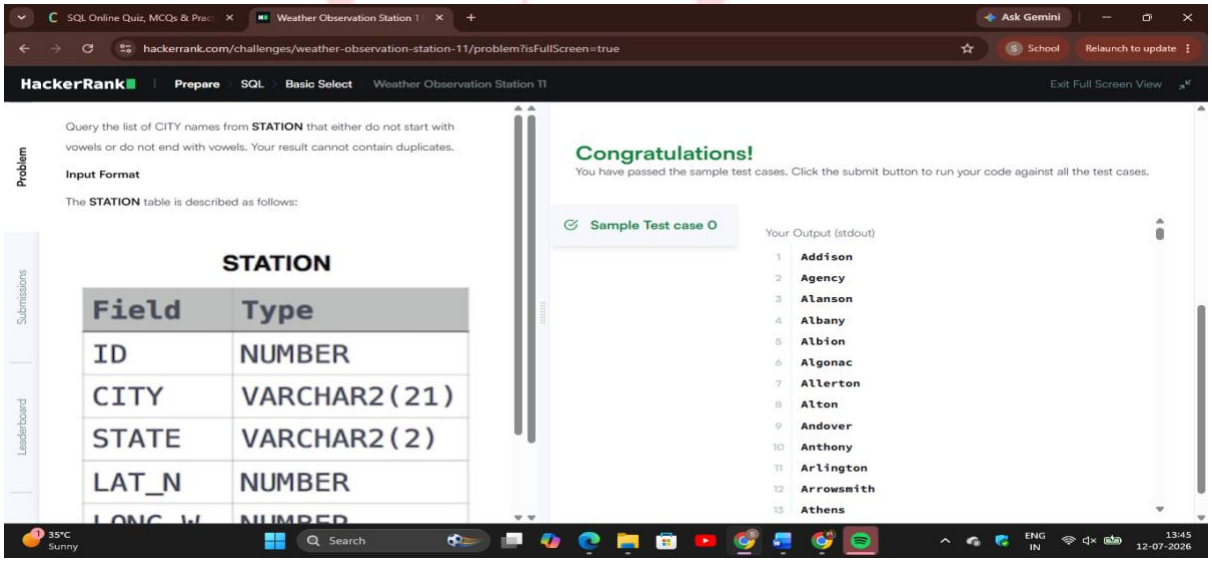
AUTONOMOUS

38.WEATHER OBSERVATION STATION 11

PROGRAM

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY NOT LIKE 'A%'
  AND CITY NOT LIKE 'E%'
  AND CITY NOT LIKE 'I%'
  AND CITY NOT LIKE 'O%'
  AND CITY NOT LIKE 'U%'
  OR CITY NOT LIKE '%a'
  AND CITY NOT LIKE '%e'
  AND CITY NOT LIKE '%i'
  AND CITY NOT LIKE '%o'
  AND CITY NOT LIKE '%u';
```

OUTPUT

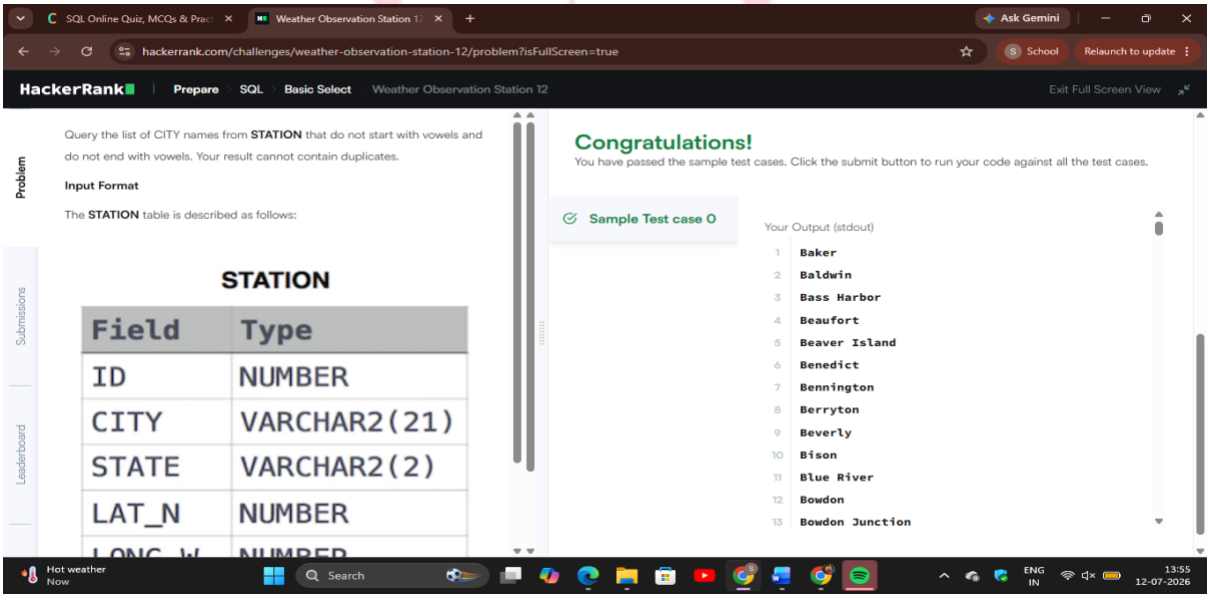


38.WEATHER OBSERVATION STATION 12

PROGRAM

```
SELECT DISTINCT CITY
FROM STATION
WHERE CITY NOT LIKE 'A%'
  AND CITY NOT LIKE 'E%'
  AND CITY NOT LIKE 'I%'
  AND CITY NOT LIKE 'O%'
  AND CITY NOT LIKE 'U%'
  AND CITY NOT LIKE '%a'
  AND CITY NOT LIKE '%e'
  AND CITY NOT LIKE '%i'
  AND CITY NOT LIKE '%o'
  AND CITY NOT LIKE '%u';
```

OUTPUT



39.WEATHER OBSERVATION STATION 13

PROGRAM

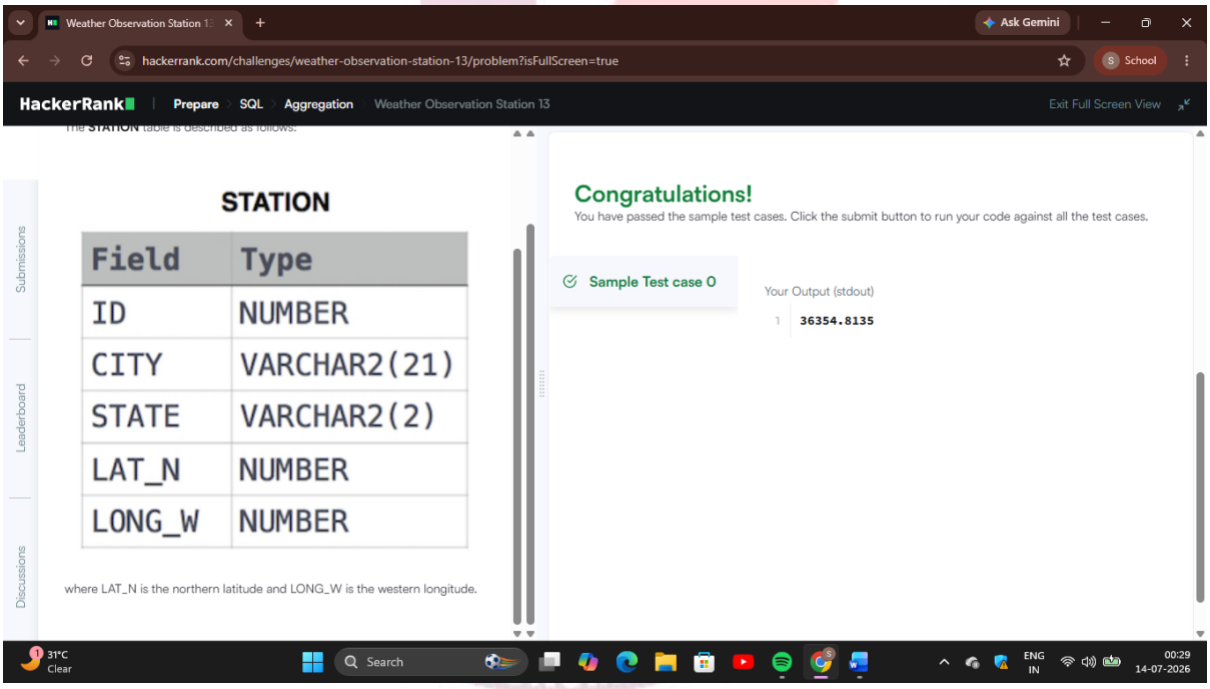
```
SELECT TRUNCATE(SUM(LAT_N), 4)

FROM STATION

WHERE LAT_N > 38.7880

  AND LAT_N < 137.2345;
```

OUTPUT



40.WEATHER OBSERVATION STATION 14

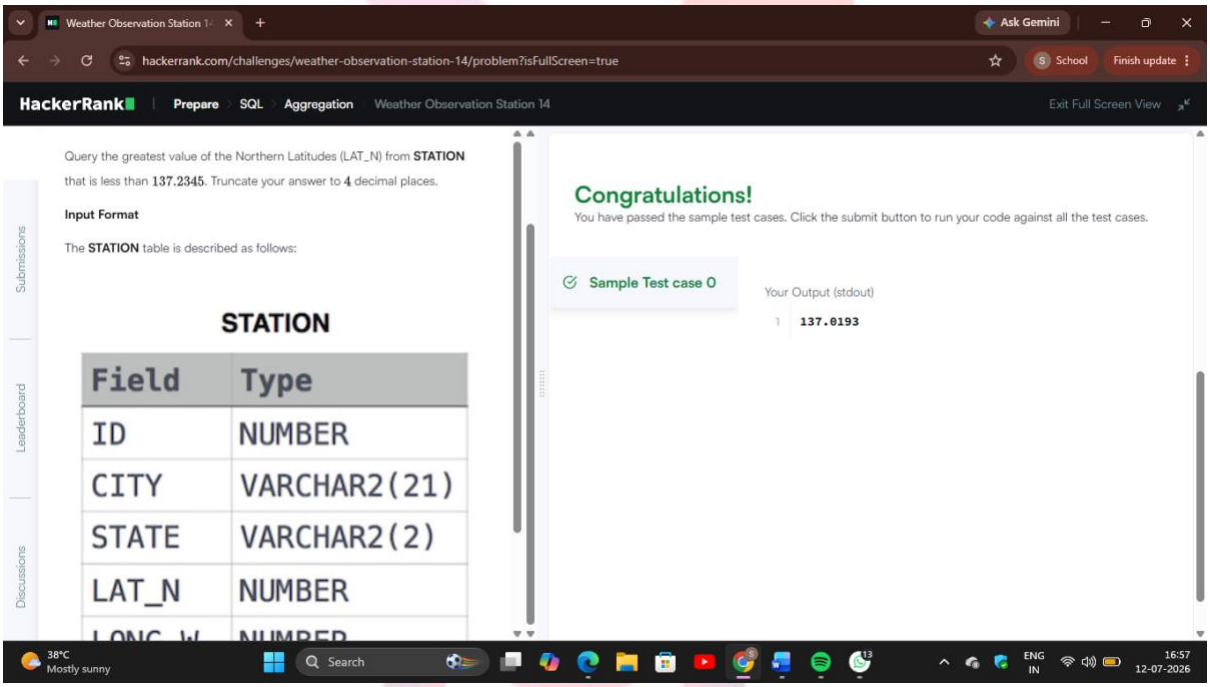
PROGRAM

```
SELECT CAST(TRUNCATE(MAX(LAT_N), 4) AS DECIMAL(10,4))

FROM STATION

WHERE LAT_N < 137.2345;
```

OUTPUT

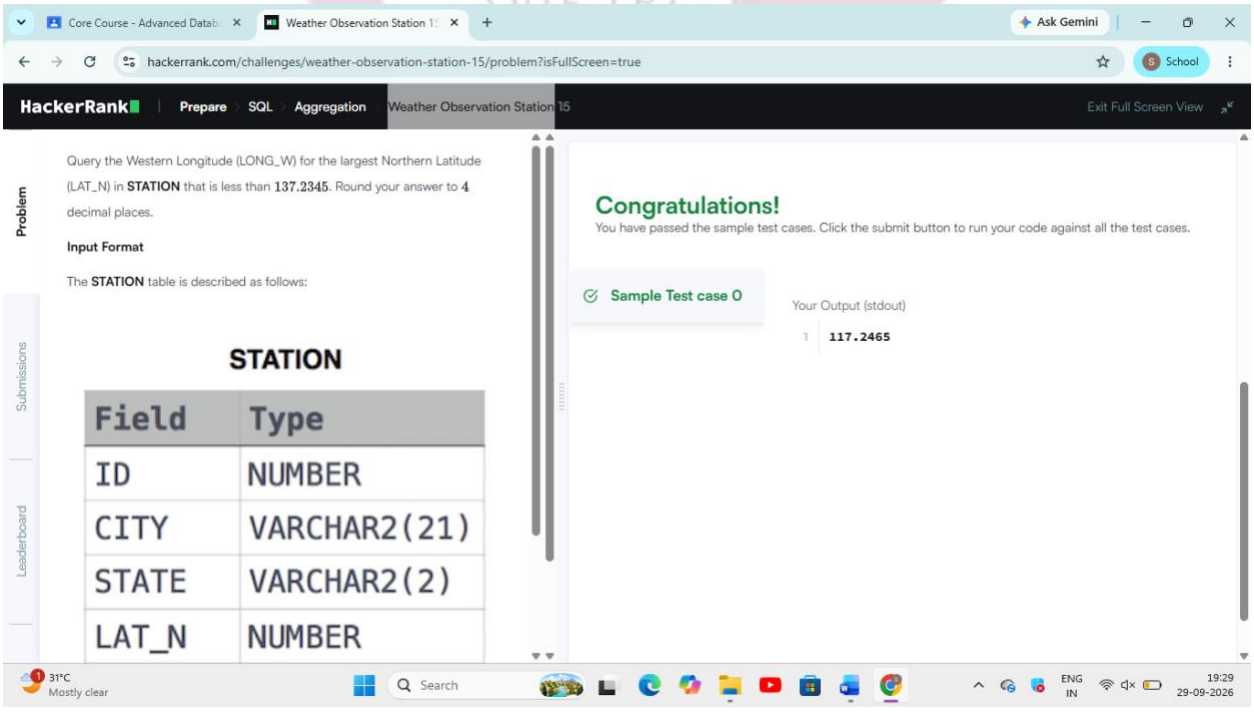


41.WEATHER OBSERVATION STATION 15

PROGRAM

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
    SELECT MAX(LAT_N)
    FROM STATION
    WHERE LAT_N < 137.2345
);
```

OUTPUT

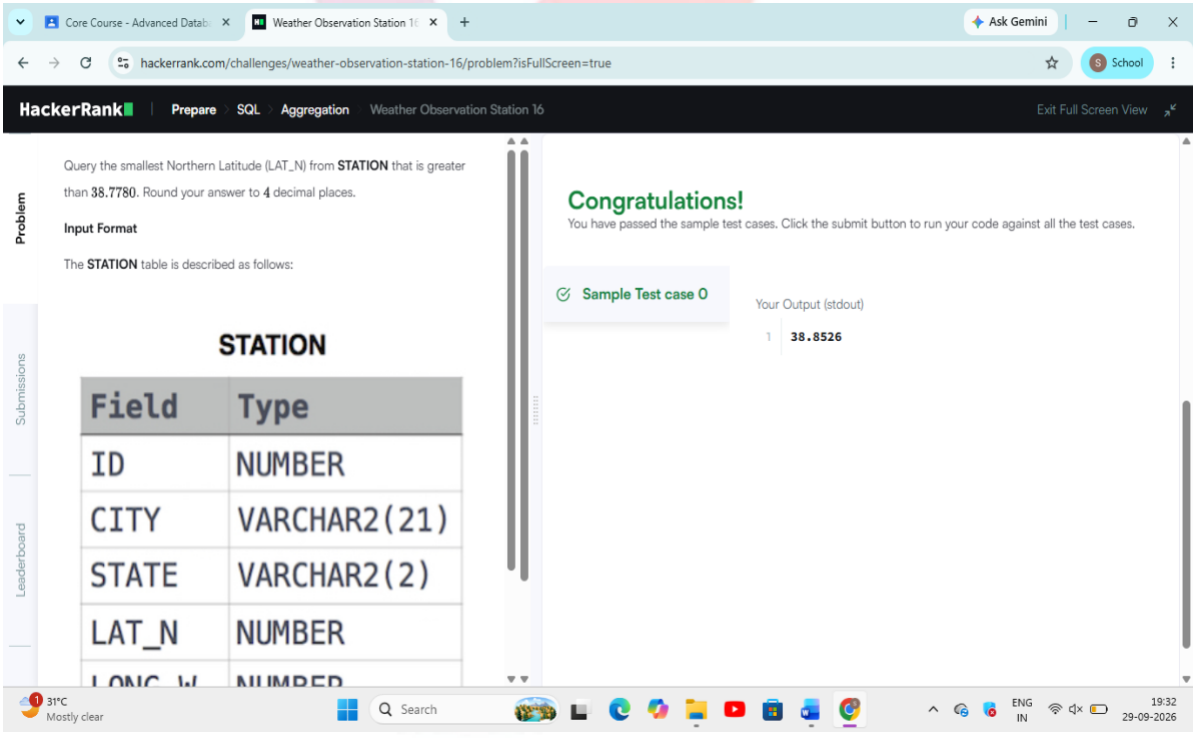


42.WEATHER OBSERVATION STATION 16

PROGRAM

```
SELECT ROUND(MIN(LAT_N), 4)
FROM STATION
WHERE LAT_N > 38.7780;
```

OUTPUT

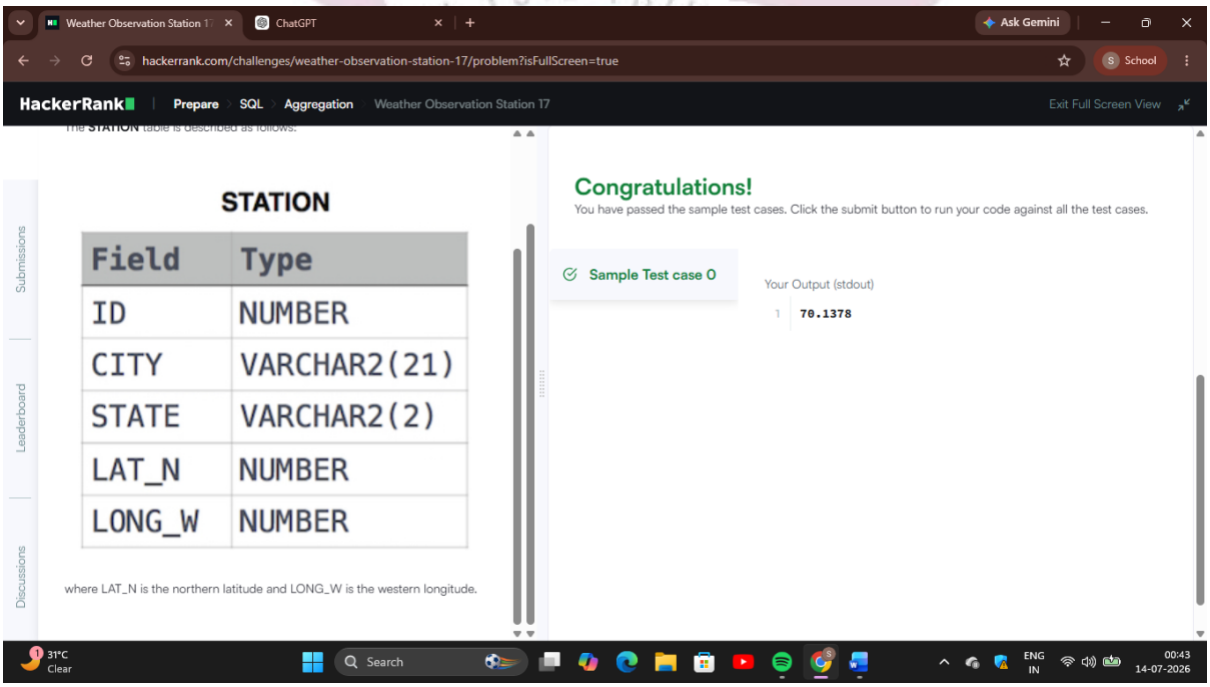


43.WEATHER OBSERVATION STATION 17

PROGRAM

```
SELECT ROUND(LONG_W, 4)
FROM STATION
WHERE LAT_N = (
   SELECT MIN(LAT_N)
   FROM STATION
   WHERE LAT_N > 38.7780
);
```

OUTPUT

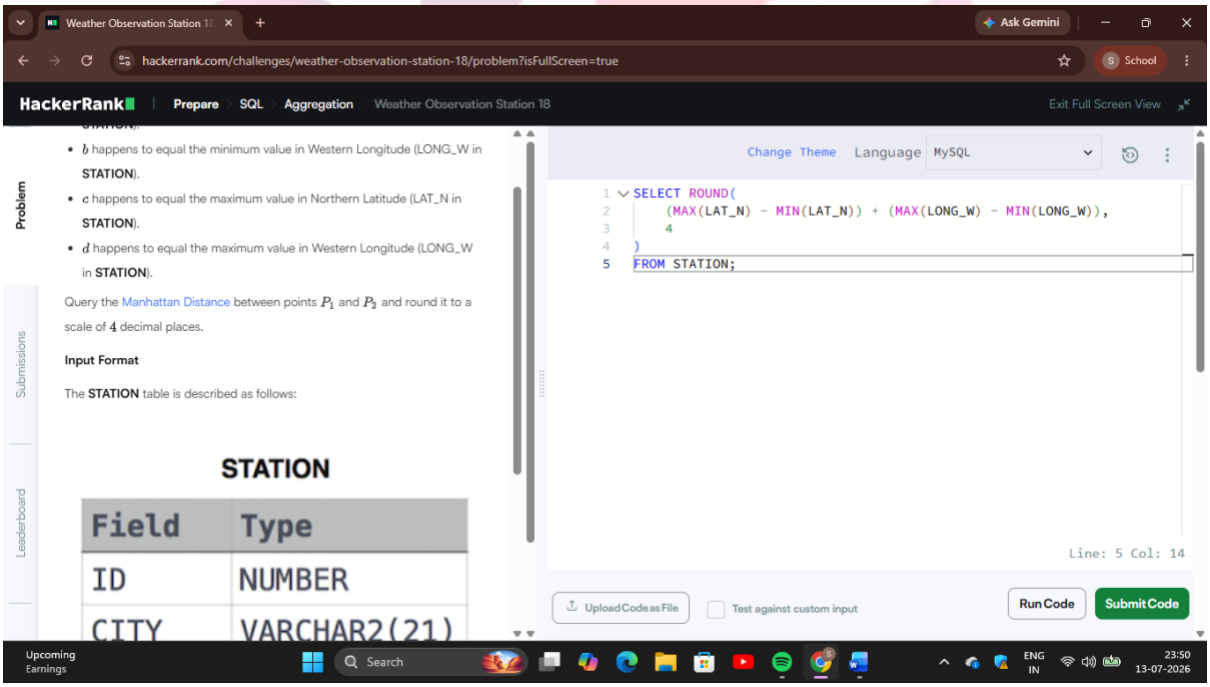


44.WEATHER OBSERVATION STATION 18

PROGRAM

```
SELECT ROUND(
   (MAX(LAT_N) - MIN(LAT_N)) + (MAX(LONG_W) - MIN(LONG_W)),
   4
)
FROM STATION;
```

OUTPUT

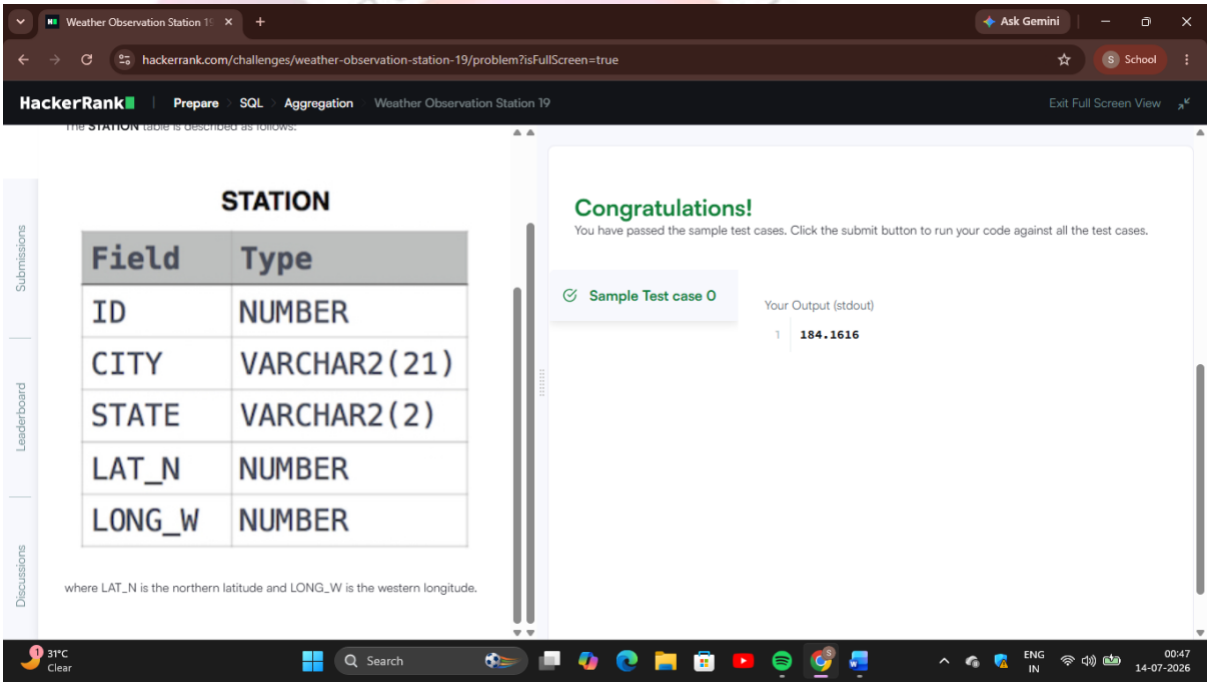


45.WEATHER OBSERVATION STATION 19

PROGRAM

```
SELECT ROUND(
   SQRT(
      POWER(MAX(LAT_N) - MIN(LAT_N), 2) +
      POWER(MAX(LONG_W) - MIN(LONG_W), 2)
   ),
   4
)
FROM STATION;
```

OUTPUT

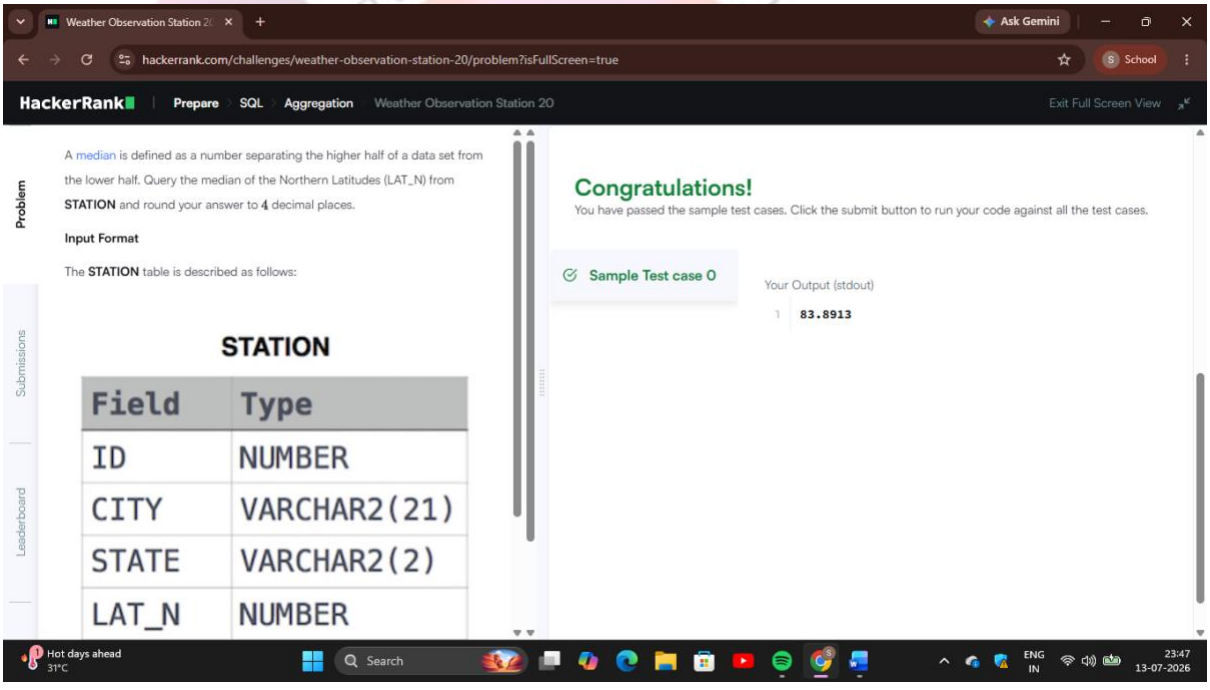


46.WEATHER OBSERVATION STATION 20

PROGRAM

```
SELECT ROUND(AVG(LAT_N), 4)
FROM (
    SELECT LAT_N,
        ROW_NUMBER() OVER (ORDER BY LAT_N) AS rn,
        COUNT(*) OVER () AS cnt
    FROM STATION
) AS t
WHERE rn IN ((cnt + 1) / 2, (cnt + 2) / 2);
```

OUTPUT



47.WEATHER OBSVERTION STATION 10

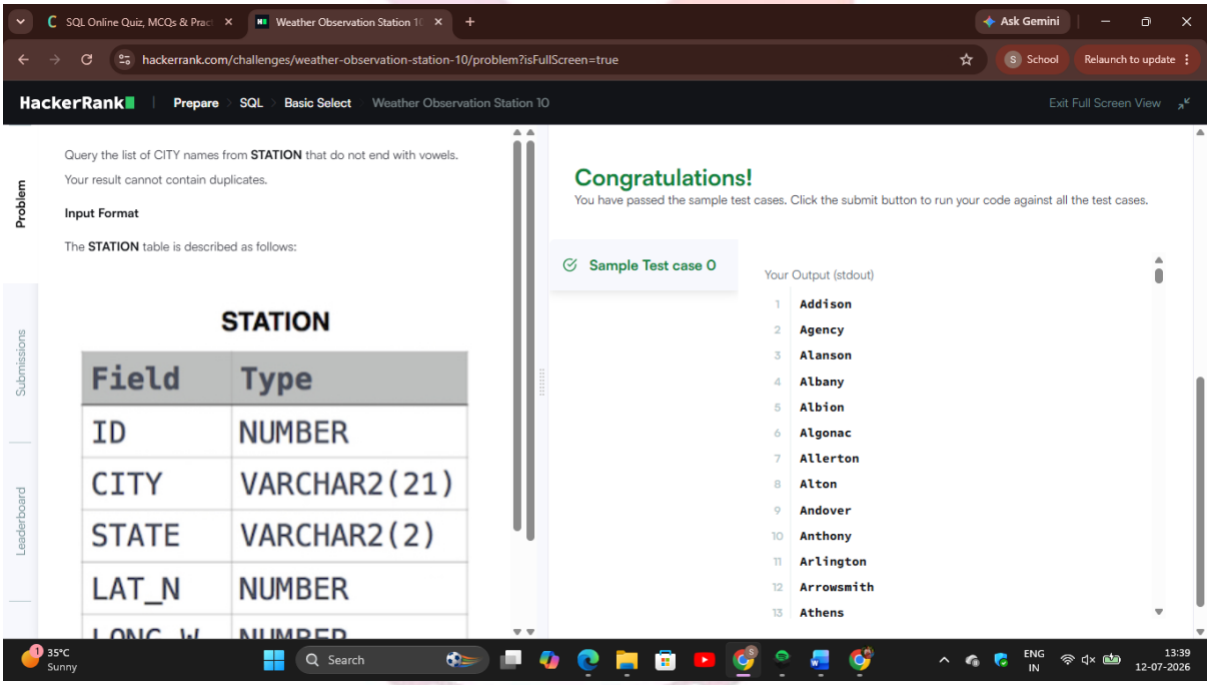
PROGRAM

```
SELECT DISTINCT CITY

FROM STATION

WHERE LOWER(RIGHT(CITY, 1)) NOT IN ('a', 'e', 'i', 'o', 'u');
```

OUTPUT



48.REVISING THE SELECT QUERY I

PROGRAM

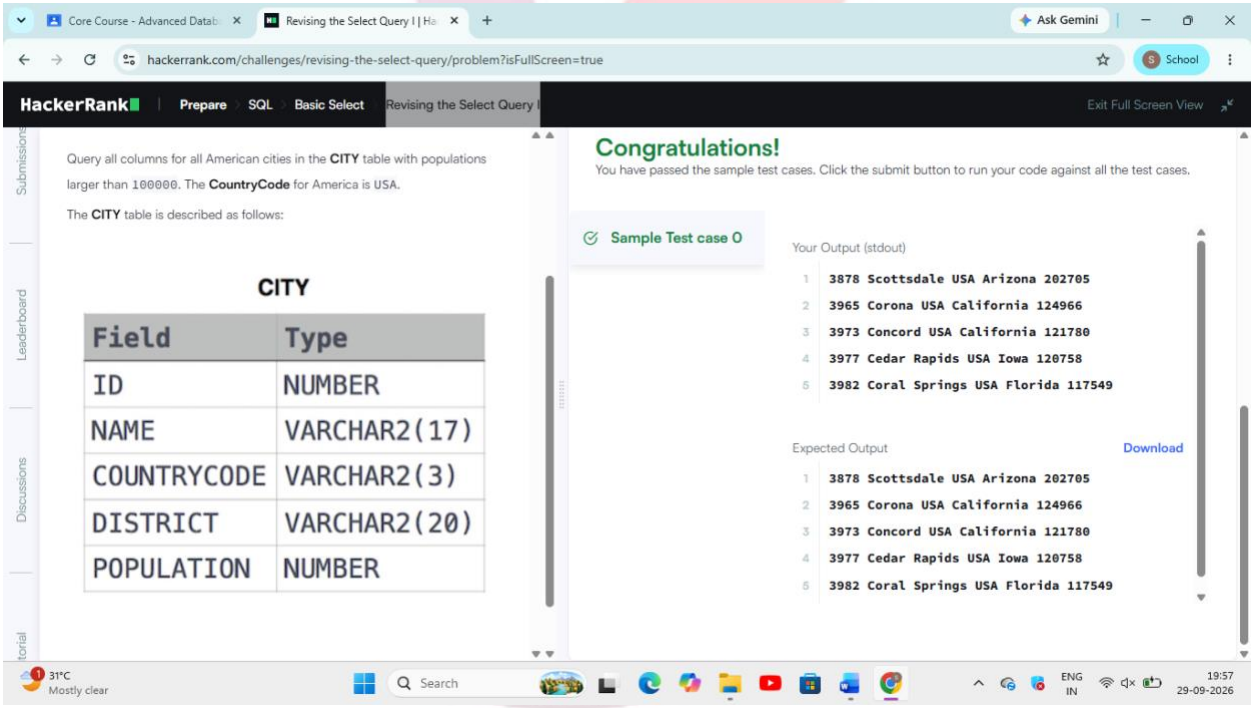
```
SELECT *

FROM CITY

WHERE CountryCode = 'USA'

  AND Population > 100000;
```

OUTPUT



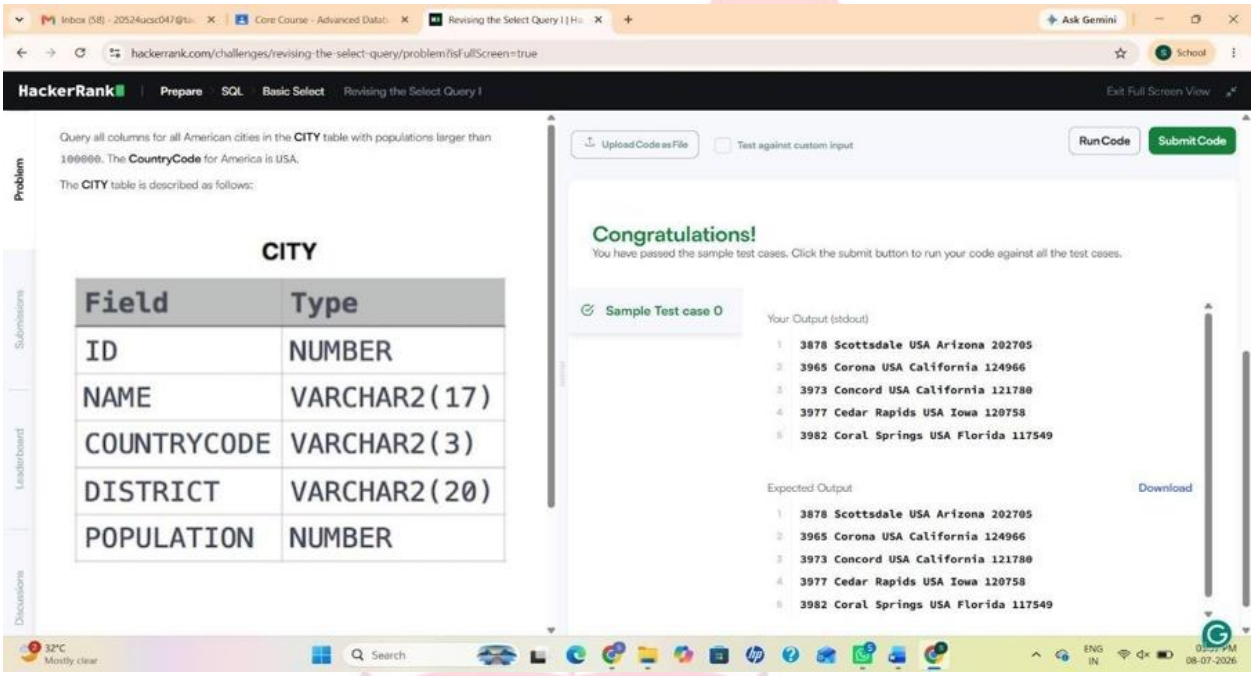
49.REVISING THE SELECT QUERY I

PROGRAM

```
SELECT * FROM CITY

WHERE COUNTRYCODE = 'USA' AND POPULATION > 100000;
```

OUTPUT



50.REVISING THE SELECT QUERY II

PROGRAM

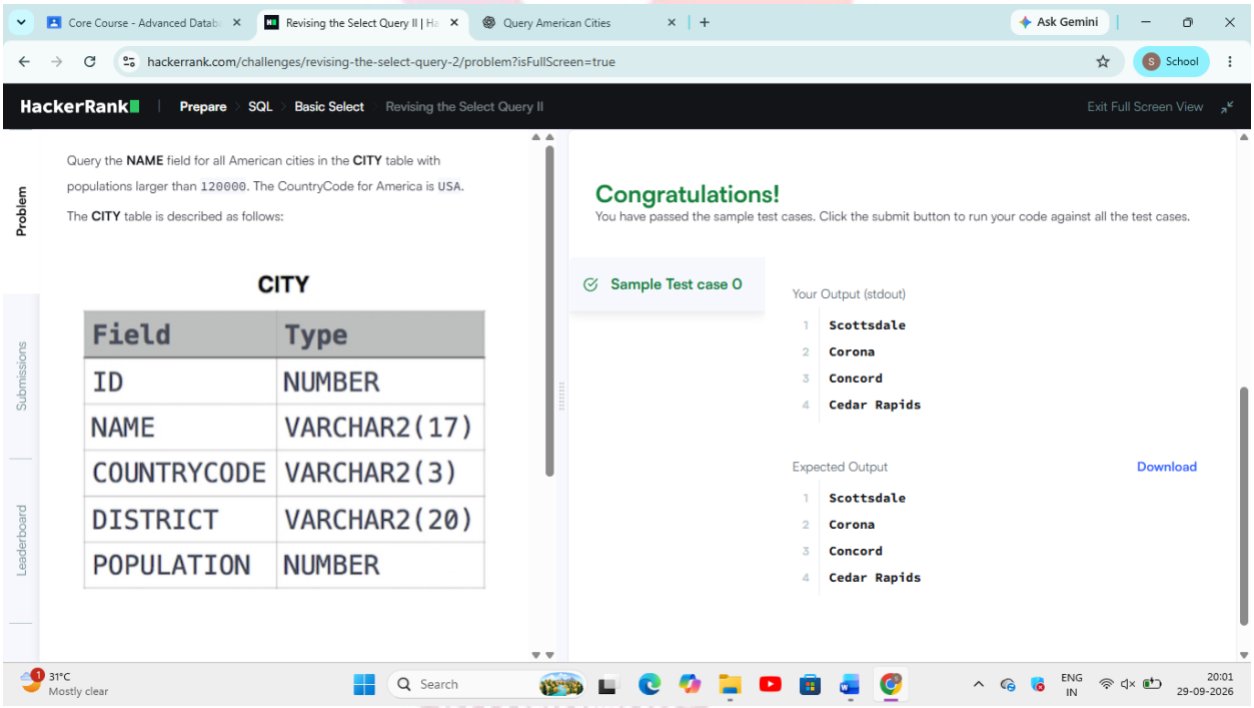
```
SELECT NAME

FROM CITY

WHERE COUNTRYCODE = 'USA'

 AND POPULATION > 120000;
```

OUTPUT

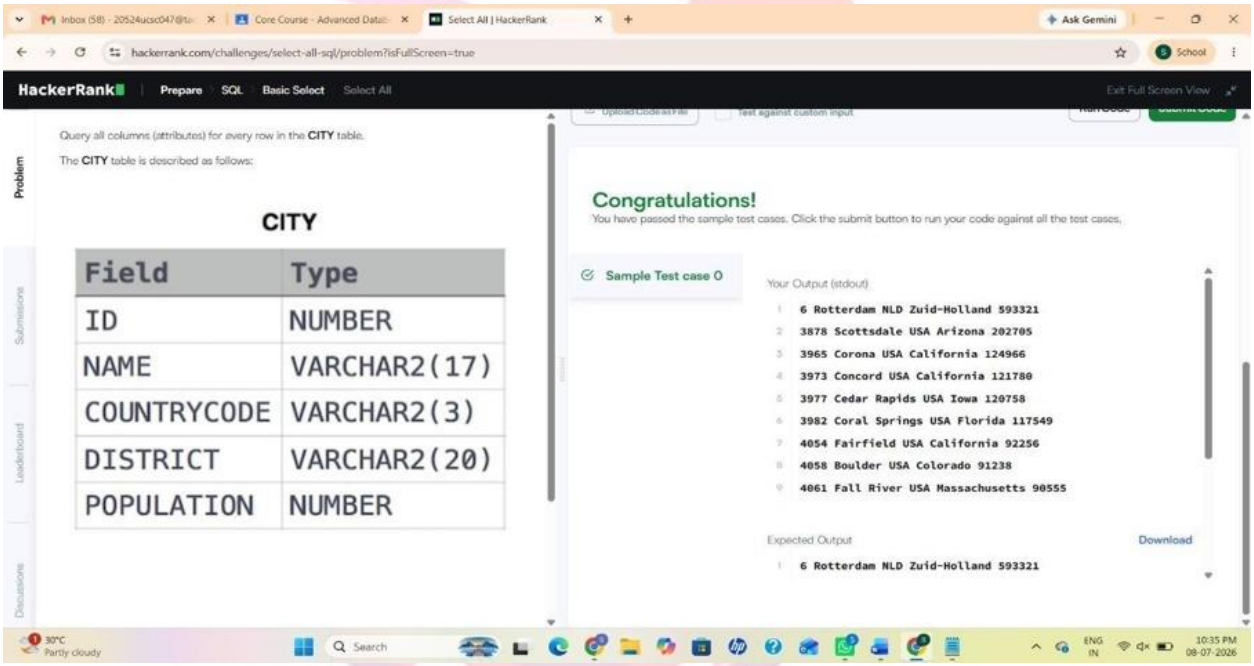


51.SELECT ALL

PROGRAM

```
SELECT t.* FROM CITY t;
```

OUTPUT



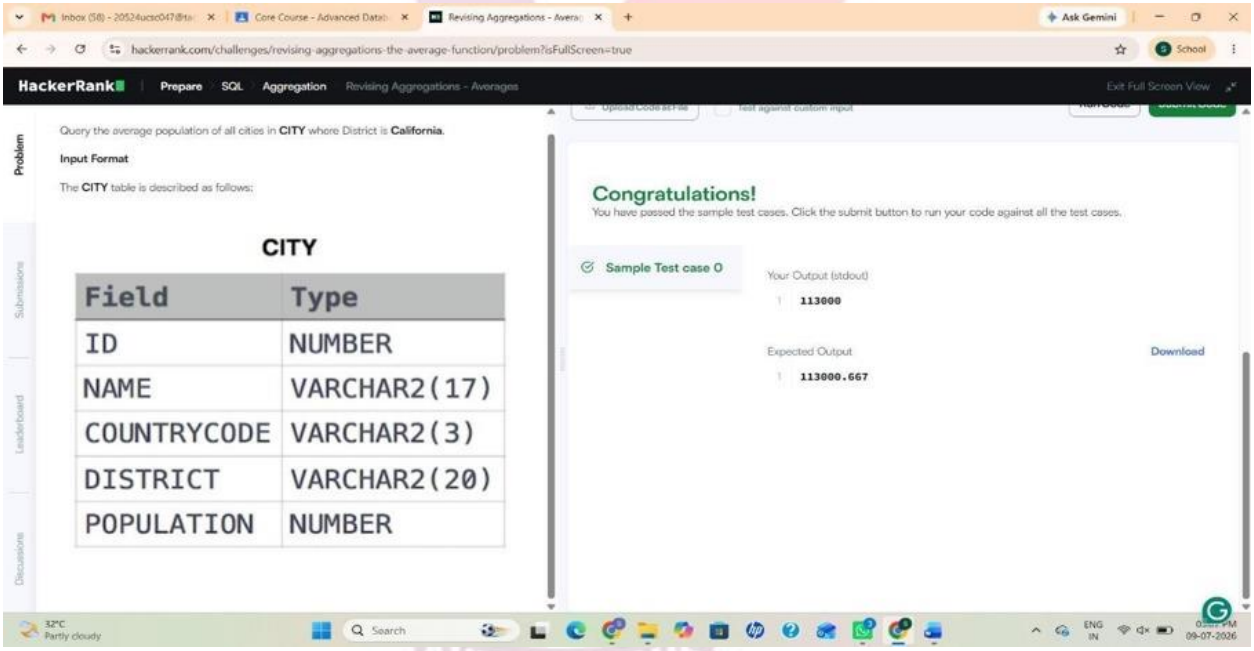
52.REVISING AGGREGATIONS – AVERAGES

PROGRAM

```
SELECT AVG(POPULATION) FROM CITY

WHERE DISTRICT = 'California';
```

OUTPUT



EX NO:14 **MINI PROJECT**

E-COMMERCE ORDER AND INVENTORY MANAGEMENT

AIM

To develop an E-Commerce Order and Inventory Management system using SQL and PL/SQL to analyze customer purchases, product sales, order values, inventory levels, and country-wise customer information.

SOFTWARE USED

- Github Codespace
- SQLite
- Oracle database
- SQL
- PL/SQL

DESCRIPTION

This project analyzes e-commerce transaction data based on the UCI Online Retail dataset. The system stores transaction details such as invoice number, product code, product description, quantity, invoice date, unit price, customer ID, country, and order status.

SQL queries are used to:

- Display e-commerce transaction records.
- Calculate product-wise sales and total sales.
- Calculate order values and identify high-value orders.
- Identify products with low inventory.
- Retrieve order and customer details.

PL/SQL functions and procedures are used to calculate order totals, retrieve customer-wise details, and generate inventory reports. Exception handling is implemented for invalid orders, missing products, insufficient stock, and invalid input conditions.

DATASET

Dataset Name: Online Retail

Source: UCI Machine Learning Repository – Online Retail and Kaggle – Online Retail

Number of records used in this mini project: 25

FIELDS

Field Name	Data Type	Description
PRODUCT_ID	NUMBER	Unique product ID
PRODUCT_NAME	VARCHAR2(50)	Name of the product
CATEGORY	VARCHAR2(30)	Product category
PRICE	NUMBER(10,2)	Price of the product
STOCK	NUMBER	Available stock quantity
CUSTOMER_ID	NUMBER	Unique customer ID
CUSTOMER_NAME	VARCHAR2(50)	Customer name
ORDER_ID	NUMBER	Unique order ID
ORDER_DATE	DATE	Date of order
QUANTITY	NUMBER	Quantity ordered

Added fields: CATEGORY and STOCK are included to support product classification and inventory management.

TABLE CREATION

Products Table

```
CREATE TABLE PRODUCTS (
    PRODUCT_ID NUMBER PRIMARY KEY,
    PRODUCT_NAME VARCHAR2(50) NOT NULL,
    CATEGORY VARCHAR2(30),
    PRICE NUMBER(10,2) CHECK (PRICE >= 0),
    STOCK NUMBER CHECK (STOCK >= 0)
);
```

INSERT PRODUCT DATA

```
insert into products values (101,'85123A','white hanging heart t-light holder',2.55,50);
insert into products values (102,'71053','white metal lantern',3.39,40);
insert into products values (103,'84406B','cream cupid hearts coat hanger',2.75,35);
insert into products values (104,'84029G','knitted union flag hot water bottle',3.39,30);
```

insert into products values (105,'84029E','red woolly hottie white heart',3.39,25);

insert into products values (106,'22752','set 7 babushka nesting boxes',7.65,20);

insert into products values (107,'21730','glass star frosted t-light holder',4.25,45);

insert into products values (108,'22633','hand warmer union jack',1.85,60);

insert into products values (109,'22632','hand warmer red polka dot',1.85,55);

insert into products values (110,'84879','assorted colour bird ornament',1.69,70);

OUTPUT:

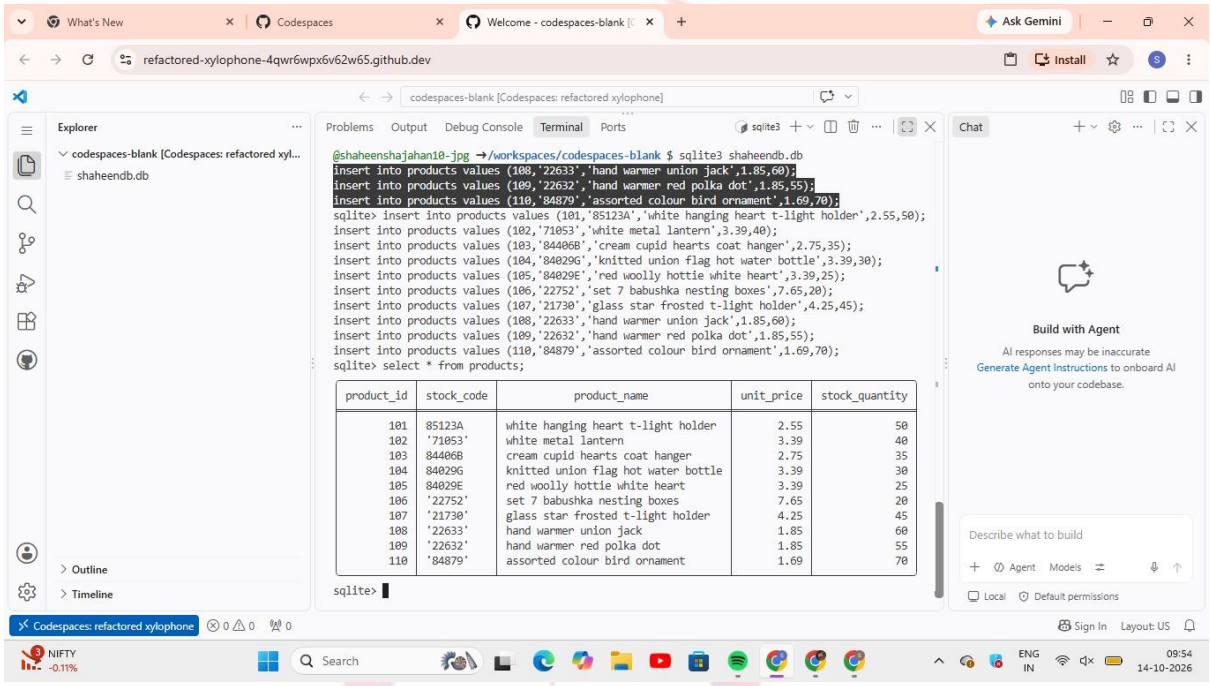


TABLE CREATION

Orders Table

```
CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    order_number TEXT,
    customer_id INTEGER,
    product_id INTEGER,
    quantity INTEGER,
    order_date TEXT,
    status TEXT
);
```

INSERT ORDERS TABLE

INSERT INTO orders VALUES

(1,'536360',101,1001,10,'2010-12-01','PLACED'),

(2,'536361',102,1002,12,'2010-12-01','SHIPPED'),

(3,'536362',103,1003,15,'2010-12-01','DELIVERED'),

(4,'536363',104,1004,8,'2010-12-01','PLACED'),

(5,'536364',105,1005,20,'2010-12-01','SHIPPED'),

(6,'536365',106,1001,5,'2010-12-01','DELIVERED'),

(7,'536366',107,1002,9,'2010-12-01','PLACED'),

(8,'536367',108,1003,14,'2010-12-01','SHIPPED'),

(9,'536368',109,1004,7,'2010-12-01','DELIVERED'),

(10,'536369',110,1005,11,'2010-12-01','PLACED'),

(11,'536370',101,1001,16,'2010-12-01','SHIPPED'),

(12,'536371',102,1002,6,'2010-12-01','DELIVERED'),

(13,'536372',103,1003,13,'2010-12-01','PLACED'),

(14,'536373',104,1004,17,'2010-12-01','SHIPPED'),

(15,'536374',105,1005,4,'2010-12-01','DELIVERED'),

(16,'536375',106,1001,19,'2010-12-01','PLACED'),

(17,'536376',107,1002,7,'2010-12-01','DELIVERED'),

(18,'536377',108,1003,15,'2010-12-01','PLACED'),

(19,'536378',109,1004,20,'2010-12-01','SHIPPED'),

(20,'536379',110,1005,10,'2010-12-01','DELIVERED'),

(21,'536380',101,1001,9,'2010-12-01','PLACED'),

(22,'536381',102,1002,11,'2010-12-01','DELIVERED'),

(23,'536382',103,1003,13,'2010-12-01','SHIPPED'),

(24,'536383',104,1004,18,'2010-12-01','PLACED'),

(25,'536384',105,1005,22,'2010-12-01','DELIVERED');

OUTPUT:

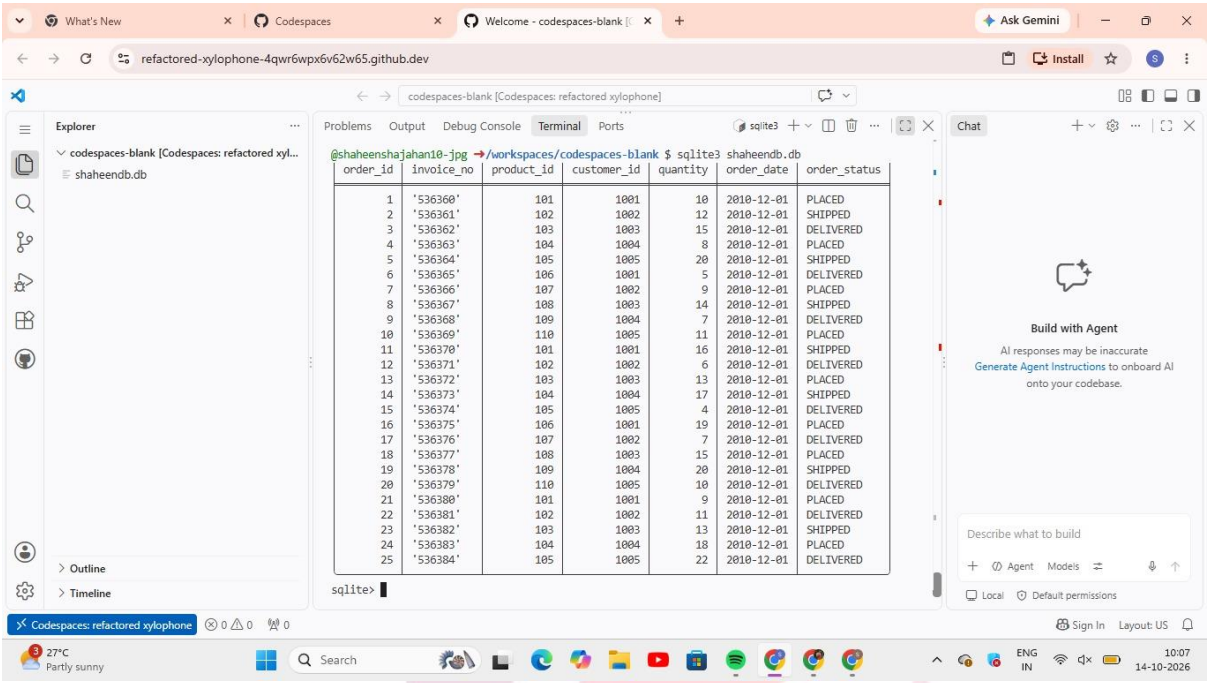


TABLE CREATION

Customers Table

```
create table customers (
    customer_id number primary key,
    customer_name varchar2(100) not null,
    country varchar2(50)
);
```

INSERT CUSTOMER DATA

```
INSERT INTO customers VALUES
(1, 'Arun', 'India'),
(2, 'Priya', 'India'),
(3, 'Kumar', 'India'),
(4, 'Divya', 'India'),
(5, 'Ravi', 'India'),
(6, 'Anu', 'India'),
(7, 'Suresh', 'India'),
(8, 'Meena', 'India'),
```

(9, 'Karthik', 'India'),

(10, 'Lakshmi', 'India'),

(11, 'John', 'USA'),

(12, 'David', 'UK'),

(13, 'Maria', 'Canada'),

(14, 'Peter', 'Australia'),

(15, 'Sara', 'USA'),

(16, 'Rahul', 'India'),

(17, 'Sneha', 'India'),

(18, 'Vijay', 'India'),

(19, 'Nisha', 'India'),

(20, 'Ajay', 'India'),

(21, 'Robert', 'UK'),

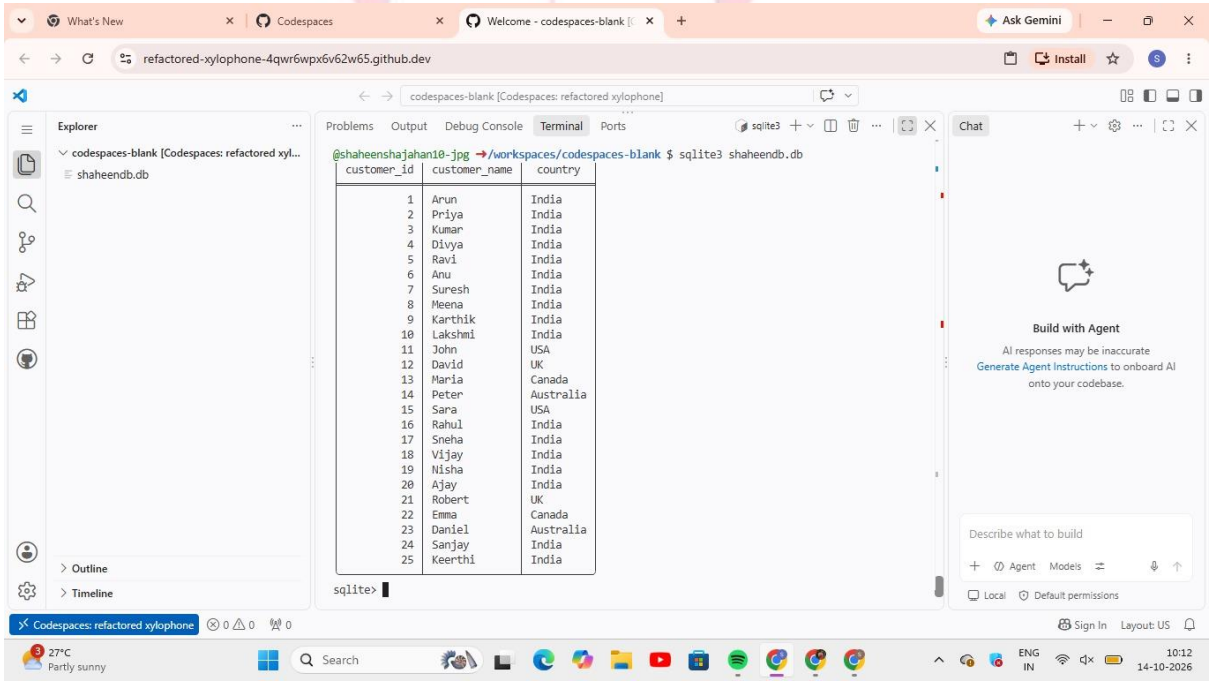
(22, 'Emma', 'Canada'),

(23, 'Daniel', 'Australia'),

(24, 'Sanjay', 'India'),

(25, 'Keerthi', 'India');

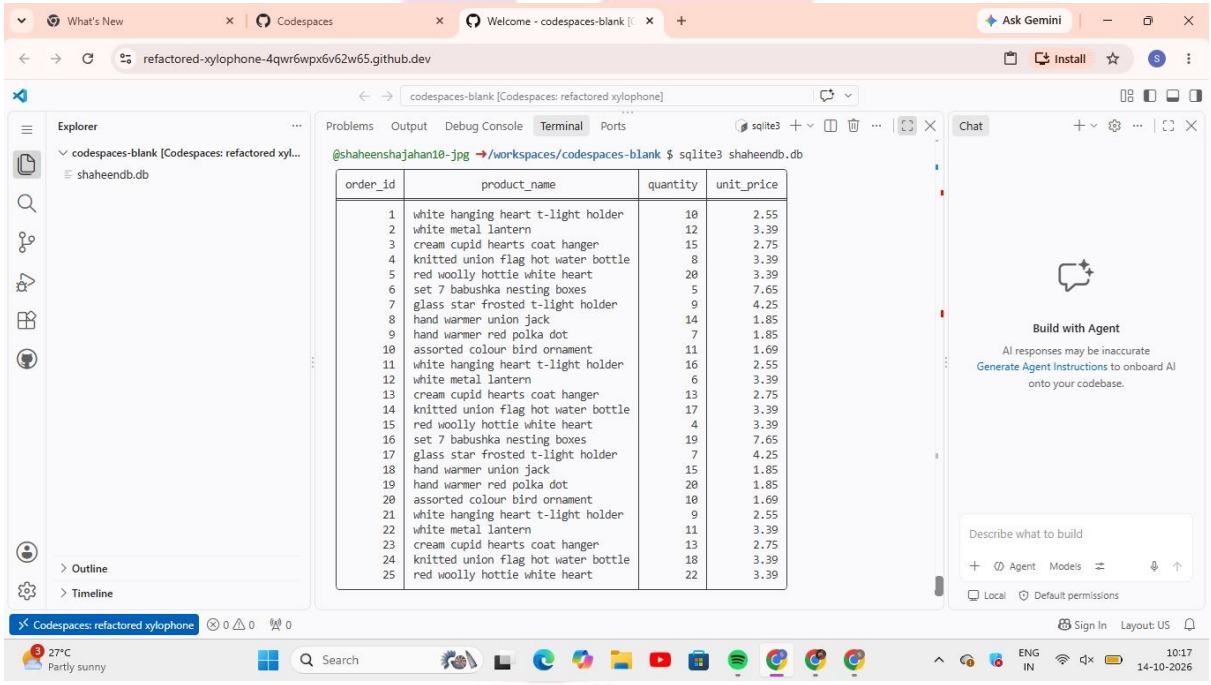
OUTPUT:



DISPLAY ORDER AND PRODUCT DETAILS

```
select
    o.order_id,
    p.product_name,
    o.quantity,
    p.unit_price
from orders o
join products p
on o.product_id = p.product_id;
```

OUTPUT:

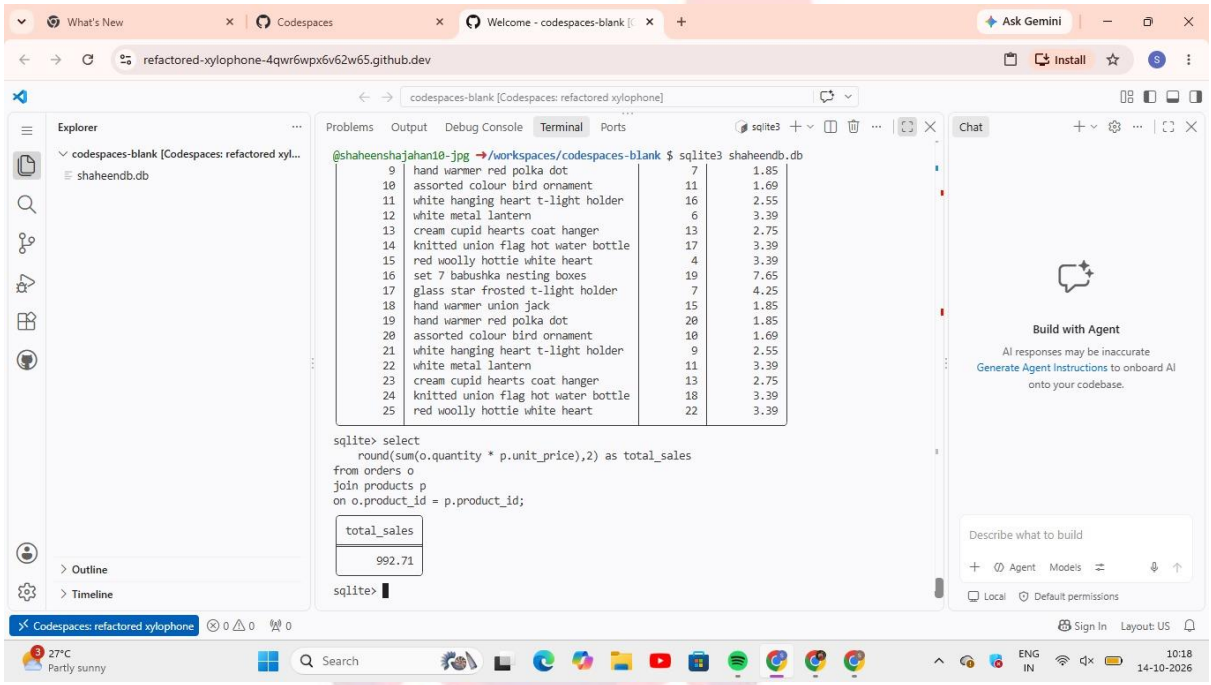


order_id	product_name	quantity	unit_price
1	white hanging heart t-light holder	10	2.55
2	white metal lantern	12	3.39
3	cream cupid hearts coat hanger	15	2.75
4	knitted union flag hot water bottle	8	3.39
5	red woolly hottie white heart	20	3.39
6	set 7 babushka nesting boxes	5	7.65
7	glass star frosted t-light holder	9	4.25
8	hand warmer union jack	14	1.85
9	hand warmer red polka dot	7	1.85
10	assorted colour bird ornament	11	1.69
11	white hanging heart t-light holder	16	2.55
12	white metal lantern	6	3.39
13	cream cupid hearts coat hanger	13	2.75
14	knitted union flag hot water bottle	17	3.39
15	red woolly hottie white heart	4	3.39
16	set 7 babushka nesting boxes	19	7.65
17	glass star frosted t-light holder	7	4.25
18	hand warmer union jack	15	1.85
19	hand warmer red polka dot	20	1.85
20	assorted colour bird ornament	10	1.69
21	white hanging heart t-light holder	9	2.55
22	white metal lantern	11	3.39
23	cream cupid hearts coat hanger	13	2.75
24	knitted union flag hot water bottle	18	3.39
25	red woolly hottie white heart	22	3.39

CALCULATE TOTAL SALES

```
select
    round(sum(o.quantity * p.unit_price),2) as total_sales
from orders o
join products p
on o.product_id = p.product_id;
```

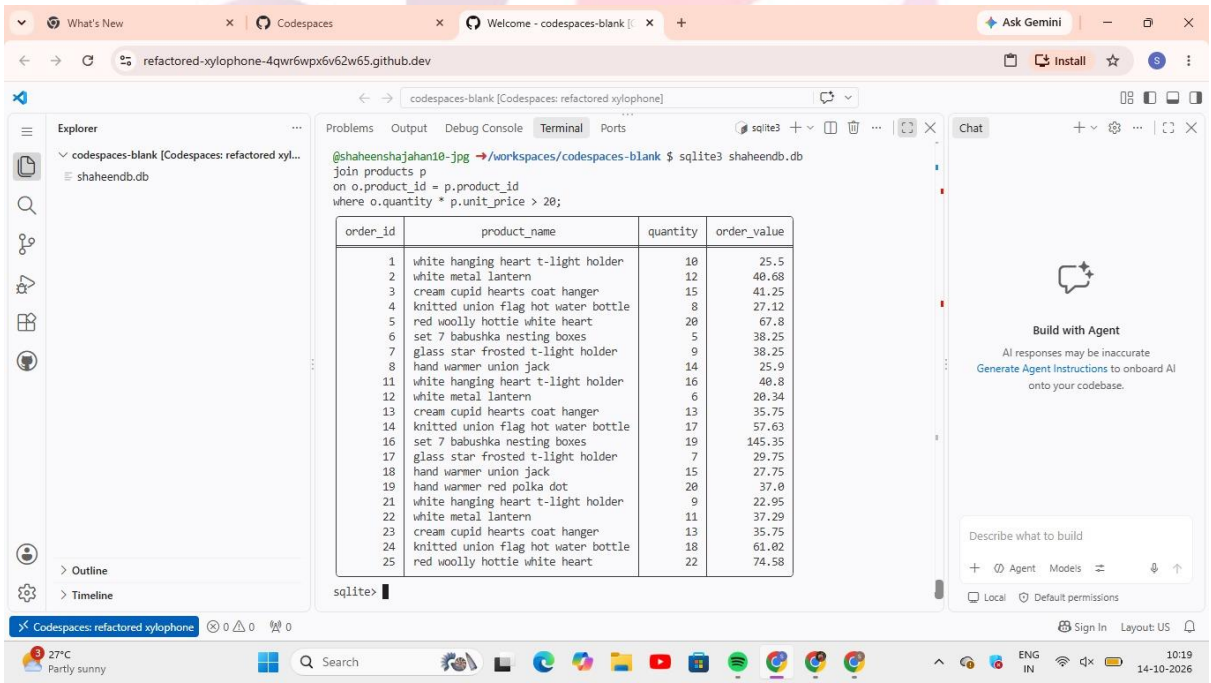
OUTPUT:



FIND HIGH-VALUE ORDERS

```
select
    o.order_id,
    p.product_name,
    o.quantity,
    round(o.quantity * p.unit_price,2) as order_value
from orders o
join products p
on o.product_id = p.product_id
where o.quantity * p.unit_price > 20;
```

OUTUT:

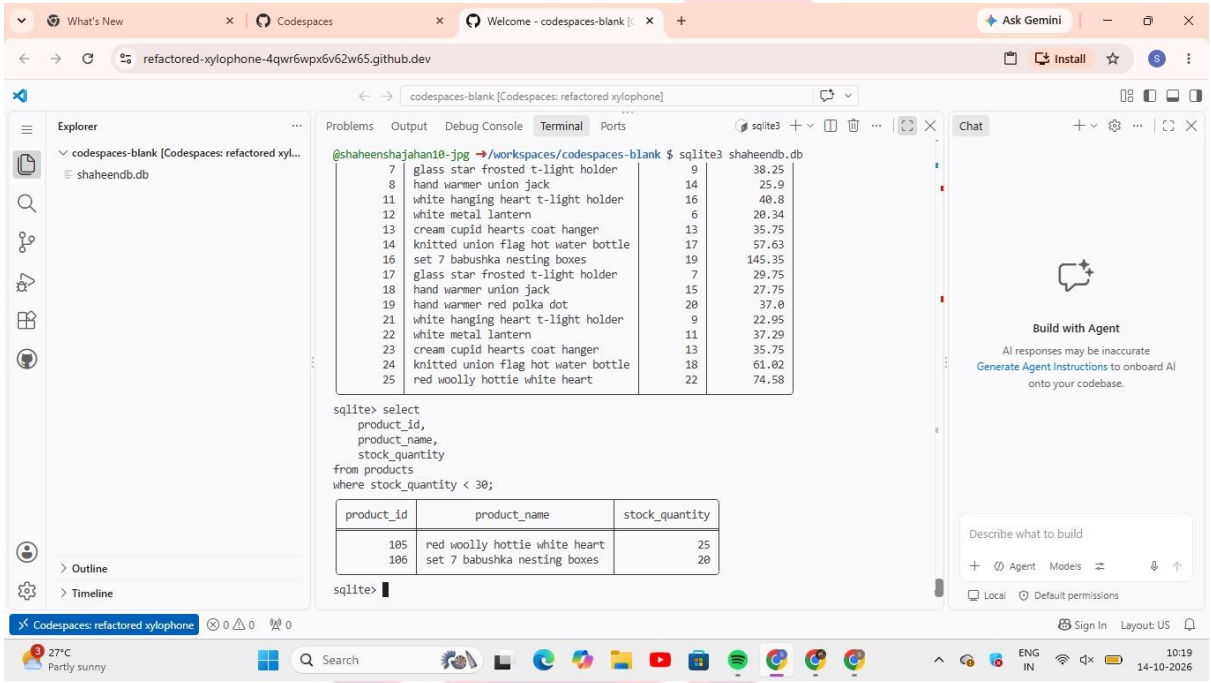


order_id	product_name	quantity	order_value
1	white hanging heart t-light holder	10	25.5
2	white metal lantern	12	40.68
3	cream cupid hearts coat hanger	15	41.25
4	knitted union flag hot water bottle	8	27.12
5	red woolly hottie white heart	20	67.8
6	set 7 babushka nesting boxes	5	38.25
7	glass star frosted t-light holder	9	38.25
8	hand warmer union jack	14	25.9
11	white hanging heart t-light holder	16	40.8
12	white metal lantern	6	20.34
13	cream cupid hearts coat hanger	13	35.75
14	knitted union flag hot water bottle	17	57.63
16	set 7 babushka nesting boxes	19	145.35
17	glass star frosted t-light holder	7	29.75
18	hand warmer union jack	15	27.75
19	hand warmer red polka dot	20	37.0
21	white hanging heart t-light holder	9	22.95
22	white metal lantern	11	37.29
23	cream cupid hearts coat hanger	13	35.75
24	knitted union flag hot water bottle	18	61.02
25	red woolly hottie white heart	22	74.58

FIND LOW-STOCK PRODUCTS

```
select
    product_id,
    product_name,
    stock_quantity
from products
where stock_quantity < 30;
```

OUTUT:

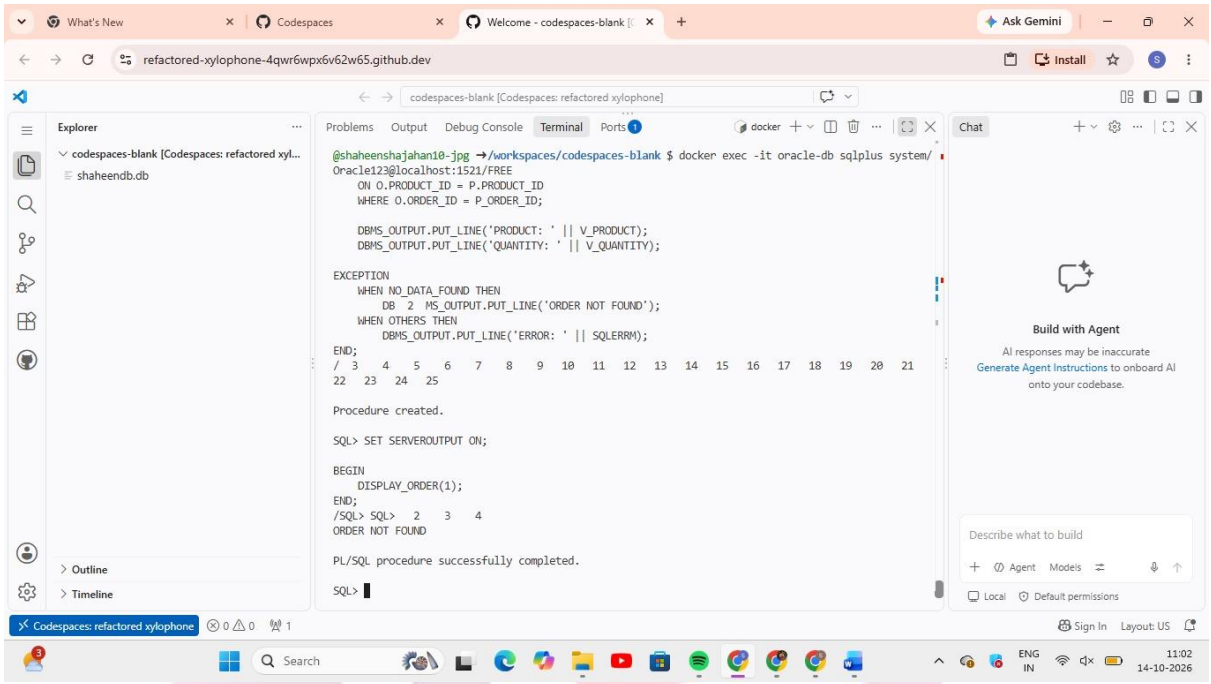


PL/SQL

DISPLAY ORDER DETAILS

```
create or replace procedure display_order
(
   p_order_id number
)
is
   v_product varchar2(100);
   v_quantity number;
begin
   select p.product_name, o.quantity
   into v_product, v_quantity
   from orders o
   join products p
   on o.product_id = p.product_id
   where o.order_id = p_order_id;
   dbms_output.put_line('product: ' || v_product);
   dbms_output.put_line('quantity: ' || v_quantity);
exception
   when no_data_found then
      dbms_output.put_line('order not found');
   when others then
      dbms_output.put_line('error: ' || sqlerrm);
end;
/
```

OUTPUT:

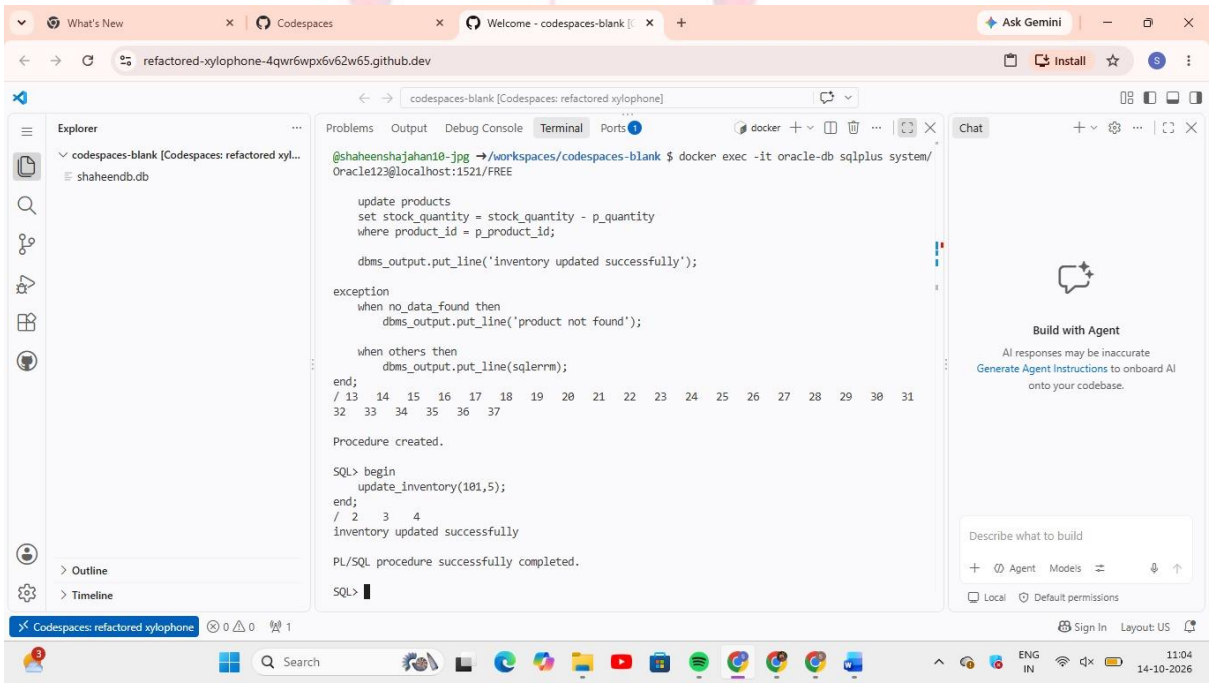


UPDATE INVENTORY

```
create or replace procedure update_inventory(
    p_product_id number,
    p_quantity number
)
is
    v_stock number;
begin
    select stock_quantity
    into v_stock
    from products
    where product_id = p_product_id;
    if p_quantity <= 0 then
        raise_application_error(-20002,
        'quantity must be greater than zero');
```

```
    end if;
    if v_stock < p_quantity then
        raise_application_error(-20001,
        'insufficient stock');
    end if;
    update products
    set stock_quantity = stock_quantity - p_quantity
    where product_id = p_product_id;
    dbms_output.put_line('inventory updated successfully');
exception
    when no_data_found then
        dbms_output.put_line('product not found');
    when others then
        dbms_output.put_line(sqlerrm);
end;
/
```

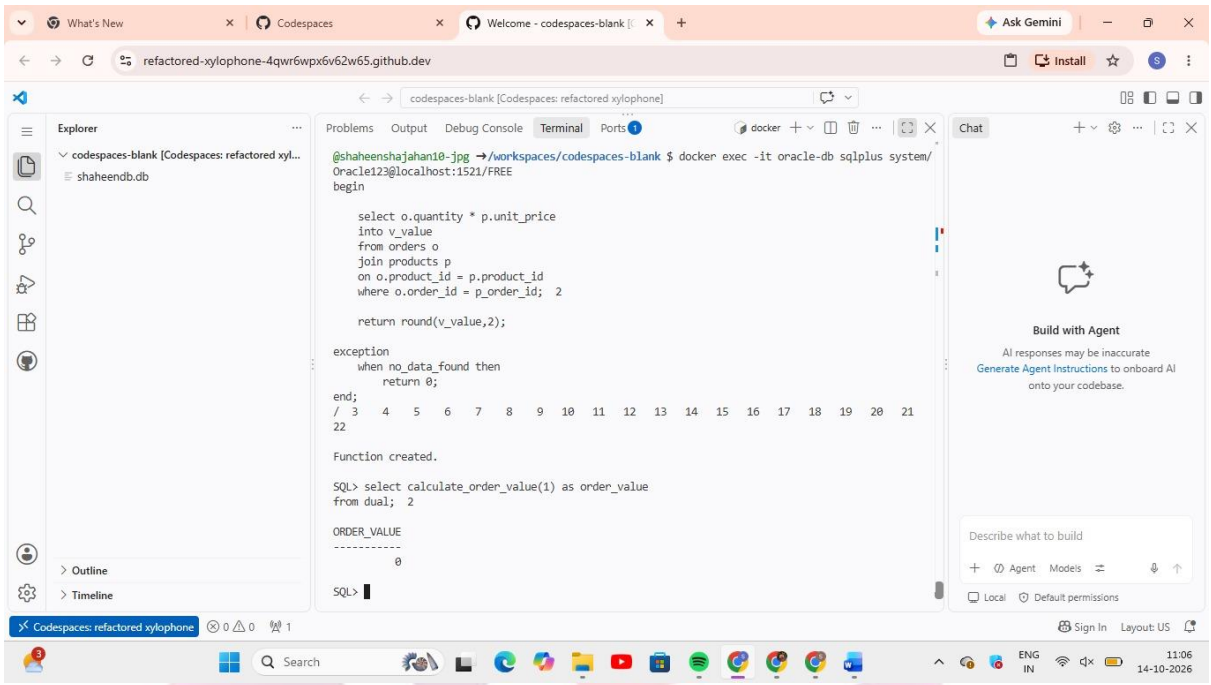
OUTPUT:



CALCULATE ORDER VALUE

```
create or replace function calculate_order_value(
    p_order_id number
)
return number
is
    v_value number;
begin
    select o.quantity * p.unit_price
    into v_value
    from orders o
    join products p
    on o.product_id = p.product_id
    where o.order_id = p_order_id;
    return round(v_value,2);
exception
    when no_data_found then
        return 0;
end;
/
select calculate_order_value(1) as order_value
from dual;
```

OUTPUT:



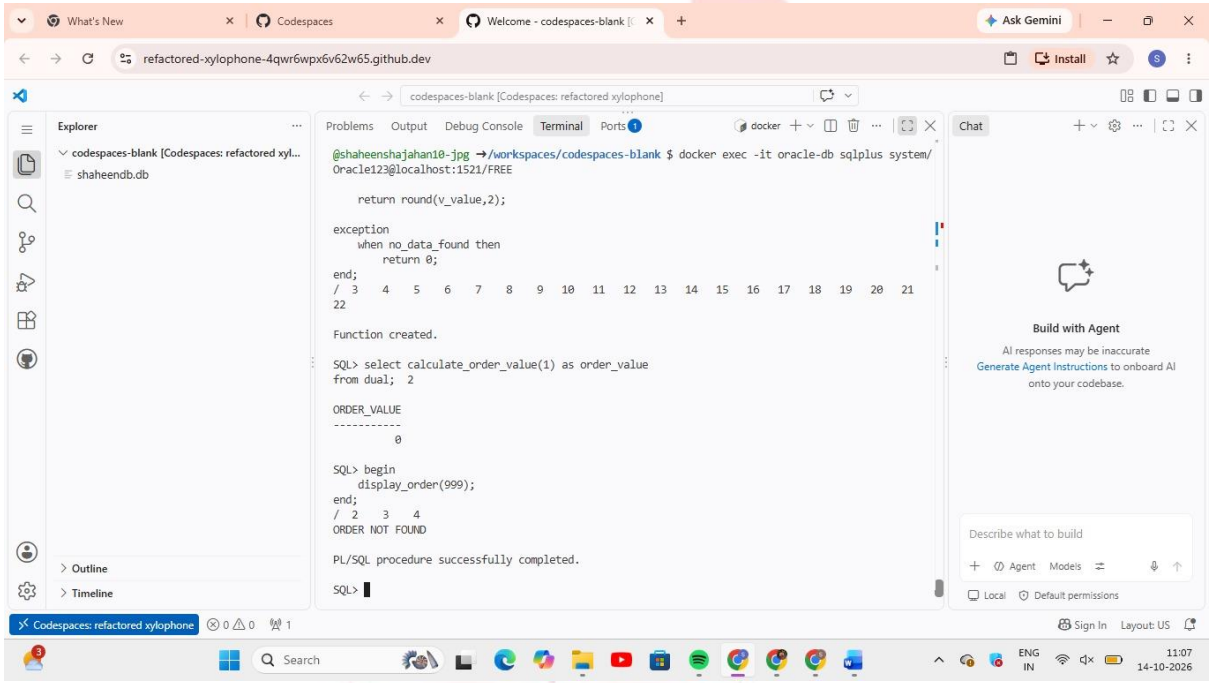
TRUTH-TOIL-TRIUMPH
AUTONOMOUS

EXCEPTION HANDLING

Missing Order

```
begin
    display_order(999);
end;
/
```

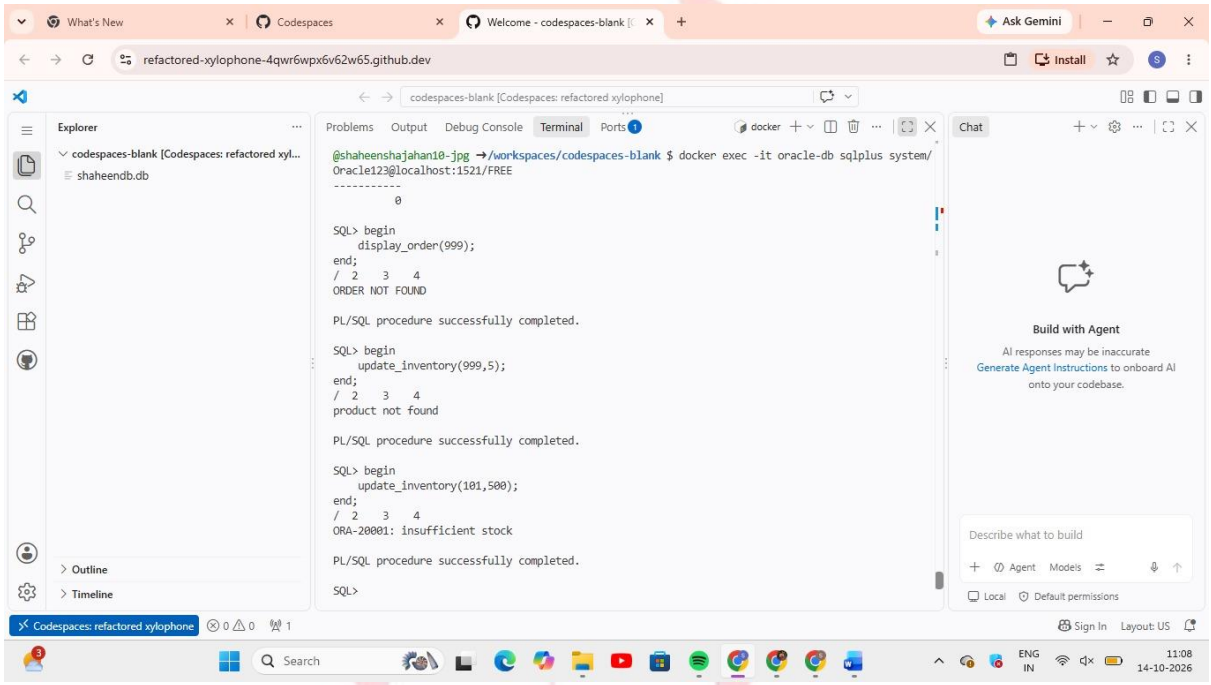
OUTPUT:



Missing Product

```
begin
   update_inventory(999,5);
end;
/
```

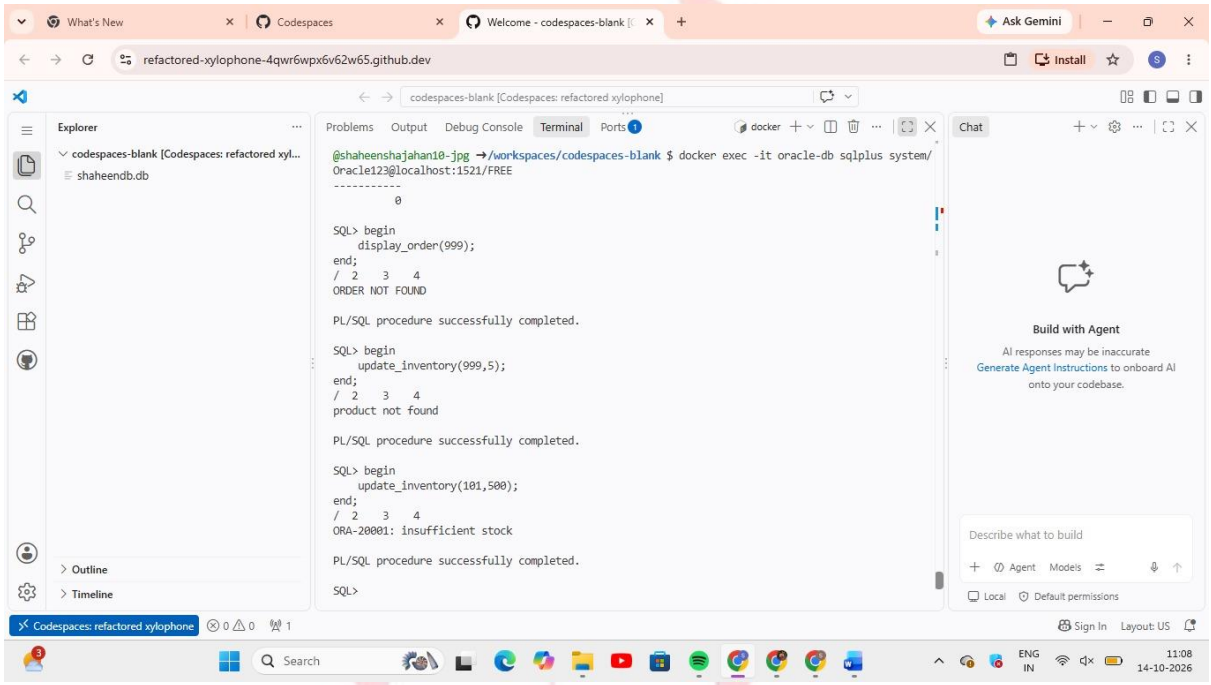
OUTPUT:



Insufficient Stock

```
begin
   update_inventory(101,500);
end;
/
```

OUTPUT:



Dataset Insights

- The dataset contains 25 order records.
- It includes 10 products and 5 customers.
- Products are divided into different categories.
- Product prices and stock quantities are maintained.
- Total order values can be calculated using SQL.
- Customer-wise sales can be analyzed.
- Low-stock products can be identified.
- PL/SQL programs handle missing records and invalid inputs.

RESULT

Thus, the E-Commerce Order and Inventory Management System was developed using SQL and PL/SQL. The system stores product, customer, and order information, calculates order values, analyzes sales, generates inventory reports, and handles missing records through exception handling.

